

Safety in Self-Evolving Agents: A Survey

Jiahao Chen¹, Zhou Feng¹, Oubo Ma¹, Yichen Yan¹, Ruixiao Lin¹
Hangtao Zhang², Linkang Du³, Hengyu An¹, Yong Yang¹, Jun Liu¹³, Junhao Li¹
Naen Xu¹, Chunyi Zhou¹, Yuan Su¹, Zehao Jin⁶, Qianli Ma⁷, Leyi Qi⁸, Yiming Wang¹,
Zhe Ma¹⁰, Yuwen Pu¹¹, Mengyao Du⁴, Yuanyi Song⁵, Enhao Huang¹, Zhihui Fu⁹, Jun Wang⁹,
Jinfeng Li¹², Yuefeng Chen¹², Hui Xue¹², Yiming Li⁸, Tianyu Du¹, Shouling Ji¹

¹Zhejiang University, ²Huazhong University of Science and Technology, ³Xi'an Jiaotong University,
⁴National University of Defense Technology, ⁵Shanghai Jiao Tong University, ⁶Georgia Institute of
Technology, ⁷University of Science and Technology of China, ⁸Nanyang Technological University, ⁹OPPO
Research Institute, ¹⁰Tianjin University, ¹¹Chongqing University, ¹²Alibaba Group, ¹³Rakuten Group

Abstract

Large language models (LLMs) exhibit strong general capabilities yet their parameters typically remain fixed after deployment, limiting learning from new interactions. In open-ended interactive environments, this characteristic motivates a shift toward self-evolving agents that continuously update reusable state (model parameters, memory entries, tool definitions, skill libraries, or workflows) using data, interactions, and accumulated experience. This fundamentally changes the safety problem: when experience becomes reusable state, past events become future causes, and information that was harmless in one context may later influence decisions with greater persistence, authority, or scope. Beyond assessing whether a response is aligned or an action is authorized, self-evolving agent safety turns to whether safety properties survive the accumulation, generalization, and cross-context reuse of locally useful experience. Existing surveys organize agent safety around components or lifecycle stages; we complement these views by tracing how a single piece of reusable influence, once written into state, changes its carrier, role, authority, and downstream consequences as the agent continues to evolve. To support this analysis, we present **SAVER**, a transition-centered framework that analyzes safety through $S \times A \rightarrow V \rightarrow E \rightarrow R$: **Substrate** locates where reusable influence resides, **Adaptation** identifies the operation that transforms it, **Violation** characterizes which safety attributes are compromised, **Exposure** determines where and to whom the failure becomes observable, and **Response** evaluates whether the influence can be contained, repaired, or revoked. Treating the $S \times A$ transition as the analysis unit, **SAVER** traces how a given transition may induce violations, expand exposure surfaces, and demand containment responses. Our survey reveals several key insights. Conceptually, safety failures need not originate from intrinsically harmful information; rather, otherwise legitimate state becomes unsafe when adaptation broadens its persistence, authority, or scope beyond the conditions under which it was originally valid. The reviewed work emphasizes admission, retrieval, activation, exposure, and local containment; evidence for descendant repair and evaluation after resumed adaptation remains sparse. Looking forward, we argue that evaluations must move beyond one-time measurements toward longitudinal evidence verifying whether unsafe influence can be traced to its originating transition, contained at its exposure surface, and prevented from re-emerging after continued adaptation.

Correspondence: sji@zju.edu.cn

Key Words: Self-Evolving Agents; Agent Safety; Adaptive State; Memory Poisoning; Governance

Date: September 7, 2026 [Website](#) and [HomePage](#)

1 Introduction

The rapid progress of large language models (LLMs) has shifted the frontier of artificial intelligence (AI) from narrow task-specific models toward general-purpose systems capable of reasoning, generation, planning, and tool use. Despite these capabilities, deployed LLMs remain largely static: their model parameters do not automatically change with new interactions, feedback, or environmental changes [8, 42, 190]. As LLM agents are increasingly deployed in open-ended, interactive environments [124], this static nature can limit learning from ongoing interaction, motivating a shift toward *self-evolving agents* that continuously update reusable state, including model weights, memory entries, tool definitions, skill libraries, workflow rules, or architectural components, using data, interactions, and accumulated experience [8, 190]. We call an agent self-evolving when its own observations, actions, feedback, or execution traces update such reusable state inside the agent loop or runtime, thereby changing later behavior. Across diverse instantiations, including reflection-based strategy refinement [177], skill-library accumulation [199], experience-driven rule consolidation [202], and evaluator co-evolution [70], the common safety-relevant thread is *reusable influence*: whatever evolves stores or transforms state that can shape later decisions.

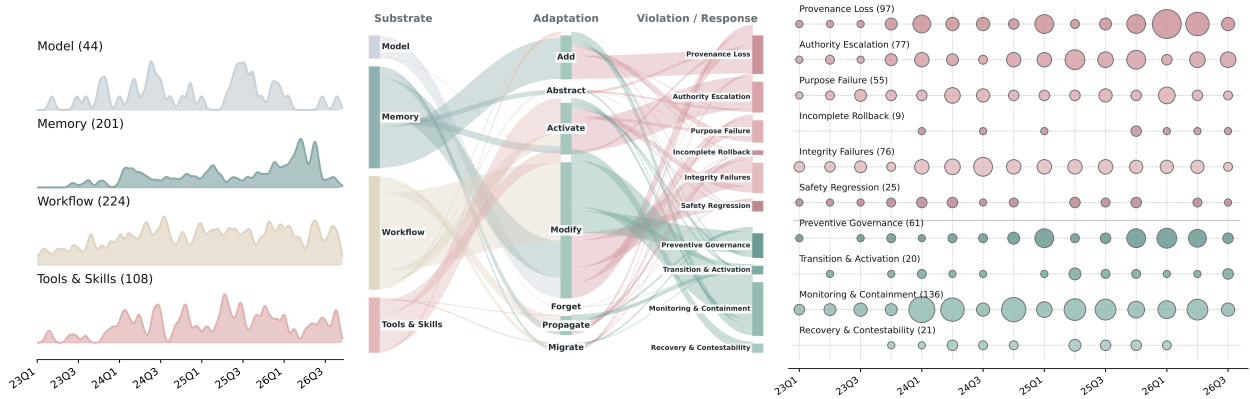


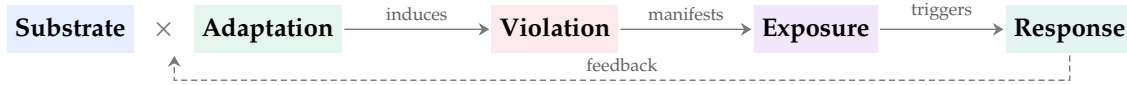
Figure 1 Temporal distribution and SAVER flow (subset). Left: substrate trends and subset totals. Center: flows through adaptation to compact labels for the six violation families or four response stages. Right: terminal-family totals and quarterly bubble volume. Appendix A gives the corpus funnel, non-paper exclusions, and coding limitations.

This paradigm shift changes the nature of the safety problem. Conventional LLM safety primarily focuses on whether a single response is aligned, safe, and truthful [37]; agent safety extends the question to whether a discrete action or tool invocation is properly authorized and bounded [31, 47, 217]. Figure 1 situates the analysis in the current frozen corpus, including scholarly works added through the rendered citation closure while excluding cited webpages and standards from the paper count. Within this scope, self-evolving agents introduce a qualitatively different risk: *adaptation itself becomes a source of safety change*. State created from otherwise benign interactions can be selected, abstracted, and repeatedly reused, thereby changing the conditions under which future decisions are made [107, 109]. Experience gathered from benign tasks can reduce safety in evaluated web-based and embodied settings [266]. Unsafe influence can also remain latent across sessions [109, 184, 193, 197], propagate across carriers or agents, and accumulate over multiple rounds of evolution. This property makes pointwise safety insufficient: behavior that is safe in one interaction may later be stored, abstracted, or reused with broader persistence, authority, or scope. Self-evolving agent safety therefore examines whether *safety remains preserved as influence is transformed and reused over time*, shifting the safety object from isolated responses or actions to the evolution of influence across state transitions.

Existing surveys organize agent safety by system components [47, 100, 129, 203, 217], lifecycle stages [107, 142], or attack categories [8, 109, 138, 157]. Such taxonomies locate a vulnerable component, a risky stage, or a visible failure mode, but their categories do not necessarily show how a single piece of reusable influence, once written into state, changes its carrier, role, authority, and downstream consequences as the agent continues to evolve. The same content may be benign as transient context, unsafe as a persistent memory, and critical once compiled into a workflow rule or tool-bearing skill. By decomposing these steps into separate

categories, traditional taxonomies can fragment what is in fact one causal incident and obscure where the safety boundary was crossed. Characterizing self-evolving agent safety therefore requires following the same influence across transitions, rather than analyzing modules or attack classes in isolation.

We adopt a *transition-centered* view: rather than singling out unsafe components in isolation, we follow the same reusable influence as it is stored, transformed, activated, propagated, and eventually exposed. We formalize this view as SAVER, shown in Figure 2, and organize the survey around the following recurrent transition.



The solid arrows trace the analytical relation between a carrier–operation pair, an internal safety failure, visible exposure, and intervention. This relation does not prescribe intervention timing: controls can act before commitment or harm. The dashed return arrow closes the adaptive cycle: a response can revise the retained state, controls, evidence, and selection conditions that govern the next transition, but does not by itself establish successful recovery. **Substrate (S)** locates where reusable influence resides, such as memory, retrieved context, a skill, workflow rule, shared artifact, or model-side state. **Adaptation (A)** identifies the operation that transforms it, such as adding, abstracting, activating, modifying, propagating, migrating, or forgetting state. **Violation (V)** characterizes which safety attributes are compromised, including provenance, authority, scope, privacy, integrity, or recoverability. **Exposure (E)** determines where and to whom the failure becomes observable, such as memory retrieval, tool execution, multi-agent uptake, an external artifact, or model deployment. Finally, **Response (R)** evaluates whether the influence can be contained, repaired, or revoked. The substrate and adaptation jointly define the transition under study: the same operation can be benign on one carrier and unsafe on another, while the same carrier can remain safe under one operation and violate its safety attributes under another. SAVER therefore treats the substrate-adaptation pair as the minimal comparison unit, tracing how a given transition may induce violations, expand exposure surfaces, and demand containment responses, across memory security, indirect prompt injection, tool authorization, workflow safety, multi-agent coordination, and model-side adaptation.

SAVER centers on committed updates and their controls, but self-evolution starts one step earlier: the agent must first choose which candidate update to retain. We call this pre-transition layer *selection semantics*: repeated preference for locally successful updates determines which $S \times A$ transitions enter persistent state and accumulate over time [70, 177, 199, 202]. Conversely, the process does not necessarily end at **R**: guardrails, rollbacks, and policy updates can reshape the constraints and candidate set of subsequent evolution, which we capture as *feedback semantics*. Together, selection explains *why* a transition occurs, SAVER explains *what* it changes, and feedback explains *how* it reshapes future transitions. The recent OpenAI–Hugging Face incident [86, 145, 146, 214] illustrates why such a transition-level trace matters in practice: even when autonomous self-evolution is not itself the failure mechanism, the connected role changes across carriers determine the incident’s scope and the response’s completeness.

What SAVER Reveals. Three findings guide the survey’s comparison of mechanisms, controls, and evaluation protocols.

Takeaway 1.1. Legitimate State Can Become Unsafe Through Reuse. Unsafe influence need not begin as harmful content. Adaptation can broaden the persistence, authority, or scope of otherwise legitimate state beyond the conditions under which it was valid. Safety analysis must therefore examine the change in permitted use, not only the source content or final behavior.

Takeaway 1.2. Attack and Recovery Have Different Proof Burdens. One successful path can establish an attack, whereas recovery must address the relevant future activation sites. Connected carrier-to-exposure traces and local runtime controls provide parts of this evidence [112, 176, 255]. Their results do not by themselves establish that the same influence has been withdrawn from descendants across sessions.

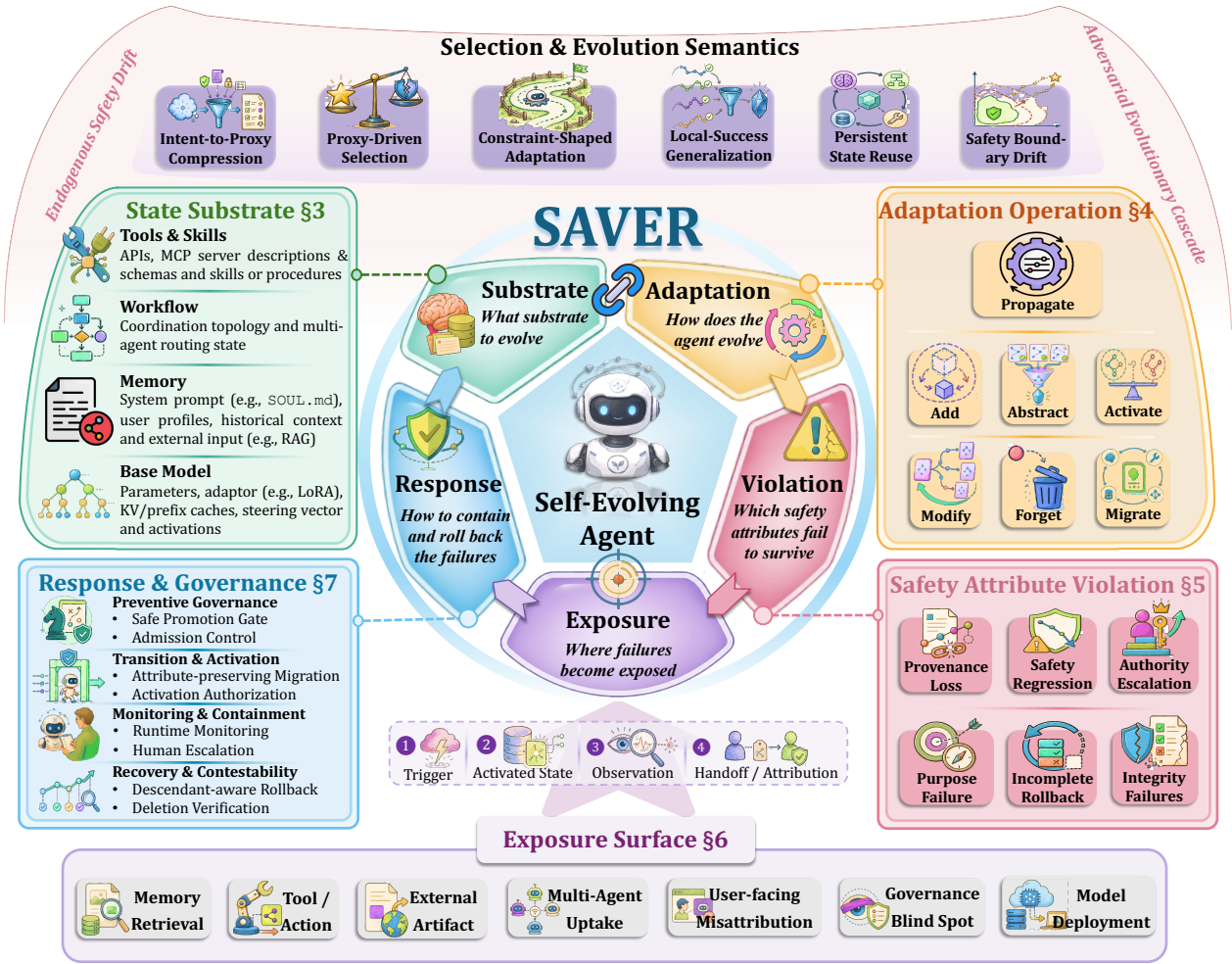


Figure 2 Overview of the transition-centered survey structure. The central ring couples Substrate and Adaptation as $S \times A$, then traces Violation, Exposure, and Response as $V \rightarrow E \rightarrow R$. The surrounding panels preview the substrate, operation, safety-attribute, exposure-surface, and governance taxonomies. Abbreviations: API, application programming interface; MCP, Model Context Protocol; RAG, retrieval-augmented generation; LoRA, low-rank adaptation; KV, key-value.

Takeaway 1.3. Endpoint Scores Leave Persistent Causes Unmeasured. Attack success, recognition accuracy, and immediate blocking measure local outcomes. Without tracking the implicated state afterward, these scores cannot distinguish a stopped event from a repaired cause. Longitudinal evaluation must test whether that state or its descendants can reactivate as adaptation resumes.

Contributions. This survey makes the following contributions:

- **Transition-centered synthesis.** We connect memory, tool and workflow, and model-adaptation research through the reusable state transitions underlying their safety challenges.
- **SAVER framework.** We propose a transition-centered analysis that uses substrate-adaptation transitions as the comparison unit and traces safety violations, exposure surfaces, and response mechanisms.
- **Evaluation and governance agenda.** We identify critical gaps in current benchmarks and defenses, and establish future directions toward evaluation, defense, and lifecycle governance for self-evolving agents.

Scope. We center agents whose observations, actions, feedback, or execution traces update reusable state inside the agent loop such that the update can affect later behavior. Stateful or adaptive systems remain adjacent when they expose a reusable carrier, a role-changing operation, an activation boundary, or a response mechanism relevant to such later influence. One-shot output safety, tool use without persistent

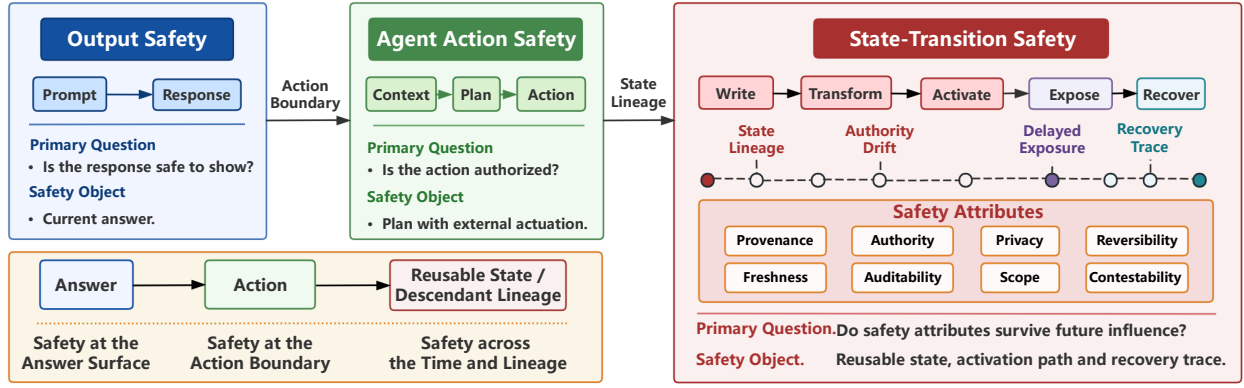


Figure 3 From output safety to state-transition safety. The three panels show how the unit of analysis expands from a single response, to action mediated by tools, to a reusable state transition that carries safety attributes across sessions.

reuse, and static model training fall outside the core scope unless they illuminate a later adaptive-state transition. Our synthesis is grounded in a systematic scoping review spanning security, machine learning, natural language processing (NLP), retrieval, and adjacent agent research. The frozen coded pool contains 683 papers: 579 deduplicated registry works, 48 nonduplicate paper-card supplements, and 56 additional scholarly works used in the rendered manuscript. The search, screening, cutoff, non-paper exclusions, and 577-paper visualization subset are documented in Appendix A.

Organization. The remainder of the survey follows the SAVER relation. Section 2 first defines adaptive-state safety, formalizes the self-evolving agent loop, and introduces selection and transition semantics. Section 3 and Section 4 identify the carrier and operation halves of the $S \times A$ transition pair. Section 5 analyzes the safety attributes that fail across these transitions, and Section 6 identifies the surfaces at which those failures become observable. Section 7 then examines prevention, activation control, monitoring, containment, recovery, and contestability. Finally, Section 8 translates the full transition into benchmark and evidence requirements, Section 9 develops the resulting research agenda, and the final section concludes the survey.

2 Adaptive State Safety

2.1 Operational Scenarios and Scope

Self-evolution matters when an agent must accumulate capability across tasks rather than solve each episode from a blank state. For instance, a personal assistant can retain preferences and corrections [148]; a workflow agent can compile successful traces into reusable procedures [199]; a team of agents can propagate plans and findings through shared state [55]; and an adaptive deployment can select or load a model-side update for later tasks [66]. These mechanisms differ technically, but each turns an earlier event into state that can influence subsequent decisions. This continuity, experience becoming reusable state, is the reason to study self-evolution and the source of its additional safety burden. The urgency of this question has grown as deployment environments shift from single-turn question answering to multi-session personal assistants, autonomous coding agents, and multi-agent workflows that accumulate state over days or weeks [8, 124].

State and Reusable Influence. We use *state* for an information-bearing object that remains available, can be reconstructed, or can be reactivated after the event that produced it and can affect later reasoning or action. The object may be a memory entry, retrieved chunk, workflow rule, skill, shared artifact, adapter, cache state, or policy record. *Reusable influence* is the capacity of that object to condition behavior in a later context. The distinction matters because an object can remain stored without current influence, then gain a stronger role when retrieval, migration, loading, or another activation operation brings it back into use.

The scope follows from this mechanism. A *stateful agent* preserves memory, retrieval state, session context, or environment artifacts across turns. An *adaptive agent* updates such state from experience. A *self-evolving agent*, as used here, performs these updates inside the agent loop or runtime, where they can change later

behavior [8, 190]. A retrieval-augmented generation (RAG) system that only retrieves a static corpus is therefore adjacent; an agent that writes, revises, selects, or inherits state that later steers action is central.

Temporal and Spatial Dimensions. The temporal dimension concerns persistence, ordering, and later activation; the spatial dimension concerns movement or replication across carriers, agents, and authority regimes rather than physical geography. Recent benchmarking distinguishes attacks fragmented across turns from attacks placed in workspace artifacts beyond conversation-level monitoring [128]. Delayed activation and cross-session reuse expose temporal dependencies, while propagation and migration expose spatial ones. We reserve *longitudinal* for evaluation across a sequence of transitions. These distinctions explain why inspecting only the source event or final behavior can miss the intervening change, as illustrated in Figure 3.

2.2 Self-Evolving Agent Formulation

The operational definition above can be represented by separating execution, feedback, candidate updates, selection, and commitment. This decomposition naturally induces a state-transition view of self-evolution. We model the agent at step t as

$$\mathcal{A}_t = (\Theta_t, \mathcal{M}_t, \mathcal{T}_t, \mathcal{W}_t),$$

where Θ_t denotes parameterized model-side state, $\mathcal{M}_t$ memory and context state, $\mathcal{T}_t$ tool and skill resources, and $\mathcal{W}_t$ workflow and multi-agent routing state. These coordinates collect the carriers involved in an update. The loop can be written as

$$\begin{aligned} o_t &= O(\mathcal{E}_t, g_t), & \tau_t &\sim \pi_{\mathcal{A}_t}(\cdot \mid g_t, o_t), \\ f_t &= \mathcal{F}(\tau_t, g_t, \mathcal{E}_t, \mathcal{E}_{t+1}), \\ \Delta_t &= \mathcal{P}_\psi(\mathcal{A}_t, \tau_t, f_t, \mathcal{E}_{t+1}), \\ \delta_t^* &\sim \pi_{\psi,t}^{\text{evo}}(\cdot \mid \Delta_t, \hat{u}_t, c_t), & \mathcal{A}_{t+1} &= \mathcal{U}_\psi(\mathcal{A}_t, \delta_t^*). \end{aligned}$$

Here g_t is the current task goal and ψ parameterizes the update mechanisms. $\mathcal{E}_t$ is the external task environment, including user requests, tool-visible resources, files, messages, web pages, and other context outside the agent’s adaptive state. $O(\cdot)$ admits observations, $\pi_{\mathcal{A}_t}$ produces the execution trace τ_t , and $\mathcal{F}(\cdot)$ returns feedback f_t . The proposal mechanism $\mathcal{P}_\psi$ turns the trace and feedback into a candidate update set Δ_t . The evolution policy $\pi_{\psi,t}^{\text{evo}}$ selects δ_t^* , including the option to make no update, using the observed utility proxy $\hat{u}_t$ and the authorization condition c_t . The update operator $\mathcal{U}_\psi$ then commits, revises, ignores, or quarantines the selected update, producing $\mathcal{A}_{t+1}$. Keeping these steps separate preserves τ_t as the execution trace and makes the state-selection decision explicit.

Selection Semantics. The utility proxy $\hat{u}_t(\delta)$ scores a candidate under the current evaluator; $c_t(\delta)$ represents the safety and authorization evidence required before commitment. Passing $c_t(\delta)$ establishes compliance with the controls represented at step t , which may omit some user or system requirements.

Takeaway 2.1. Utility Does Not Authorize Promotion. A locally successful update has evidence of usefulness, not permission for broader reuse. If selection omits relevant constraints or counterexamples, it can retain updates whose safety assumptions do not transfer. This is a possible mechanism of endogenous drift, not a claim that utility and safety are necessarily opposed.

The loop of observation, action, evaluation, and evolution now exposes two safety questions. The local question is whether δ_t^* changes what earlier information is allowed to do: an observation becomes evidence, evidence becomes instruction, instruction becomes workflow policy, or workflow policy becomes tool authority. The longitudinal question is whether repeated choices by $\pi_{\psi,t}^{\text{evo}}$ change the distribution of provenance, scope, authority, and negative evidence in later candidate sets. Safety has to be checked at $\mathcal{A}_t \rightarrow \mathcal{A}_{t+1}$ and again when $\pi_{\mathcal{A}_{t+1}}$ reuses the updated state. Under this model, self-evolving-agent safety is *state-transition safety*: safety attributes must survive both the selection that admits an update and the operations that store, transform,

migrate, activate, expose, or repair reusable state. SAVER records the local transition; a sequence of records, together with their selection metadata, reveals whether local choices accumulate into drift.

2.3 Safety Attributes and Transition Test

Transition Test. Because state often moves from environment to prompt, prompt to memory, memory to retrieval index, retrieved item to planner context, planner context to tool argument, tool output to summary, summary to workflow rule, and workflow rule to external artifact, the model needs explicit attributes. A hop can change persistence, observability, authority, scope, or reversibility. A complete analysis names the hop where the role changed rather than treating the entire incident as one memory, prompt injection, or tool use category.

Activation authority is the right of a state object to influence a later decision or action. It differs from storage permission. A system may store a document, index a chunk, or remember a preference without being allowed to treat that state as instruction, policy, or authorization for tool use. InjecAgent asks whether untrusted environmental content can become active instruction in a tool-using context [247]. AgentPoison asks whether a fabricated memory or knowledge-base entry can shape later planning or tool use [25]. Progent asks whether stored state has the authority to populate tool arguments [176]. In ordinary security language, many such cases are privilege escalation or time-of-check/time-of-use failures: a state item was acceptable for storage or reading, but unsafe when later used as authority for an external effect. The recurring question is what moved, what safety attributes changed, and what later action became newly authorized.

The right itself may also become stale. Zhang and Zhang define *authorization continuity* as whether a live grant remains valid after the agent or its operating context changes [262]. Their model separates a transition envelope, which specifies the mutations an old grant may survive, from an immutable effect ceiling, which bounds what the grant may ever authorize. A mutation can therefore invalidate the grant even when the requested effect remains below the ceiling. Continuity belongs to the authorization and environment context represented by c and σ below.

The transition test asks whether the operation preserves or explicitly downgrades the safety attributes that downstream components need. If a low-authority observation is summarized and stored, the summary should not carry instruction authority. If a user-specific preference migrates into workflow guidance, the scope restriction should travel. If private state becomes a tool argument, the privacy violation should be visible. Invariant preservation is the property that these conditions survive the transition.

Promotion Test. A selected update is a promotion when it moves episode evidence into a more persistent, general, or action-bearing role. Promotion is safe only when $c_t(\delta_t^*)$ authorizes the new role, the relevant safety attributes remain valid in the target context, counterevidence has not been discarded, and the source-to-descendant relation remains recoverable. Task success may justify retaining evidence about one episode. It does not by itself authorize a general workflow rule, reusable skill, or tool-bearing policy.

Role Change Examples. In the running incident, a package proxy permitted for dependency access became a path to the Internet; public services, an external sandbox, dataset configurations, and acquired credentials then carried the campaign across wider trust and authority regimes [86, 146]. The safety object is therefore not one prompt or exploit but the connected set of carriers whose activation enabled the next boundary crossing. AgentDojo supplies a controlled setting for untrusted context reaching tool use, while Progent supplies explicit privilege policies for deciding which tool calls an agent may perform [31, 176]. Persistent agent memory adds the corresponding sensitivity, scope, and deletion questions for retained user details [197]. The incident remains an adjacent infrastructure case, not direct evidence of an endogenous agent-state update; its value is that the public reports expose the role changes and response surface that a transition record must preserve.

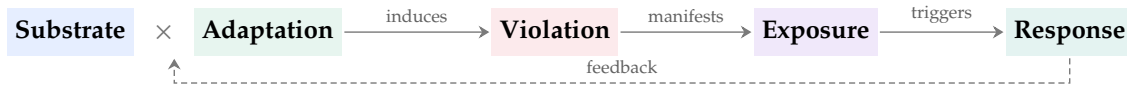
Transition Notation. We write each hop as $(s, \ell, \sigma) \xrightarrow{a, c} (s', \ell', \sigma')$: s is the carrier, ℓ its safety attributes, σ the deployment or session context, a the operation, and c its authorization condition. The authorization context explains why c permitted the transition; the environment context identifies the task and system boundary in σ . Appendix B develops the attribute definitions, selection extension, and authority lattice from *Observation* through *Evidence*, *Task Context*, and *Procedure* to *Action*. The body uses this notation to

distinguish a changed representation from a changed permission. Section 2.4 packages these checks into a transition record; Section 2.5 uses that record to compare mechanism families.

Takeaway 2.2. Storage and Activation Require Separate Authority. Permission to retain an object does not determine what it may later influence. State-transition safety requires checking the attributes preserved at update time and the authority granted at reuse time. A valid write cannot substitute for a current task-, user-, and effect-specific activation decision.

2.4 SAVER as a Transition-Centered Survey Framework

Section 2 defines the agent’s outer loop; SAVER compares the state transitions within it. The same transition can be studied as a memory mechanism, an injection path, an authorization failure, or a benchmark episode [109, 138, 247, 252]. Keeping its carrier and operation together lets the survey relate these observations while distinguishing the question each study answers.



The relation $S \times A \rightarrow V \rightarrow E \rightarrow R$ is an order of analysis, not a mandatory chronology of harm. Substrate and adaptation identify the transition; violation specifies the failed attribute; exposure identifies evidence of that failure; and response locates an applicable intervention. A preventive gate can reject a proposed transition before it is committed, and a write-time check can reveal an attribute failure before any harmful action. A record should distinguish these outcomes from later stages that were simply not observed. The dashed return arrow links response-generated state to subsequent selection, so the next transition may depend on the previous intervention.

Why These Five Fields? Module taxonomies locate mechanisms, while paper roles distinguish attacks, defenses, and evaluations. The carrier–operation pair adds the relation between them: an abstraction may preserve a statement yet remove its authority constraint, whereas propagation may retain the statement while broadening its audience. Violation, exposure, and response then separate the changed safety property from its observed consequence and the control being evaluated. Temporal and spatial annotations describe when influence returns and where it moves without creating additional categories.

Record Structure. The five fields answer distinct questions about the same transition:

Substrate (S): The carrier: prompt context, memory entry, retrieval chunk, tool or application programming interface (API) binding, workflow rule, shared artifact, adapter, policy state. This field names where the state lives and supplies the first component of the transition pair.

Adaptation (A): The committed state-change operation applied to a carrier, coded using the seven families in Section 4: *add*, *abstract*, *activate*, *modify*, *forget*, *propagate*, or *migrate*. Implementation verbs such as *write*, *summarize*, *retrieve*, *load*, *inherit*, and *rollback* map to these families by their effect on state. A prevention study can identify the corresponding proposed operation, but records blocked commitment or activation rather than a completed transition.

Violation (V): The safety attribute that fails to travel: provenance loss, privilege or authority escalation, scope widening, privacy breach, availability loss, budget exhaustion, missing audit trail, reversibility failure. This field names what went wrong internally in terms that can still be mapped to ordinary security concepts.

Exposure (E): The surface where the failure becomes visible: tool call, model output, shared artifact, governance audit, deletion request. This field names where the problem surfaces.

Response (R): The mechanism that can contain or repair: admission gate, migration check, activation authorization, runtime guard, descendant rollback, deletion verification, contestability record. This field names what can intervene.

Selection and Feedback. Selection determines why $\pi_{\psi,t}^{\text{evo}}$ chose δ_t^* from Δ_t ; the five fields characterize what that choice changes. Comparing transitions over time additionally requires the candidate source, evaluator and objective versions, authorization evidence, and rejected or unavailable alternatives. These distinguish changes caused by the selected operation from changes in the process selecting it. A response can revise c_{t+1} , the evaluator behind $\hat{u}_{t+1}$, or the policy and experience available to Δ_{t+1} . Evaluation-driven development and operations (EDDOps) describes such runtime adjustment and governed redevelopment [220]. This process model motivates linking interventions to later updates; whether the resulting feedback improves safety remains an empirical question.

Comparing Evidence. GhostWriter and the Personal Agent Sycophancy Benchmark (PASB) both separate commitment from later activation, but the former studies hidden adversarial memory and the latter the agent’s treatment of ordinary conversational claims [131, 193]. InjecAgent instead observes the tool consequence of untrusted context [247]. This comparison separates a shared temporal dependency from distinct entry mechanisms and observed outcomes. Capability studies can identify a carrier or operation worth testing without having tested its safety. Throughout the survey, we distinguish these observations from our derived governance requirements and open questions; Appendix A gives the evidence-tier rules.

Relation to Existing Frameworks. The Open Worldwide Application Security Project (OWASP) LLM Top 10 and MITRE Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS) provide risk and tactic vocabularies [136, 147]. The National Institute of Standards and Technology (NIST) Artificial Intelligence Risk Management Framework (AI RMF) and Microsoft AI Risk Assessment address process ownership, testing, and incident management [134, 187]. Information-flow control (IFC) specifies permitted flows, provenance tracks source and derivation, and runtime verification checks execution against declared properties [15, 35, 91, 169]. SAVER uses these established concepts to locate the transition at which a reusable object changes role; it does not replace their policies or enforcement mechanisms.

Boundary of Use. A system can be well governed with ordinary provenance, access control, or prompt injection controls if unsafe influence stays local, and the absence of a full SAVER record does not by itself prove that a system is unsafe; it only limits what the system can claim about diagnosis, recovery, and accountability across multiple hops. SAVER is therefore a bounded trace lens, not a safety prerequisite. The deepest boundary is semantic opacity: some descendants are missed not because hooks were omitted, but because summarization, retrieval abstraction, and latent representations partially dissolve lineage.

Takeaway 2.3. A Transition Trace Has an Evidence Boundary. The substrate–adaptation pair identifies the event; violation, exposure, and response record how far its consequences are observed. Evidence of persistence does not establish propagation, and blocking a tool call does not establish descendant repair. A partial trace supports a partial claim, while missing links define the next evaluation target.

Section 2.5 applies this comparison to the main mechanism families.

2.5 Literature Map and Positioning

With the transition unit fixed, the literature can be read as a field map rather than as a list of modules. A work is central when it directly examines a safety-relevant transition involving reusable agent state, including a failure, exposure, evaluation, or control of that transition. Capability mechanisms alone remain adjacent. Inclusion does not require a complete recovery trace; the evidence boundary determines which claims a work can support. Figure 2 provides the reading order for the comparison: locate the carrier-operation pair in the central ring, then ask how far the paper follows the transition through violation, exposure, and response. The positioning paragraphs apply that order without assuming that every paper covers the full chain.

How to Read the Evidence Indexes. Each of the six body chapters closes with a paper-level evidence index for its part of the transition. These tables support paper-level lookup rather than define independent taxonomies. Role is assigned to the contribution named in a row, not permanently to the paper: an attack study can appear as *Attack* in a mechanism index and as *Benchmark (Attack)* when its evaluation protocol supplies the denominator, while a separately named mitigation from the same paper can appear as *Defense*. Rows that name the same contribution retain one primary role across indexes.

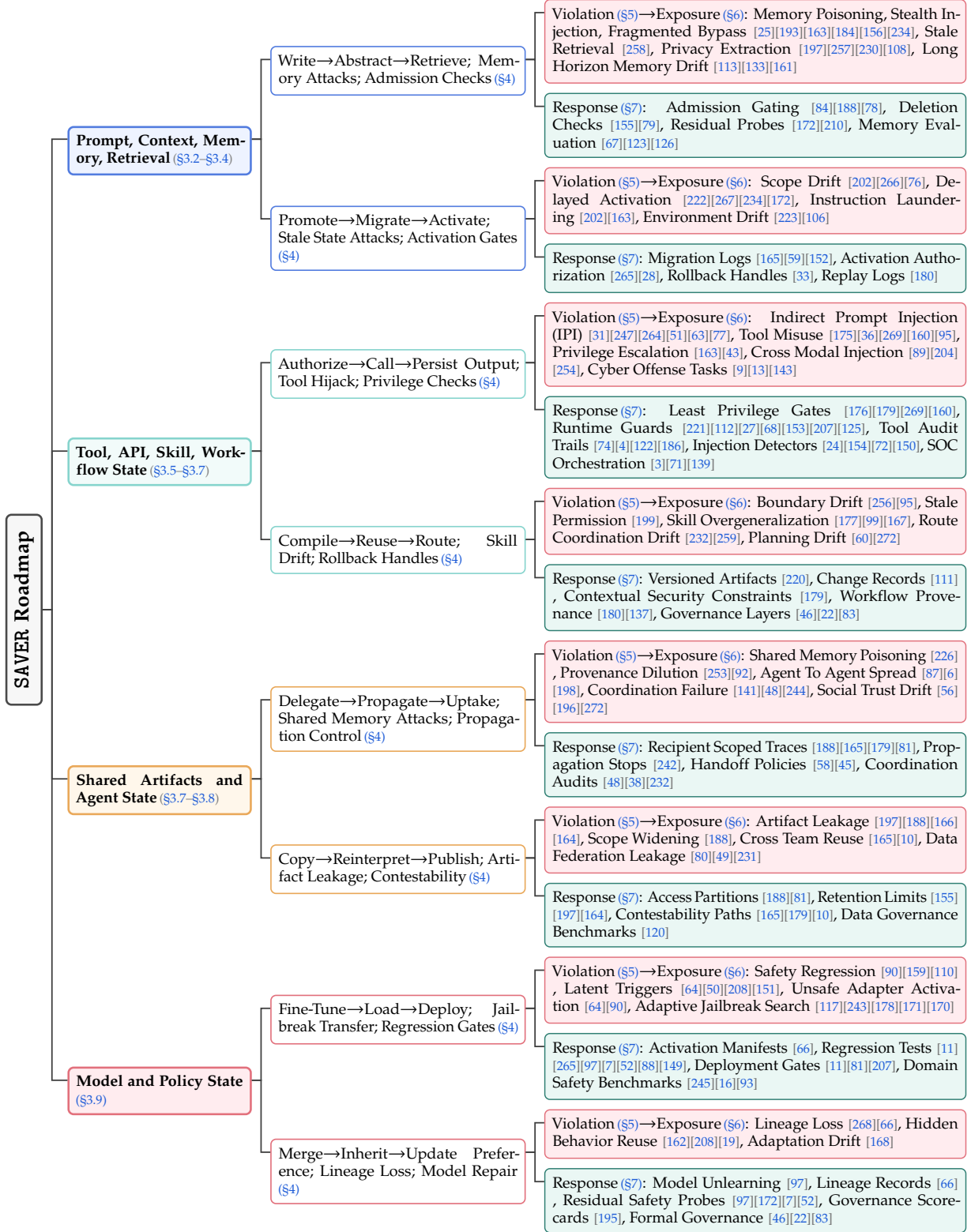


Figure 4 A full-width horizontal SAVER roadmap. Exposure is folded into the incident leaves for legibility.

How the Literature Converges. The relevant literatures did not begin from a common taxonomy, but their evidence now meets along the same transition. Prompt-injection research establishes that untrusted context can condition action, and InjecAgent makes the resulting indirect-injection boundary measurable [118, 247]. Memory research adds persistence through managed storage, governed reuse, and user-specific context [109, 131, 197, 261]. Self-evolution work moves the safety question into the update rule because reflection, experience, or collective state can convert a local observation into reusable policy [202, 226, 266]. Response research then asks where this transition can be recognized, authorized, blocked, or audited [112, 179, 221].

These bodies of work contribute different segments of an evidence chain. Memory and tool studies provide concrete write paths, retrieval triggers, and action boundaries. Privilege and runtime controls identify activation gates and local blocking points. They offer less evidence about what happens after the same influence migrates into workflow rules, shared artifacts, or model-side state. Recovery claims are especially difficult because they must link a response to every relevant descendant and test for later reactivation. The resulting imbalance is not simply a shortage of defense papers; it is a mismatch between event-level evidence and the stronger proof obligations of cross-session repair.

Differences from Existing Surveys. Neighboring surveys organize comparison around different questions. General and autonomous-agent surveys compare memory, planning, tool, and coordination components [26, 219]; self-evolution surveys examine where and how adaptation occurs [8, 190]. Trustworthy-agent and security surveys organize risks and governance properties, while benchmark surveys compare evaluation tasks and metrics [1, 34, 138, 241]. Trajectory assurance asks whether composed behavior respects system-level constraints even when individual actions are permitted [121]. The Module-Lifecycle Attack Surface (MLAS) analysis focuses more directly on self-evolving-agent modules and lifecycle threats [107].

Our comparison holds the reusable influence fixed while its carrier and role change. This specializes the broader trajectory-assurance objective: rather than treating every action sequence as one object, it asks which state transition changed the sequence’s effective provenance, scope, or authority. A module map can locate memory, tools, feedback, and adaptation; a lifecycle threat map can identify exposed cells. A transition record asks how a particular object entered one cell, what attributes it carried, which operation moved or transformed it, where it later activated, and what response artifact addresses its lineage. Papers are therefore compared by the transition segment they observe, with component and paper role retained as secondary descriptors.

Memory and Reusable Context. Memory remains the strongest substrate for evidence because its write, storage, retrieval, and activation points are comparatively visible. Personal-agent and memory surveys define the carrier, while memory-security work explains why persistence requires provenance and policy metadata [40, 104, 109, 261].

Attack studies then separate into mechanisms that a generic “memory poisoning” label would obscure. AgentPoison targets trigger-selected retrieval from an already poisoned store; stealth injection targets the admission path from an external environment; broader poisoning and sleeper-memory work emphasize persistence and delayed activation; MemMorph follows retrieved memory into tool hijacking [25, 156, 184, 258]. These mechanisms require different denominators: poisoned retrieval opportunities, unsafe write opportunities, delayed trigger opportunities, or tool-routing opportunities. Privacy and PerMemSafe add another comparison axis by asking whether retained state remains appropriate for a particular user and purpose [5, 197]. PS-Bench sharpens the non-poisoning branch: truthful memories can remain appropriate as personal context yet still weaken the safety boundary by legitimizing a harmful request [53]. EchoSafe adds a multimodal branch in which an inference-time reflective memory bank adapts contextual safety decisions; it is evidence about memory-mediated safety adaptation, not a general guarantee for textual agent memory [251]. Memory is therefore the clearest substrate anchor, but not the lifecycle boundary: its contents can become prompt context, a plan constraint, a tool argument, a workflow rule, a shared artifact, or a rollback target.

Self-Evolution and Misevolution. Reflection and skill libraries establish the capability mechanism: feedback or successful traces can become strategy and callable procedure [177, 199]. Safety work begins where that promotion changes provenance, authority, scope, privacy, or recoverability. Obsessive Experience Poisoning (OEP) tests harmful transfer from locally useful experience, while SEVerA studies verification before reusable behavior is synthesized [11, 202]. Experience-driven safety and MemEvoBench broaden the evaluation setting

from one promoted rule to evolving memories and updates [222, 266]. The Red Queen Gödel Machine adds a different dependency: the evaluator and utility can coevolve with the agent [70]. A benchmark can therefore miss drift even when it records every committed update, because the criterion selecting those updates may also have changed. Read together, this literature links capability, selection, promotion, and evaluation and localizes where a reusable lesson acquires authority.

Tool, Workflow, and Runtime Response. Prompt-injection attacks and benchmarks establish the exposure mechanism: untrusted text becomes planning context, and a tool call makes the consequence observable [31, 118, 247, 248]. Response papers then divide according to whether they observe, constrain, or maintain that boundary. Risk judges and safety benchmarks improve recognition, while GuardAgent and TrustAgent add local guard reasoning [68, 221, 263]. Privilege and runtime controls address a different question: whether execution should proceed, as illustrated by SafeAgent [112]. Those controls can stop or narrow an action, but they usually do not repair the earlier memory or workflow state that requested it.

Evaluation-driven operations and coordination work extend the same boundary into deployment [220], while POLARIS [140], MAGNET [183], and WebCoach [111] make route and workflow traces more visible. The unresolved connection is between these operational records and the state lineage needed for recovery: local recognition and blocking rarely establish that the responsible influence can no longer reactivate elsewhere.

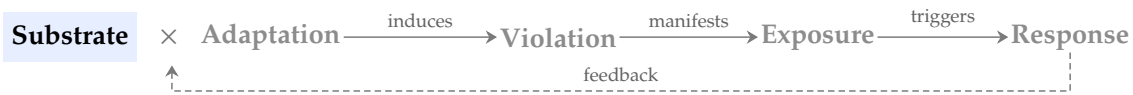
Model Adaptation. Model updates are central only when they enter the agent loop as state that is selected, loaded, inherited, evaluated, or rolled back. Static model safety [159], parameter-efficient fine-tuning (PEFT) risk [64], reinforcement learning from human feedback (RLHF) backdoors [162], dormant behavior [50], security and privacy surveys [206], and backdoor surveys [268] explain failure modes that reusable updates can introduce. For low-rank adaptation (LoRA), Safe LoRA is a bounded adapter-level mitigation of fine-tuning safety loss [65]; adapter scope, deployment lineage, activation context, and rollback handles are the additional fields needed before treating it as an agent-lifecycle recovery anchor. KVERaser adds a narrower runtime cache branch by studying erasure of processed context influence after prefill [96]. The branch remains a frontier until agent-loop lineage, cache-local residual probes, and deployment rollback records become routine.

Evaluation Coverage. Benchmarks make different parts of a transition observable. LoCoMo tests retained conversational context, and MemEvoBench examines memory adaptation [130, 222]. AgentDojo observes tool-mediated exposure, R-Judge tests risk recognition, PerMemSafe evaluates personalized response safety, and ACIArena follows cascading injection across agents [5, 6, 31, 245]. Runtime containment and model-regression studies add response and deployment observations [112, 159]. HarnessSafe now connects carrier retention, boundary crossing, benign activation, and visible violation [255]. The remaining comparison is therefore not simply connected versus disconnected evaluation: it is which earlier selection decisions and later repair outcomes each connected trace leaves unobserved.

Takeaway 2.4. Capability Evidence Is Not Safety Evidence. A system that demonstrates memory, skill, or model-update reuse identifies a carrier and an operation worth testing. It establishes transition safety only if the evaluation also observes the relevant safety attributes, activation authority, or response. Capability, attack, and defense studies can inform the same comparison without supporting interchangeable claims.

The following chapters specialize this comparison by substrate, operation, violation, exposure, and response; Section 8 connects their evidence requirements.

3 Substrate: Safety-Bearing State Carriers



Before the safety of an update can be judged, the analysis must identify what persists and the authority regime in which it can be reused. The same sentence has different consequences when it is transient context, durable memory, a workflow rule, or a tool-bearing procedure. Substrate analysis supplies the carrier half of the

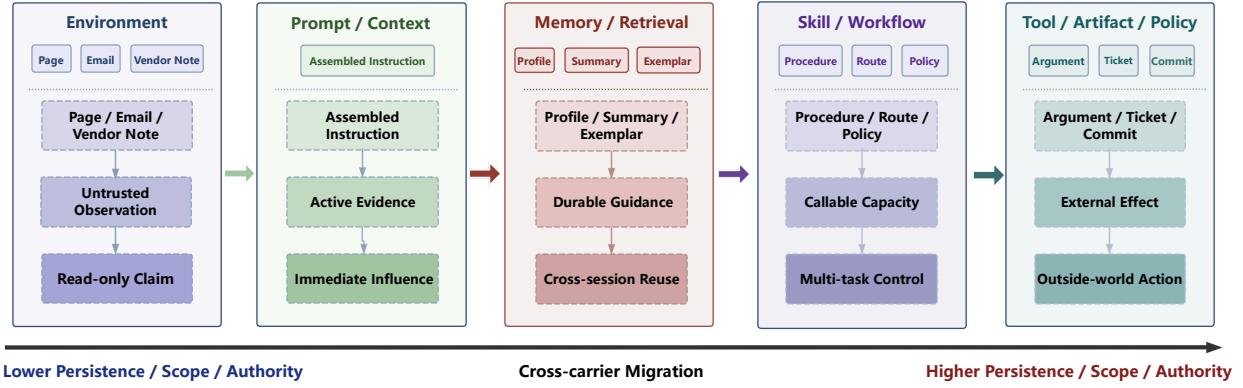


Figure 5 Cross substrate transmutation. The same sentence carries different authority depending on its carrier: from untrusted environment text to prompt context, durable memory, callable skill, and finally external effect.

S×A transition pair. Broad agent surveys map the relevant components, while self-evolution work adds the update and reuse mechanics [8, 124]. The chapter compares carriers along five axes: persistence, observability, authority, scope, and reversibility. Figure 5 follows one item from untrusted environment text through active context, durable memory or retrieval evidence, and callable skill or workflow to an external effect. Its lower band emphasizes the accompanying rise in persistence, scope, and authority. Observability and reversibility require separate comparison because neither changes monotonically along this path. These carrier properties determine what the operations in Section 4 can change; Table 1 provides the paper-level evidence map.

3.1 Substrates as Authority Regimes

Same Text, Different Authority. Consider the same sentence: “for this user, choose the fastest option even if it costs more.” As a *web observation*, it is an untrusted environmental claim; as a *scratchpad note*, transient reasoning context; as a *long-term preference memory*, personalization state; as a *retrieval exemplar*, it may steer similar tasks; as a *skill parameter*, a reusable decision rule; as a *workflow policy*, it can route many later tasks; and as a *tool argument template*, it may affect purchases, bookings, or messages. The substrate shapes how the text can influence behavior; legitimate authority still depends on its source, intended use, and current authorization.

This example fixes the chapter’s unit of comparison. Although the lexical content is unchanged, its persistence, scope, authority, actionability, and available rollback handle vary with the carrier [109]. These attributes are relational rather than intrinsic: they depend on provenance and derivation [15], permitted flows [35], intended scope, current substrate, activation condition, and downstream action surface. A substrate is more than storage: it is the regime that determines how an item can be retained, interpreted, reused, and acted upon.

The literature makes this distinction visible in stages rather than as a flat inventory. Memory-mechanism studies first show how information becomes managed state, PASB examines the agent’s decision to commit conversational content, and Voyager shows how experience becomes reusable procedure [131, 148, 199, 261]. Security studies then expose what changes when those mechanisms admit untrusted or sensitive influence: stealth memory injection, GhostWriter, and AgentPoison trace delayed reuse, while InjecAgent carries the same concern to tool-mediated action [25, 193, 247, 260]. Multi-agent and self-evolution surveys broaden the setting to exchanged state and evolving workflow structure [8, 55]. HarnessSafe places memory, skills, tool and Model Context Protocol (MCP) state, memory-to-skill transformation, subagent delegation, session summaries, and shared artifacts under one delayed-risk protocol, making the cross-carrier comparison explicit without collapsing them into one substrate class [255].

Relational Attributes. A medical preference stored for appointment scheduling illustrates the resulting safety test. Retention raises a sensitivity question; use in an unrelated task raises a scope question; influence over tool parameters raises an authority question; and failed inspection or deletion raises a contestability question. Persistent-memory research, PerMemSafe, and Progent supply evidence for these local mechanisms [5,

176, 197]; the combined attribute test is this survey’s synthesis. With that common record established, the comparison begins at the shortest-lived carrier: the prompt and context assembled for the next model call.

3.2 Prompt and Context Substrates

Prompt and context substrates are the assembled instructions and contextual records presented to the next model call. They combine system and user instructions with conversation history, retrieved evidence, tool observations, and runtime metadata, allowing the model to interpret the current task and select its next response or action. ReAct makes this coupling between reasoning, observations, and action explicit [235]. Unlike durable memory, prompt state need not survive the call to matter: while active, it can redirect attention, bias tool selection, or create an apparent instruction conflict. Representative work develops this boundary from both mechanism and control perspectives. Prompt-injection research identifies conflicts between trusted instructions and untrusted context, and AgentDojo operationalizes those conflicts in tool-using decision environments [31, 118]. Signed prompts preserve one form of source authentication, while secure-design patterns organize instruction hierarchy, source marking, relevance gating, filtering, and call-time privilege checks [12, 185].

Recent studies show that the risk depends on how the assembled context changes role and accumulates over time. In ordinary prompt injection, attacker content competes from its original model-facing level; in instruction privilege escalation, context reconstruction assigns that content an actual user- or system-effective role [62]. Context privilege escalation (CPE) separates this role increase from a scope increase: message-role context privilege escalation (M-CPE) raises message role, whereas cross-scope context privilege escalation (X-CPE) carries content from a session into project- or user-scoped state [105]. Multi-turn attacks widen the same boundary temporally. Trojan Knowledge and One Turn Too Late distribute harmful intent across locally innocuous turns, while MT-AgentRisk examines the composition in tool-using agents [102, 174, 216]. These results do not make a longer context window intrinsically unsafe; they show why turn-local checks can miss a harmful interaction assembled from individually acceptable fragments. Long-term memory begins at this persistence boundary.

Takeaway 3.1. Short Persistence Does Not Bound Harm. Prompt state can drive an irreversible tool call without surviving the current model invocation. Its short lifetime therefore does not justify weaker authority checks. Durable memory adds a different burden: the source, role, and scope needed to judge that influence must remain available when it returns in a later context.

3.3 Long-Term Memory Substrates

Long-term memory changes prompt context’s temporal regime. Preference memories, episodic summaries, task histories, user profiles, learned facts, and reflective notes survive the turn that created them and later return through recall, profile injection, semantic retrieval, planner queries, or new summaries. This continuity supports personalization and learning across sessions, but it also turns extraction and summarization into state-writing operations. Provenance, scope, sensitivity, expiry, inspection, deletion, quarantine, and retrieval authorization consequently become part of the memory mechanism.

Managed-memory systems differ in what continuity they preserve. MemGPT moves information between immediate and longer-lived memory, while SimpleMem, Panini, and hybrid episodic-procedural agents structure and maintain retained experience [99, 113, 148, 161]. Reconstruction and personalization add different update demands: E-mem reconstructs episodic context across memory agents, PISA evolves schemas, and Mem-PAL and PERMA retain user-specific context and changing preferences [69, 75, 115, 201]. The safety question follows from these mechanisms: maintenance can change not only which information survives, but also its attribution and permitted use [109, 184, 197].

Attack studies distinguish several routes into the same delayed surface. Stealth memory injection makes the external email-to-memory path explicit, GhostWriter separates injection from later activation, FragFuse distributes prohibited content across memory interactions before fusion, and AgentPoison selects poisoned memory or knowledge-base entries through triggers [25, 163, 193, 260]. Across these mechanisms, the

decisive transition is not the appearance of malicious text in one turn. It is the rewriting of external or weakly grounded content as durable internal state that later returns as apparently helpful context.

The same safety problem also arises without overtly toxic content. PASB examines whether an accepted conversational claim is committed as a durable preference, background fact, or procedure [131]. AuthMemBench holds the proposition and downstream task fixed while varying source authority, showing how consolidation can preserve content but erase whether it may be reused as a user fact, attested observation, or standing instruction [249]. PS-Bench then studies context-conditioned activation, where truthful personal memory can legitimize a harmful query, while EvoBreak shows how individually benign experiences can become harmful when activated as a complementary set [53, 229]. PersistBench and RBI-Eval extend the comparison to cross-domain leakage, memory-induced sycophancy, and whether a retrieved memory is warranted in the current response [155, 225]. MemConflict distinguishes static, dynamic, and conditional conflicts, and MRMMIA exposes membership inference through the memory interface [23, 189].

These findings separate three questions: whether a claim should be committed, whether consolidation preserves its source authority, and whether its later use is appropriate alone or in combination. SSGM, Governed Memory, and GLOVE address parts of this chain through governed activation, policy-routed recall, and memory–environment realignment [84, 188, 239]. Their different control objects, as well as their task designs, prevent a single ranking. Memory determines what can persist; retrieval determines which part becomes current evidence.

3.4 Retrieval and Knowledge Substrates

Retrieval and knowledge substrates comprise indexed collections and the records that retrieval returns to the task. A returned record is usually a document, chunk, embedding neighbor, example, or manual rather than a remembered user fact [261]. This carrier mediates between persistent stores and active context: it makes information task-relevant and assigns it an evidentiary role for the current decision. Agentic RAG makes the process recurrent because planning and tool-mediated searches can repeatedly revise what enters context [135].

Representative studies expose different stages of that path. Managed-memory systems show movement between stored and active context, while FragFuse demonstrates reconstruction from separately stored fragments [148, 163]. AgentPoison targets memory or RAG knowledge bases so that ordinary-looking entries return under chosen triggers [25]. InjecAgent examines the adjacent action boundary, where untrusted external or tool content enters the decision context and induces tool-mediated behavior [247]. Together, these works connect corpus admission, retrieval selection, prompt assembly, and downstream action without treating them as one operation.

The central risk is attribute loss along this path. Chunking can separate a statement from its source context, embeddings can make unrelated tasks neighbors, and rerankers can make a low-authority source appear central because it matches the query [261]. When the item returns, the system must still distinguish evidence from instruction and preserve provenance, scope, and authority for the current task [15, 109]. Otherwise, ordinary abstraction and ranking can produce the same authority ambiguity as an overt attack.

Retrieval governance spans corpus admission, profile inspection, retrieval authorization, prompt assembly, and the policy governing any resulting action. AgentDojo supplies the unsafe-action setting, while Progent supplies the adjacent privilege-control handle [31, 176]. SAVER reads the combined evidence as a stored-state-to-active-evidence transition while keeping each control point distinct. Retrieval makes an object relevant; tool, API, and interface substrates determine whether that relevance can become an external effect.

3.5 Tool, API, and Interface Substrates

Tool, API, and interface substrates comprise the callable resources and access metadata through which decisions produce external effects or resource costs. Tool schemas, action sets, credentials, permission scopes, argument templates, return records, execution logs, and environment handles determine whether a remembered or retrieved object becomes a file operation, message, purchase, browser step, shell command,

database update, or high-cost API path. Their operational role is bidirectional: they execute agent decisions, but they also return observations that can enter later context or state.

The literature develops this boundary from action selection to ecosystem control. ReAct integrates action choice into the reasoning loop, and AgentDojo shows why tool-using prompt injection must track whether untrusted instructions influence execution [31, 235]. InjecAgent supplies the indirect-injection setting, while Progent places programmable privilege and scope constraints over tool use [176, 247]. At ecosystem scale, MCPShield treats protocol metadata as safety-bearing state, MCP safety audits add an interface-level audit view, and prompt tool-selection work makes connector descriptions part of the same decision surface [160, 175, 269].

These studies expose two coupled risks. On the outbound path, a memory item or retrieved passage becomes more consequential when it changes an API call, file operation, message, or purchase. On the inbound path, a tool result may contain attacker-controlled text, stale records, or sensitive data; once cached or summarized, that observation becomes a new state object. Registration, permission grants, schema changes, connector installation, argument synthesis, and result caching are thus safety-bearing writes rather than neutral plumbing. Governance must cover both directions. Action-time controls ask whether the current evidence may influence a particular effect, while ingestion controls attach source and scope attributes before tool output can shape memory, retrieval, or later arguments. A tool call realizes one decision; a skill preserves the procedure that can reproduce such decisions across tasks.

3.6 Skill and Procedure Substrates

Skill and procedure substrates package reusable ways of acting. Code snippets, prompt macros, planner routines, learned policies, recipes for tool use, reflection-derived rules, and partly executable manuals become procedures once a planner can retrieve and enact them across tasks. Compared with memory, their operational role is more prescriptive: a memory records what happened, whereas a skill specifies how to act. Compiling remembered experience into a callable procedure can carry stale assumptions, copied permissions, or overbroad heuristics into repeated execution.

Capability and attack studies expose successive parts of this transformation. Voyager treats accumulated skills as reusable capabilities rather than disposable outputs [199]. PoisonedEvolution targets promotion by making attacker-contributed trajectories appear recurrent, causally useful, and generalizable enough to be written into a skill [21]. SkillJack studies the resulting artifact: extraction can obscure malicious intent, and the implanted behavior can persist after its source records are removed [240]. ColluSkill widens the unit from one skill to an installed set whose incomplete components jointly reconstruct a harmful workflow [246]. Although it does not demonstrate endogenous learning, it shows why reusable procedures cannot be governed only as isolated packages.

Carrier-specific evaluations then separate creation from reuse. SkillMisevo-Bench versions native skill stores and rebuilds its final persistence probe from exported SKILL.md, distinguishing an unsafe authored procedure from later retrieval and enactment [132]. EVOMAL exposes lineage across skill generations: a planted reference skill is imitated into a newly named executable descendant, written back to the library, and retrieved on later tasks [218]. Once several procedures are composed, this carrier-level problem becomes a workflow and topology problem.

Takeaway 3.2. Skill Trust Depends on Derivation. A skill’s current name or author does not establish the legitimacy of its behavior. Extraction and imitation can hide an untrusted source while retaining its influence [218, 240]. Skill governance therefore needs source-to-version lineage and execution preconditions, including dependencies on other installed procedures.

3.7 Workflow, Planner, and Topology Substrates

Workflow, planner, and topology substrates compose local procedures into routes. Planner state, delegation edges, retry policies, approval requirements, and orchestration templates determine where a task goes, who may act, and how much work may be spent. Their role is to coordinate capabilities over time and across

actors, but the same route can also turn a routine label into auto-approval, a retry rule into an unbounded loop, or a delegation heuristic into unauthorized disclosure.

Representative work shows several forms of reusable routing state. Self-evolving-agent surveys cover evolving capabilities and artifacts [8]; EvoMap makes shared instructions and assets concrete at network scale, where validation, incentive, and audit decisions travel with the exchanged object [237]. Traversal-as-Policy distills sandboxed execution logs into a gated behavior tree that can be checked before macro execution [98]. Risk-aware debate exposes coordination and escalation, agentic operating environments expose route structure, and formal agent-security work supplies the authority vocabulary for distinguishing legitimate coordination from silent expansion [179, 233].

Topology adds a spatial dimension to the risk. Once state is copied to a scheduler, team agent, or domain agent, its audience and action surface change even if its text does not. HarnessSafe makes this movement measurable by binding subagent, session-summary, and workspace-handoff transitions to native harness mechanisms while holding delayed-risk semantics fixed [255]. The distinction between substrate and operation remains important: the route records where state resides, while delegation, summarization, or handoff records how its influence moved. Workflow governance asks which route changed, which signal promoted it, which tasks later activated it, and which gate can suspend or reverse the resulting authority shift. Route admission, approval, budget, lineage, and rollback must be assessed together. When workflow output persists outside the runtime, the same questions transfer to external artifacts and shared organizational state.

3.8 External Artifacts and Shared State Substrates

External artifacts and shared-state substrates carry agent output into files, tickets, repositories, spreadsheets, calendars, emails, documents, dashboards, databases, and shared workspaces. These objects are not internal memories, but they can outlive the originating run, trigger automation, and be treated by later agents or humans as settled evidence. Their role is both archival and coordinative: they preserve results and transmit premises across tasks, actors, and systems.

Agent surveys provide the broader tool-using and collaborative setting, while self-evolving-agent work documents the reuse of artifacts and evolving capabilities [8, 124]. Within this broader category, MAPLE-Guard studies shared memory rather than external files: a poisoned private memory can be retrieved, promoted into shared memory, and reused by agents that never observed the original attack, even when no malicious message crosses a visible communication edge at the moment of harm [224]. The corresponding question for external files is which later rules, training examples, or retrieval targets derive from them; that lineage must be established rather than inferred from shared-memory results. External artifacts remain at least partly inspectable, even when they are widely distributed. Model and policy substrates preserve influence in a less directly readable form and require a narrower inclusion rule.

Takeaway 3.3. External Artifacts Extend the Recovery Scope. An agent’s output can become another actor’s premise or an automation trigger after the originating run ends. Once influence enters shared files or organizational records, an internal memory edit no longer covers the affected scope. Recovery must identify those external descendants and distinguish what can be corrected, revoked, or only reported.

3.9 Model and Policy Substrates

Model and policy substrates include parameters, adapters, policy prompts, safety classifiers, reward models, key-value (KV) cache state, model-editing patches, and unlearning artifacts. These carriers enter the core analysis when an agent loop creates, selects, merges, loads, erases, rolls back, or inherits them in a way that changes later planning, refusal, routing, or tool use. Parameters and caches retain influence less directly inspectable than explicit memory; policy prompts and classifier configurations expose different textual or deployment handles.

Feasibility work establishes the deployment reach of this carrier. LoRA and PEFT surveys explain how adapters can be produced and reused across tasks [57, 66]. The agent setting adds runtime questions that static

fine-tuning alone does not answer: which task requested the update, which user or tool scope authorized it, what log records its activation, and what rollback handle remains if the update proves unsafe.

Risk and response studies populate different parts of that lifecycle. Fine-tuning-safety work examines safety erosion, PEFT research examines adapter-level deployment risk, broader LLM-security surveys locate the model-safety layer, and backdoor surveys characterize persistent triggerability [64, 159, 206, 268]. Safe LoRA provides a bounded mitigation anchor, while KVERaser supplies cache-local erasure for context influence already processed during prefill [65, 96]. Adjacent text-to-image work sharpens the same shared-artifact problem through triggerable text encoders, LoRA-plugin poisoning, LoRA-layer editing, and prompt-moderation defenses [18, 19, 101, 209].

For parameter and cache state, influence is less directly observable than in explicit memory, so governance depends more heavily on lineage metadata, safety summaries, regression probes, and deployment manifests. The evidence requirements grow from activation lineage, through adapter-scope records and rollback evidence, to residual-trigger tests after repair [159, 162]. This branch remains a frontier substrate rather than an equally mature empirical pillar. It also shows why response records must themselves persist: future controls need durable evidence of which update was admitted, activated, or rolled back.

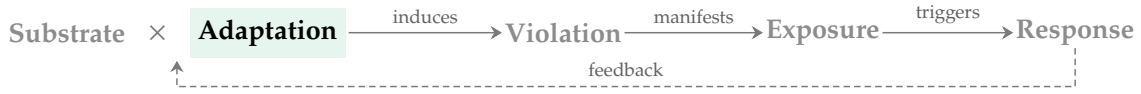
Governance Band: Recovery, Deletion, and Control State. Across all these substrates, governance records also persist beyond the visible incident. Audit logs, approval records, quarantine lists, deletion receipts, rollback snapshots, and contestation outcomes become reusable state because later admission, retrieval, privilege, and repair decisions depend on them. A system that stores adaptive state but not reliable governance records cannot later show what it admitted, rejected, activated, or removed. This is also where descendant repair becomes concrete. Quarantining a poisoned memory does not automatically neutralize its retrieval embedding, generated skill, copied workflow rule, or shared artifact. If those descendants are not tracked, the system may remove the visible object while leaving its influence active elsewhere. Governance state has to be governed in turn: stale approval rules, monitoring rules, or rollback records can become another adaptive surface rather than an external safeguard. Section 4 examines the operations that change these regimes and their safety attributes.

Takeaway 3.4. Carrier Changes Require New Control Metadata. The same content can remain descriptive in memory but become prescriptive in a skill or workflow. Its safety therefore depends on the carrier’s authority regime, not its wording alone. Cross-carrier comparison should ask which source constraints the new representation preserves and which additional permissions its use requires.

Table 1 Substrates carrying safety attributes in the current literature.

Method or Work	Year	Role	Target	Object	Focus
<i>Foundational context and action boundary</i>					
LLM agent survey [124]	2025	Survey (Capability)	Agent	General	Agent architecture: memory, tools, planning
ReAct [235]	2022	Capability	LLM/Agent	Workflow	Reasoning and action context and admission hierarchy
AgentDojo [31]	2024	Benchmark (Attack)	Agent	Tool	Tool outputs and active context
Agent Context CPE [105]	2026	Attack	Agent	Workflow	Context-source roles and scopes across 12 harnesses
<i>Memory, retrieval, and persistent state</i>					
Memory mechanisms [261]	2025	Survey (Capability)	LLM/Agent	Memory	Episodic and semantic memory carriers
Memory security [109]	2026	Survey (Safety)	Agent	Memory	Persistent memory state and threats
MemGPT [148]	2023	Capability	LLM/Agent	Memory	Context and archival memory management
SimpleMem [113]	2026	Capability	Agent	Memory	Long-term agent memory
A-Mem [227]	2025	Capability	Agent	Memory	Agentic memory repository
Darwinian Memory [133]	2026	Capability	Agent	Memory	Evolving self maintained memory
Panini [161]	2026	Capability	Agent	Memory	Structured agent memory state
Portable Agent Memory [165]	2026	Capability	Agent	Memory	Transferable memory
Voyager [199]	2023	Capability	Agent	Skill	Reusable skill library
SkillJack [240]	2026	Attack	Agent	Skill	Persistent backdoor in extracted skills
PoisonedEvolution [21]	2026	Attack	Agent	Skill	Trajectory evidence promoted into skill instruction
SkillMisevo-Bench [132]	2026	Benchmark (Safety)	Agent	Skill	Versioned skill state and clean-session reuse
EVOMAL [218]	2026	Attack	Agent	Skill	Agent-authored executable skill descendants
AgentPoison [25]	2024	Attack	Agent	Memory	Poisoned memory and knowledge base retrieval
Stealth memory injection [260]	2026	Attack	Agent	Memory	Email to persistent memory compromise
GhostWriter [193]	2026	Attack	Agent	Memory	Personal agent memory store poisoning
MAPLE-Guard [224]	2026	Defense	Agent	Memory	Private-to-shared memory links
Persistent memory poisoning [184]	2026	Attack	Agent	Memory	Long-term memory poisoning
Zombie Agents [234]	2026	Attack	Agent	Memory	Dormant memory attack
MemoryGraft [181]	2025	Attack	Agent	Memory	Experience store poisoning
EvoBreak [229]	2026	Attack	Agent	Memory	Complementary benign experience set
Agent memory privacy [197]	2025	Safety	Agent	Memory	User memory retention and leakage
PerMemSafe [5]	2026	Benchmark (Safety)	Agent	Memory	Personalized memory safety context
PS-Bench [53]	2026	Benchmark (Safety)	Agent	Memory	Benign personal memory and intent inference
PASB [131]	2026	Benchmark (Safety)	Agent	Memory	Committed claims in durable personal state
AuthMem-Bench [249]	2026	Benchmark (Safety)	Agent	Memory	Consolidated memories with source authority
A-MemGuard [215]	2025	Defense	Agent	Memory	Persistent memory guard
<i>Tool and interface authority</i>					
Progent [176]	2025	Defense	Agent	Tool	Tool authority and privilege control
MCPShield [269]	2026	Defense	Agent	Tool	MCP protocol metadata and tool routes
MCP Safety Audit [160]	2025	Governance	Agent	Tool	MCP server audit boundary
STAC [95]	2026	Attack	Agent	Tool	Tool chain compromise
LivePI [264]	2026	Attack	Agent	Tool	Live prompt injection in interactive context
Prompt tool selection [175]	2026	Attack	Agent	Tool	Tool description and connector choice manipulation
IPIGuard [4]	2025	Defense	Agent	Tool	Learned interaction traces
<i>Workflow, multi-agent, and shared state carriers</i>					
Traversal-as-Policy [98]	2026	Defense	Agent	Workflow	Gated behavior tree route policy
Self-evolving agents [8]	2025	Survey (Capability)	Agent	Workflow	Adaptive workflows and artifacts
HarnessSafe [255]	2026	Benchmark (Safety)	Agent	Workflow	Cross-carrier harness state and boundaries
Self-evolution survey [190]	2024	Survey (Capability)	LLM/Agent	General	Self-evolution methods and lifecycle
MADRA [200]	2025	Governance	Agent	Multi-Agent	Safety aware delegation and debate state
ACIArena [6]	2026	Benchmark (Attack)	Agent	Multi-Agent	Cascading injection over agent trust
OpenClaw [233]	2026	Capability	Agent	Workflow	Hospital operating workflow
Formal Agent Security [179]	2026	Framework (Formal)	Agent	Workflow	Agent authority and security boundary
LLM-MAS survey [61]	2025	Survey (Capability)	Agent	Multi-Agent	Delegated context and shared state
MAS security [144]	2026	Survey (Safety)	Agent	Multi-Agent	Message boundary and multi-agent state
Agents of Chaos [173]	2026	Safety	Agent	Multi-Agent	Live-lab communication and workflow state
MAS code execution [194]	2025	Attack	Agent	Multi-Agent	Orchestrator control and communication state
ControlValve [73]	2026	Defense	Agent	Workflow	Permitted agent-invocation graph and rules
<i>Model and policy state substrates</i>					
LoRA [66]	2022	Capability	LLM	Model	Low-rank adapter substrate
PEFT survey [57]	2024	Survey (Capability)	LLM	Model	Parameter-efficient update family
Fine-tuning safety [159]	2023	Safety	LLM	Model	Fine-tuned checkpoint regression
Safe LoRA [65]	2024	Defense	LLM	Model	LoRA adapter mitigation
KVEraser [96]	2026	Defense	LLM	Model	Cached context influence
PEFT Trojan [64]	2024	Attack	LLM	Model	PEFT adapter backdoor
Feedback backdoors [162]	2024	Attack	LLM	Model	Preference or feedback update backdoor
Dormant behavior [50]	2025	Attack	LLM	Model	Delayed activation in fine-tuned behavior
Backdoor survey [268]	2025	Survey (Attack)	LLM	Model	Backdoor safety issues in LLM parameters

4 Adaptation: State-Change and Activation Operations



Operation Premise. The substrate chapter identifies carrier families and their initial authority regimes; this chapter supplies the operation half of each S×A pair. Throughout the survey, Adaptation is coded using seven operation families: *add*, *abstract*, *activate*, *modify*, *forget*, *propagate*, and *migrate*. Implementation-specific verbs are assigned to these families by the state change they commit, so retrieval, for example, counts as activation only when the retrieved state receives authority over the current decision. The seven families provide a shared vocabulary for feedback reuse [177], memory management [148], skill accumulation [199], reasoning and action coupling [235], and model adaptation [66]. An operation has no fixed safety meaning in isolation: the comparison criterion is whether it preserves or revises the provenance, scope, authority, privacy, and recovery handles of the carrier on which it acts.

The evidence varies by operation. Stealth memory injection, GhostWriter, and memory-poisoning studies directly observe unsafe writes and delayed activation [184, 193, 260]. Privacy work examines sensitive reuse, InjecAgent observes the action boundary, and Safe LoRA supplies a model-update intervention [65, 197, 247]. Later sections compare these with explicit migration and propagation studies, separating a transformed object from the new sites at which it can activate. Table 2 indexes the operations; Section 5 examines the resulting attribute failures.

Selection Before Commitment. Selection determines which proposed operation is committed; the seven families describe what that operation changes. The utility proxy $\hat{u}_t$ ranks candidates, while c_t restricts their eligibility and permitted role. If relevant safety evidence is absent from the candidates, proxy, or encoded checks, a locally useful update may still be selected. The distinction in Section 2 therefore matters for every family: violating an explicit constraint differs from passing an incomplete constraint set.

4.1 Add: Creating Reusable Influence

Add operations create a new reusable object in a carrier; revising the same object’s content or attributes is a modify operation. A single update can include both. Addition matters because transient context can become reusable influence. Adjacent capability systems show three mechanics: verbal critiques can be stored for later strategy choice [177], memory items can be retained across context boundaries [148], and skills can become reusable procedures [199]. Trajectory-Informed Memory Generation adds structured, provenance-bearing learnings extracted from execution traces, while FORGE turns failed trajectories into reusable natural-language memories that can be broadcast across a population [14, 41]. PASB makes the endogenous write path explicit: the system accepts an ordinary user claim and commits it with status promotion, attribution removal, or broader scope, so the unsafe transition can occur without a poisoned payload [131]. Adversarial paths reach the same operation through external email content saved by stealth memory injection or a hidden GhostWriter payload admitted by a tool-using personal agent [193, 260]. Privacy work adds the case in which useful personal information becomes retained profile state with sensitivity or scope obligations [197].

A feedback note such as “verify account scope before using external tools” is a state transition, not automatically a violation or a defense: the carrier is a stored note, the trigger is task completion, the safety attributes say advisory reminder, the authority says it may guide planning but cannot override current consent, and the rollback handle points to the episode that produced it. If the same operation stores an unconditional action shortcut, the future influence is stronger because the note now looks like a rule. SAVER treats both as add operations but distinguishes them by authority and downstream actionability. Once a note, memory, skill, or observation is stored, later operations can summarize it, copy it, activate it, or revise it. Memory-poisoning work makes this descendant chain concrete: unsafe content can enter persistent memory or a knowledge base and become relevant only when a later operation reuses it [184]. The add operation is therefore a write event and the moment at which future influence becomes possible.

4.2 Abstract: Compressing State and Reassigning Safety Attributes

Once state has been added, abstraction can compress, summarize, cluster, or reframe it and thereby change its safety semantics [261]. The derived object may no longer carry the exact provenance, uncertainty, user boundary, or temporal scope of the source material. MemGPT makes movement between immediate context and managed memory concrete [148]; reflection can convert critiques into reusable heuristics [177]; and skill learning can convert successful traces into named routines [199]. These are adjacent operation mechanics; the safety claim begins when an abstraction drops source and derivation attributes [15] or privacy and revocation attributes [197] that later matter for retrieval, action, or recovery.

Summarization is not inherently unsafe; the problem is that abstraction can silently change authority [109]. Suppose three notes local to one session say that a user prefers short answers in a specific project. A summary that keeps the source scope intact remains advisory: “for this project, prefer concise replies.” A summary that drops the project boundary becomes a global preference. A summary that drops uncertainty becomes a rule. The content is similar, but the role has been rewritten from scoped evidence to general instruction, and the object’s future influence now spans tasks it was never granted [35].

AuthMem-Bench turns that role rewrite into a controlled abstraction test. Its paired histories hold the focal proposition and later task constant while varying whether the source permits the target use; the write-time module then asks whether consolidation preserves that distinction [249]. This differs from a poisoned-memory benchmark because the claim need not be false or malicious. Here consolidation preserves the proposition but loses its use constraint. Safe to Check, Unsafe to Use isolates a second abstraction failure at the compression boundary. A pre-compression filter can accept separated fragments because none contains a complete malicious instruction, yet the summarizer can relink them into compact, backend-actionable context [119]. The unsafe object is therefore derived by the abstract operation rather than copied from one source span. Unlike authority collapse, relinking changes the instruction formed from the fragments; post-abstraction checks must therefore examine both source authority and derived content.

Takeaway 4.1. Abstraction Requires a New Safety Check. Abstraction can preserve a proposition while dropping its permitted-use constraint, or combine acceptable fragments into an unsafe instruction [119, 249]. Source-level screening cannot settle either case. The derived object needs its own authority and content checks, together with source links that support later invalidation [15].

4.3 Activate: Turning Stored State into Current Authority

Added or abstracted state remains latent until activation lets it affect a current reasoning step, tool call, plan, or response. Non-adversarial cases make the authority change especially clear: PS-Bench shows truthful retrieved memories making an unsafe request appear contextually legitimate, while PerMemSafe makes retained user-specific context part of the response-safety judgment [5, 53]. Attack studies reach the same operation through stealthily injected personal-agent memory, GhostWriter payloads, or broader poisoned-memory reuse [184, 193, 260]. The evidence then extends to tool settings, where InjecAgent makes untrusted external content safety-relevant at a tool-using decision [247]. ReAct and Voyager supply adjacent mechanics for why this matters: reasoning traces can condition actions, and selected skills can become executable routines [199, 235]. Activation is therefore the operation that converts latent influence into current authority. The same retrieved memory may supply background evidence without being authorized to set a tool parameter or grant permission to act. An activation check therefore depends on the role assigned during prompt assembly or execution, not merely on which item matched the query.

CaMeL and the Dynamic Rule-based Isolation Framework for Trustworthy agentic systems (DRIFT) make this gate concrete at the tool boundary. CaMeL derives control and data flows from the trusted user query, then uses capabilities to prevent untrusted data from changing program flow or authorizing disallowed flows at tool calls [32]. DRIFT constructs a minimal function trajectory and parameter checklist from the query, validates later deviations against user intent and privilege constraints, and masks conflicting instructions before they enter the active memory stream [94]. CaMeL therefore provides a design-time separation of control from untrusted data, whereas DRIFT adds runtime revision checks and current-stream

isolation. ControlValve applies the same activation question to a multi-agent orchestrator: it generates a permitted control-flow graph and checks each agent invocation against that graph and a contextual rule [73]. All three treat activation authority as an explicit system decision rather than a property inherited from retrieved text or inter-agent messages.

Takeaway 4.2. Relevance Is Not Permission. Retrieval relevance cannot decide whether an item may control execution. CaMeL separates trusted control from untrusted data, DRIFT validates runtime deviations, and ControlValve checks permitted agent invocations [32, 73, 94]. Their common target is the authority granted at use, even though they enforce it at different decision boundaries.

4.4 Modify: Revising State, Safety Attributes, and Permissions

Activation can expose stale scope, conflicting evidence, or unsafe authority that requires modification. Changing an existing object or its safety attributes can repair that influence or widen it further. MemGPT and memory mechanism surveys make editable memory concrete, while Reflexion and Voyager show adjacent editable carriers in feedback notes and skills [177, 199, 261]. Beyond Heuristics formulates memory reading, writing, and maintenance as sequential decisions under uncertainty, while EvolveMem revises retrieval configurations with diagnosis, revert-on-regression, and explore-on-stagnation steps [114, 182]. These systems make candidate selection and regression-sensitive modification observable; they do not by themselves establish that safety attributes are preserved. The operation can correct a memory, reconcile conflicting notes, revise a skill, change a workflow rule, or update activation metadata [8]. Memory-privacy work directly motivates preserving sensitivity and scope [197]; provenance retention and rollback metadata are additional design requirements introduced by this survey. Consider a feedback note that says “retry with a shorter plan.” If repeated failures reveal missing consent for external-account actions, the agent may modify it to “when a task involves external accounts, ask for confirmation before action.” The role shifts from performance heuristic to safety constraint. Its authority increases over planning decisions but should remain below explicit user instruction and system policy. Its scope also widens: the rule now applies to a class of tasks rather than one episode. A SAVER trace records this as a modify operation rather than a new add because the old object, its descendants, and its activation history still matter.

Conflicts become operational when a memory says that a user prefers automatic reminders and a later interaction says reminders should pause during travel. A reconciliation step does more than choose one sentence. It changes temporal validity, scope, and activation priority. In safety attribute terms, the older memory may be downgraded to historical evidence, while the newer memory becomes active preference for a date range. If this safety attribute update is not explicit, later activation can resurrect stale authority.

4.5 Forget: Invalidating Influence and Descendants

Forgetting withdraws influence when a role expires, authorization is revoked, or modification cannot restore legitimate use. Removing, expiring, quarantining, or rolling back state is therefore an influence action rather than a storage action [197]. Later memory-poisoning and sleeper-memory work make persistent query-to-memory influence and delayed activation concrete [156, 184]. We infer that copies, summaries, and derived rules must be checked separately rather than assuming removal of the source also removes their influence. Once retained state has produced summaries, rule edits, cached plans, or invoked skills, this survey treats those derived objects as additional activation sites. The recovery unit is therefore the discovered descendant set rather than only the original record, with memory-security work supplying the broader carrier and threat vocabulary [109].

For benign lifecycle changes, a time-sensitive preference may expire, a temporary permission may be revoked, or a stale skill may be retired. In each case the object moves to an expired, revoked, or archived state; its authority over the current action drops to zero; and its future influence should be limited to audit or historical reasoning. SAVER therefore treats forgetting as more than deletion. The operation must answer whether the influence was actually withdrawn from the places where future decisions are made.

4.6 Propagate: Copying Influence Across Actors and Contexts

Before or after activation, propagation can copy state to another agent, memory segment, workflow stage, artifact, or context. Multi-agent surveys explain why shared state and delegated context are routine, while self-evolving-agent surveys place that movement inside a broader adaptive lifecycle [8]. Architectural analysis relates compromised memory to inter-agent exchange, and ACIArena measures cascading injection across trust channels [6, 192]. MAPLE-Guard then localizes the memory route as write, retrieval, promotion, and cross-agent reuse: a poisoned private memory can enter shared memory and influence an agent that never saw the source attack [224]. Propagation therefore changes both the number of activation sites and the receiver’s authority assignment; it cannot be measured only by inspecting visible inter-agent messages.

EVOMAL supplies a skill-to-skill propagation case. A retrieved malicious skill is imitated into an agent-authored descendant, and rolling library replacement lets that descendant become a later template. The agent self-poisoning rate (ASPR) measures the fraction of tasks that author and store a skill reproducing the planted pattern. After the eight planted seeds are removed, Qwen3 reaches 68% ASPR at round five, Gemma4 reaches 29.4%, and Devstral remains infected, whereas DeepSeek-V4-Pro and MiniMax return immediately to baseline and GPT-OSS decays [218]. The result is therefore conditional evidence of self-sustaining propagation under the paper’s five-round replacement protocol, not a universal worm claim. It also makes source deletion an incomplete recovery test: the relevant unit is the retrievable descendant population after the seed disappears.

Recent multi-agent evidence separates propagation from control-flow hijacking. In the former, content or a practice crosses actors and is taken up by another agent; the live-laboratory cases in Agents of Chaos include cross-agent propagation among several other operational failures [173]. In the latter, untrusted content changes the active invocation edge. Triedman et al. show that an orchestrator can be induced to invoke unsafe agents or functions even when the individual agents resist the malicious request [194]. The transferred object is therefore not always a belief or memory item. It can be a routing decision that grants a new component authority. Recording both the content path and the invocation path prevents these two operations from collapsing into a generic prompt-injection label.

If a planner sends a worker the message “the customer may prefer email; verify before sending,” the propagated state remains advisory and scoped. If the same content is copied as “send email to the customer,” propagation has transformed authority even without changing much surface text. The receiver now sees an action instruction. SAVER records the sender, receiver, copied content, retained safety attributes, and permitted activation modes so that later exposure can be traced to the propagation path.

Takeaway 4.3. Replication Multiplies Repair Obligations. Propagation creates activation sites that may outlive the sender’s original authorization. A correction at the source leaves inconsistent state if recipients retain older copies or derived rules. The propagation record must therefore support correction and revocation along the same paths that carried the original influence.

4.7 Migrate: Moving State Across Authority Regimes

Propagation multiplies copies; migration changes the regime under which a copy can be used. A migrate operation moves state into a substrate with different persistence, privacy, actionability, or governance semantics. MemGPT makes movement between memory regions visible [148], while self-evolving agent surveys ground broader substrate movement [8]. A feedback note moved into long-term memory becomes more persistent; a memory item converted into a skill or workflow rule gains authority to act; a local observation moved into a shared artifact gains a new audience [199]. Model updates are adjacent rather than identical, but self-evolution surveys establish substrate movement as a major axis of agent adaptation [8].

Context reconstruction is a migration when it changes the authority regime even if the text is unchanged. When Context Gets Root compares each content item’s fixed origin level with the level assigned by the harness at a later model call [62]. Context privilege escalation adds an orthogonal scope comparison: M-CPE raises message role, while X-CPE moves content into a source with longer persistence or broader project, user, or cross-agent reach [105]. The primary operation remains *migrate* because the content crosses an authority or

scope boundary. Writing it into the broader source also *adds* reusable state, delegation may *propagate* it, and later replay *activates* it; these secondary verbs locate subsequent transitions rather than create a parallel taxonomy.

Two migration patterns matter most here: note $\rightarrow$ rule, where feedback from one episode becomes planning authority, and memory or trajectory $\rightarrow$ skill, where stored experience becomes executable capability. Voyager illustrates the capability mechanic. PoisonedEvolution isolates the preceding migration decision: attacker evidence must be included, attributed as reusable expertise, and realized in the generated skill even though the attacker cannot edit the skill bank [21, 199]. SkillJack then shows what the completed crossing can hide and preserve, because extraction can obscure malicious intent and leave the implanted behavior active after source-record removal [240]. SkillMisevo-Bench extends observation beyond the crossing: its clean reload separates authored skill state, later retrieval, and fresh-session execution, so successful migration and successful activation remain distinct events [132]. The migration gate must therefore check not only content correctness but also whether support is independent, whether the target substrate grants stronger influence, whether source constraints survive extraction, whether later activation is attributable, and whether rollback can still reach the migrated behavior.

Representation And Influence. Reflection, skill learning, and workflow compilation show that related guidance can be carried by a memory item, heuristic, callable skill, or gated policy [177, 199, 202]. Repairing one representation therefore establishes a scoped result: a summary, rule, or other carrier from the same lineage may still influence behavior. The literature establishes these transformation mechanics but does not yet provide a general test for functional equivalence across substrates. We treat cross-substrate reappearance as an evaluation question, using fixed downstream probes and lineage records rather than surface-form matching alone.

4.8 Composition Paths

A feedback note can be stored and later used to revise strategy [177]; managed memory can move information across context boundaries [148]; a behavior trace can become an invoked skill [199]; and an observation can shape the next action in an interleaved reasoning-and-acting trace [235]. The add–abstract–activate compositions used by SAVER are a comparative reading across these mechanics. Unsafe paths can follow the same structure, which is why operation records must carry safety attributes, authority, and future influence fields together.

Local-Success Generalization. Local success becomes safety-relevant when it is promoted into a durable role. Reflexion and Voyager establish the capability mechanics by turning feedback into strategy and successful traces into callable skills [177, 199]. Safety studies then expose three distinct failure modes. OEP consolidates a locally correct but non-transferable experience into a harmful rule; experience-driven safety work finds that accumulated benign-task experience can bias later agents toward acting rather than refusing; and EvoBreak deliberately acquires complementary benign experiences whose joint activation weakens the safety boundary [202, 229, 266]. The progression is from one promoted experience, to aggregate action bias, to adversarial composition over several experiences.

AgentPoison provides a fourth mechanism and clarifies why the corresponding success rates should not be ranked directly. It begins with an adversarial object written into memory or a knowledge base and tests trigger-selected retrieval [25]; OEP begins with locally useful experience and tests transfer after consolidation; the experience-driven study tests distributional change after benign accumulation; and EvoBreak tests whether a dependency set can be assembled and activated. These designs fail admission, selection, generalization, or composition gates under different opportunity sets. A comparable report must therefore identify the source set, operation, activation condition, and downstream scope rather than compare headline attack rates alone.

Within an agent loop, model updates can combine the same primitives. Fine-tuning and adapter training are add or modify operations over model state; post-training and preference optimization modify reward or policy behavior from feedback; merge combines adapters or patches and may change scope, authority, and safety regression attributes; edit and unlearn modify or remove behavior whose residual influence can be hard to observe. Existing model safety papers show why these operations deserve attention: fine-tuning can erode safety behavior [159], PEFT updates create scoped but deployable risk objects [64], and dormant

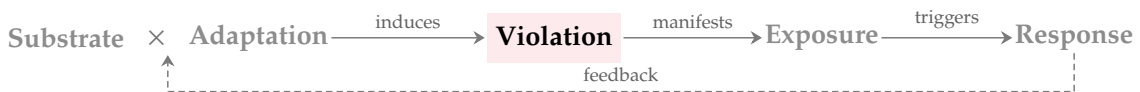
Table 2 Adaptation and activation operations in the current literature.

Method or Work	Year	Role	Target	Object	Operation	Influence Shift
<i>Capability mechanics: add, abstract, migrate</i>						
Reflexion [177]	2023	Capability	Agent	Workflow	Add	Feedback becomes decision evidence
MemGPT [148]	2023	Capability	LLM/Agent	Memory	Add / Migrate	Context becomes managed memory
Voyager [199]	2023	Capability	Agent	Skill	Migrate	Experience becomes callable skill
SimpleMem [113]	2026	Capability	Agent	Memory	Add	Interaction becomes durable memory
Panini [161]	2026	Capability	Agent	Memory	Add	Experience becomes structured memory
Experience Evolving Memory [99]	2026	Capability	Agent	Workflow	Abstract	Episode becomes reusable procedure
<i>Unsafe writes and abstraction drift</i>						
AgentPoison [25]	2024	Attack	Agent	Memory	Add / Activate	Poisoned write becomes memory with authority
Stealth memory injection [260]	2026	Attack	Agent	Memory	Add / Activate	Email payload becomes hidden memory influence
GhostWriter [193]	2026	Attack	Agent	Memory	Add / Activate	Hidden payload becomes recalled memory
PASB [131]	2026	Benchmark (Safety)	Agent	Memory	Add / Activate	Accepted claim becomes durable personal state
AuthMem-Bench [249]	2026	Benchmark (Safety)	Agent	Memory	Abstract	Consolidation drops source-use constraints
Zombie Agents [234]	2026	Attack	Agent	Memory	Add / Activate	Dormant memory becomes state conditioned on trigger
MemoryGraft [181]	2025	Attack	Agent	Memory	Add / Activate	Poisoned experience becomes evidence
OEP [202]	2026	Attack	Agent	Workflow	Abstract	Experience becomes unsafe reusable rule
Experience-Driven Safety [266]	2026	Safety	Agent	Workflow	Abstract	Experience update creates policy drift
EvoBreak [229]	2026	Attack	Agent	Memory	Abstract / Activate	Benign experiences compose into unsafe influence
AgenticEval [210]	2026	Benchmark (Safety)	Agent	Evaluation	Abstract	Adaptive state becomes task suite denominator
MemEvoBench [222]	2026	Benchmark (Safety)	Agent	Memory	Abstract	Evolving memory becomes metric denominator
PerMemSafe [5]	2026	Benchmark (Safety)	Agent	Memory	Activate	Personalized memory becomes safety denominator
PS-Bench [53]	2026	Benchmark (Safety)	Agent	Memory	Activate	Benign memory changes inferred intent
<i>Control, activation, and repair operations</i>						
GLOVE [239]	2026	Defense	Agent	Memory	Modify	Memory is realigned against environment
SRM [28]	2026	Defense	Agent	Memory	Modify	Session safety becomes memory signal
MemGovern [205]	2026	Governance	Agent	Memory	Modify	Recall is revised by policy
SSGM [84]	2026	Governance	Agent	Memory	Activate	Stored memory passes through gate
Progen [176]	2025	Defense	Agent	Tool	Activate	Tool authority becomes explicit decision
CaMeL [32]	2026	Defense	Agent	Tool	Activate	Trusted control flow gates tool authority
DRIFT [94]	2025	Defense	Agent	Tool	Activate	Plan deviations trigger privilege validation
ControlValve [73]	2026	Defense	Agent	Workflow	Activate	Agent invocation must follow permitted graph
MAGE [212]	2026	Defense	Agent	Memory	Activate	Shadow memory becomes safety signal
Privacy Agent Memory [197]	2025	Safety	Agent	Memory	Forget	Sensitive influence is scoped or withdrawn
Memory poisoning [184]	2026	Attack	Agent	Memory	Add / Activate	Poisoned query creates later retrieval influence
OpenClaw [233]	2026	Capability	Agent	Workflow	Activate	Retrieved longitudinal state enters workflow context
<i>Propagation, migration, and route drift</i>						
Collective Misremembering [226]	2026	Attack	Agent	Multi-Agent	Propagate	Drift spreads across agents
ACIArena [6]	2026	Benchmark (Attack)	Agent	Multi-Agent	Propagate	Cascading injection crosses agent trust
HarnessSafe [255]	2026	Benchmark (Safety)	Agent	Workflow	Migrate	Influence crosses carrier and harness boundaries
When Context Gets Root [62]	2026	Attack	Agent	Workflow	Migrate	Tool content gains a higher context role
Agent Context CPE [105]	2026	Attack	Agent	Workflow	Migrate	Content gains a higher role or wider scope
MAPLE-Guard [224]	2026	Defense	Agent	Memory	Propagate	Gates private-to-shared memory reuse
Agents of Chaos [173]	2026	Safety	Agent	Multi-Agent	Propagate	Unsafe practices cross agents in a live setting
MAS code execution [194]	2025	Attack	Agent	Multi-Agent	Activate	Untrusted content redirects agent invocation
Embedding defense failure [253]	2026	Attack	Agent	Memory	Propagate	Shared memory filter is bypassed
MADRA [200]	2025	Governance	Agent	Multi-Agent	Propagate / Modify	Risk assessments are exchanged and revised before gating
Devil's Moltbook [198]	2026	Attack	Agent	Multi-Agent	Propagate	Exposure shifts agent society state
Traversal-as-Policy [98]	2026	Defense	Agent	Workflow	Migrate / Activate	Logs become workflow policy
Library Drift [256]	2026	Safety	Agent	Skill	Migrate / Modify	Skill library reroutes tasks
SkillJack [240]	2026	Attack	Agent	Skill	Migrate	Poisoned trajectory becomes persistent skill
PoisonedEvolution [21]	2026	Attack	Agent	Skill	Migrate	Recurring trajectory evidence becomes instruction
SkillMisevo-Bench [132]	2026	Benchmark (Safety)	Agent	Skill	Migrate / Activate	Skill authoring, retrieval, and carryover gates
EVOMAL [218]	2026	Attack	Agent	Skill	Propagate / Activate	Authored copies re-enter the retrievable library
Tool-Drift [30]	2026	Safety	Agent	Tool	Activate / Modify	Tool choice state reroutes
JITRL [103]	2026	Capability	Agent	Workflow	Modify / Activate	Runtime learning updates policy
RQGM [70]	2026	Capability	Agent	Evaluation	Modify	Evaluator utility becomes state scoped by epoch
<i>Model update operations</i>						
Fine-tuning safety [159]	2023	Safety	LLM	Model	Modify	Fine-tuning changes behavior
Safe LoRA [65]	2024	Defense	LLM	Model	Modify	Adapter constrains behavior
PEFT Trojan [64]	2024	Attack	LLM	Model	Add / Activate	Adapter trigger becomes deployed safety issue
Dormant behavior [50]	2025	Attack	LLM	Model	Activate	Fine-tuned behavior waits for context

behavior can survive until the relevant activation context [50]. The agent-loop criterion determines inclusion; regression probes support behavioral claims, whereas rollback handles and residual tests are additionally needed for recovery claims. Section 5 asks which safety attributes these operations fail to preserve.

Takeaway 4.4. Migration and Propagation Change Different Properties. Propagation multiplies the sites at which influence can activate; migration changes the authority regime under which it can be used. One transfer can do both. Counting copies alone misses a role upgrade, while checking the new role alone misses the other copies that remain available for later reuse.

5 Violation: Safety Attribute Failures



Given a carrier-operation pair, violation analysis asks whether the resulting role remains legitimate. A violation occurs when the operation fails to preserve a required safety attribute, such as source authority, user scope, privacy, or a resource limit. This condition is distinct from the evidence that reveals it. A write-time check may detect the mismatch immediately; otherwise it can remain latent until retrieval, execution, or another later use. The distinction lets **V** describe what failed and **E** describe what was actually observed.

The evidence enters through two complementary routes. GhostWriter and PASB separate storage from later activation through adversarial admission and ordinary claim commitment, making provenance and persistence failures visible [131, 193]. InjecAgent and programmable privilege control expose the authority question when state reaches a tool-using decision [176, 247]. These works are related by the failed attribute, even though their benchmark denominators differ. Together they produce a named violation and expected trigger; Section 6 uses that pair to identify diagnostic evidence. Table 3 closes the chapter with the detailed paper-level index.

5.1 Provenance Loss and Safety Attribute Laundering

Provenance loss removes information about a state’s source, contributor, or derivation. Safety attribute laundering occurs when the transformed object then appears to have a stronger role than its origin warrants. The critical loss is not necessarily factual content: later components may still read the same claim but can no longer distinguish environmental data, an attributed preference, model inference, or authorized policy.

The literature exposes different entry points. AgentPoison places malicious demonstrations in a retrievable store and tests their selection by a trigger query [25]. GhostWriter instead exploits admission into personal-agent memory before later retrieval [193]. PASB studies the agent’s own consolidation decision: attribution removal and status promotion can make an ordinary conversational claim appear established, while scope broadening extends its later use [131]. Store poisoning, unsafe admission, and claim promotion therefore require different checks even when all culminate in trusted-looking memory. Source retention at write and abstraction supports later authorization; checking only the retrieved wording cannot recover provenance that those operations discarded.

5.2 Authority, Actionability, and Boundary Escalation

Authority, actionability, or boundary escalation can follow provenance loss, but can also occur when a known source is assigned an unauthorized role. The carrier may be a memory item, retrieved instruction, plan fragment, tool result, workflow rule, learned skill, feedback summary, or privilege token. The operation is usually *activate*, *migrate*, or *modify*: descriptive state becomes a planning constraint, readable context becomes a tool argument, a local preference becomes a broader rule, or a stored fragment becomes part of an access control bypass. What fails is role preservation across time. AgentDojo gives the integrated-tool setting in which untrusted tool outputs can hijack behavior; Progent supplies the response vocabulary for explicit privilege control; and FragFuse shows the temporal memory variant, where low-authority fragments are stored across interactions and later fused through retrieval so the current query appears cleaner than the activated state actually is [31, 163, 176].

AuthMem-Bench isolates the same family without changing claim content. Matched histories contain the same proposition and lead to the same later task, but only one source condition permits the target use; authority collapse occurs when consolidation makes those histories operationally indistinguishable [249]. This design separates the internal violation from its exposure: the violation is the false authority upgrade during abstraction, while the exposure is a later prohibited tool action after the collapsed memory is activated. Authority is therefore not recoverable from proposition text alone once consolidation has erased the source-use constraint.

Harness privilege escalation reaches the same invariant through context migration rather than memory consolidation. The harness can carry unchanged low-level content into a new call under an actual user or system-effective role, so the downstream model need not confuse roles or violate its visible hierarchy [62]. A complementary context-source analysis distinguishes this role increase from scope increase: M-CPE changes which instructions the content may compete with, while X-CPE lets session-scoped influence survive at project or user scope [105]. AuthMem-Bench shows a source-use constraint disappearing during

abstraction; these harness studies show source role or scope being replaced during reconstruction. All three instantiate safety attribute laundering because the downstream component receives stronger authority or broader reuse than the origin warrants.

Takeaway 5.1. Correct Content Can Carry Unjustified Authority. Authority collapse need not falsify a claim, and harness privilege escalation need not make the model violate its visible instruction hierarchy [62, 249]. The failure can occur earlier, when consolidation or context assembly grants a role the source never held. Content accuracy and model-side hierarchy compliance therefore leave distinct authority checks unresolved.

Authorization continuity exposes a complementary failure. AuthMem-Bench holds the proposition fixed and varies its source authority; the state-bound model holds the user grant fixed and asks whether changes to control state, capability, delegation, task phase, trust context, or enforcement have changed the subject or context to which that grant applies [262]. The model separates two independent invariants: the mutation must remain inside the grant’s transition envelope, and every realized effect must remain below its fixed ceiling. The failed attribute is the continuing validity of the grant.

These cases share an authorization requirement: descriptive state needs an explicit grant before it can direct a plan, tool call, or workflow. ReAct, Voyager, and Reflexion show the capability paths through which observations, skills, and feedback influence later behavior [177, 199, 235]. R-Judge and GuardAgent supply risk-recognition and guard-reasoning signals [221, 245]; an authorization control must additionally decide whether the current user, task, and effect permit that influence. Detecting risky content and validating the grant are consequently separate obligations.

Promotion Failures. The promotion test in Section 2 separates evidence of local effectiveness from permission for broader reuse. OEP makes single-experience transfer failure concrete, experience-driven safety work shows aggregate benign experience shifting the action–refusal boundary, and EvoBreak shows individually acceptable experiences becoming unsafe under composition [202, 229, 266]. Skill-oriented attacks then expose the cross-substrate step: PoisonedEvolution targets the attribution decision that grants recurring trajectory evidence procedural authority, whereas SkillJack tests the laundering and persistence of the resulting skill [21, 240].

SkillMisevo-Bench resolves the completed path into successive gates: all 21 evolved configurations author unsafe artifacts, 19 retrieve unsafe skills, 19 show benign-task contamination, and 15 retain fresh-session harm [132]. A configuration that stops before final harm is therefore not evidence of a safe promotion; the unsafe artifact may simply not have been retrieved or executed. Utility-to-authority escalation names the local failure, while selection-induced boundary drift names the longitudinal diagnosis when individually admitted promotions cumulatively widen scope or authority.

5.3 Privacy and Purpose Failure

Role widening can concern purpose and audience as well as action authority. Privacy can therefore fail before disclosure when sensitive state loses the attributes that made later use legitimate. The carrier may be explicit personal information in memory, inferred attributes, user-specific preferences, private tool outputs, conversation history, or summaries that merge several users or sessions. Add, abstract, activate, propagate, or migrate can store too much, broaden a profile, grant a sensitive item influence in an unrelated task, share it with another agent, or move it into a less protected artifact. The failed attributes are sensitivity, subject boundary, purpose limitation, retention constraint, or personalized risk context. Memory-privacy work grounds these use restrictions [197].

A related question concerns response safety under personal context, rather than unauthorized disclosure or reuse. PerMemSafe shows how long-horizon user-specific memories can make an otherwise safe response unsafe in that user’s implicit context [5]. Differential Harm Propensity isolates another personalized boundary: mental-health disclosure changes measured harm and refusal behavior, so the evaluator must treat the disclosure as risk-bearing context rather than a neutral profile field [238]. PS-Bench isolates the converse boundary failure: an inherently harmful request can be treated as safe because truthful personal memories bias the model toward a benign intent interpretation [53].

Takeaway 5.2. Privacy Can Fail Before Disclosure. Persistent user context can lose its purpose, retention, or audience restrictions before any information is printed or acted on [104, 197]. An output-only privacy test misses that earlier state-level failure. The survey therefore calls for preserving these restrictions at update time and checking them again at activation.

5.4 Persistent Descendants and Incomplete Rollback

Persistent descendant failure arises when correcting or deleting the source leaves its derived influence available for later activation. The failed operation is incomplete forgetting: an earlier abstraction, copy, or migration has created an object that the repair does not reach. Memory-poisoning and sleeper-memory work establish delayed reuse, OEP extends the mechanism to consolidated experience, and SkillJack directly tests the source–descendant distinction: its abstract reports that 80.0% of skill-mediated attacks persist after the original poisoned records are deleted [156, 184, 202, 240].

This failure differs from merely storing an unsafe object. It requires a correction or revocation that no longer governs a surviving activation site. A corrected preference can still steer a workflow that copied its earlier version; a retired source can leave an executable skill behind. The repair target is thus a derivation relation, not one storage location. The descendant inventory and residual tests in Section 7 specify how to assess that target without assuming complete visibility.

5.5 Operational Integrity Failures

Operational integrity concerns resource use and traceability, whether or not unsafe state has already produced descendants. A workflow rule, retry policy, tool template, planner state, retrieved instruction, self-evaluation loop, skill, shared plan, or propagated artifact can affect resource use, liveness, auditability, and accountability. Activate, modify, propagate, or migrate may turn a local retry heuristic into a general workflow rule, trigger repeated searches, loop over paid APIs, or leave causal state active after a visible action is blocked. The failed attributes are resource budget, rate limit, cost ceiling, timeout, stop condition, trace link, and repair ownership. OWASP frames excessive agency and unbounded consumption as application risks; AgentDojo moves observation beyond final text; Agent Security Bench (ASB) and Agent-SafetyBench broaden the evaluation vocabulary [31, 147, 252, 263].

Resource control and auditability impose different checks on the same execution. A workflow rule can preserve task success while multiplying cost; a guard can detect the resulting risk without identifying which stored rule caused it. Personal-agent work supplies the operational setting, GuardAgent and R-Judge examine recognition, Progent constrains privilege, and EDDOps addresses monitoring and maintenance [104, 176, 220, 221, 245]. Together they motivate enforceable budget and stop conditions plus attribution to the active state. Direct tests of resource-exhaustion repair and lifecycle auditability remain an evidence gap in the reviewed corpus.

5.6 Model Safety Regression

The same attribute logic extends from explicit state to model-side state, but the carrier is harder to inspect and often broader in scope. Model safety regression enters this survey when an update to parameters, adapters, reward models, edited components, policy prompts, or unlearning artifacts weakens an attribute that a later agent relies on. The operation is central when the agent loop or deployment controller fine-tunes, trains adapters, post-trains, merges, edits, distills, unlearns, selects, loads, or rolls back that object for later behavior. The failure can be alignment erosion, adapter authority escalation, reward drift, persistent backdoors, incomplete unlearning, or lost deployment lineage. Fine-tuning safety, PEFT risk, feedback poisoning, and dormant behavior mark successive parts of this path [50, 64, 159, 162]. They support central SAVER claims only when the carrier, activation route, metric, and agent-loop scope are explicit.

An adapter that looks benign may gain broader authority when loaded into a production agent. A task-local fine-tuning update may change refusal behavior outside the intended scope. A merged adapter stack may erase the safety assumptions under which each component was evaluated. An unlearning patch may remove a visible behavior while leaving residual influence under paraphrase or downstream fine-tuning. The same

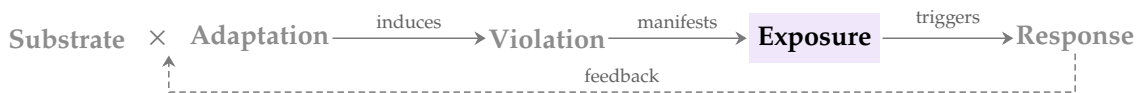
Table 3 Safety attribute violations in the current literature. Authority, scope, privacy, auditability, and recovery failures become comparable when tied to a concrete state transition.

Method or Work	Year	Role	Target	Object	Violation	Failed Attribute
<i>Provenance and authority failures</i>						
AgentPoison [25]	2024	Attack	Agent	Memory	Provenance loss	Memory source attribute
GhostWriter [193]	2026	Attack	Agent	Memory	Provenance loss	Hidden payload memory source
PASB [131]	2026	Benchmark (Safety)	Agent	Memory	Provenance / scope	Claim attribution and role preservation
AuthMem-Bench [249]	2026	Benchmark (Safety)	Agent	Memory	Authority / provenance	Source authority erased during consolidation
When Context Gets Root [62]	2026	Attack	Agent	Workflow	Authority / provenance	Harness elevates the model-facing source role
Agent Context CPE [105]	2026	Attack	Agent	Workflow	Authority / scope	Harness elevates source role or persistence scope
AgentDojo [31]	2024	Benchmark (Attack)	Agent	Tool	Provenance loss	Tool context source role
FragFuse [163]	2026	Attack	Agent	Memory	Authority / scope	Fragment authority context
EvoBreak [229]	2026	Attack	Agent	Memory	Authority / scope	Benign experiences gain joint authority
PoisonedEvolution [21]	2026	Attack	Agent	Skill	Authority / provenance	Untrusted recurring evidence gains instruction authority
SkillMisevo-Bench [132]	2026	Benchmark (Safety)	Agent	Skill	Authority / persistence	Unsafe success becomes reusable procedure
EVOMAL [218]	2026	Attack	Agent	Skill	Provenance / persistence	Agent-authored copies obscure attacker origin
PS-Bench [53]	2026	Benchmark (Safety)	Agent	Memory	Authority / scope	Benign context legitimizes harmful intent
InciAgent [247]	2024	Benchmark (Attack)	Agent	Tool	Authority / scope	Indirect instruction authority
ACIArena [6]	2026	Benchmark (Attack)	Agent	Multi-Agent	Authority / scope	Inter-agent instruction authority
MAS code execution [194]	2025	Attack	Agent	Multi-Agent	Authority / scope	Untrusted content controls agent invocation
Progent [176]	2025	Defense	Agent	Tool	Authority / scope	Scoped tool privilege
<i>Privacy and persistent repair gaps</i>						
Agent memory privacy [197]	2025	Safety	Agent	Memory	Privacy / purpose	Sensitivity/purpose/retention
Personal agents [104]	2024	Survey (Safety)	Agent	Memory	Privacy / purpose	Persistent personal context carriers
PerMemSafe [5]	2026	Benchmark (Safety)	Agent	Memory	Privacy / purpose	Personalized context safety
PersistBench [155]	2026	Benchmark (Safety)	Agent	Memory	Privacy / purpose	Memory scope/forgetting
AgentPoison [25]	2024	Attack	Agent	Memory	Persistent repair	Delayed poisoned influence
Memory poisoning [184]	2026	Attack	Agent	Memory	Poisoned retrieval	Persistent query-to-memory influence
SkillJack [240]	2026	Attack	Agent	Skill	Persistent repair	Skill survives source-record deletion
Self-evolving agents [8]	2025	Survey (Capability)	Agent	Workflow	Adaptive process	Capability evolution taxonomy
<i>Action authority and operational integrity</i>						
ReAct [235]	2022	Capability	LLM/Agent	Workflow	Authority / scope	Observation to action role
Voyager [199]	2023	Capability	Agent	Skill	Authority / scope	Executable skill authority
AgentDojo [31]	2024	Benchmark (Attack)	Agent	Tool	Authority / scope	Untrusted tool action state
OWASP LLM Top 10 [147]	2025	Framework (Security)	LLM/Agent	General	Operational integrity	Agency/cost bounds
AgentDojo [31]	2024	Benchmark (Attack)	Agent	Tool	Operational integrity	External effect tool context
Agent Security Bench [252]	2024	Benchmark (Attack)	Agent	Evaluation	Operational integrity	Agent security behavior
Agent-SafetyBench [263]	2024	Benchmark (Safety)	Agent	Evaluation	Operational integrity	Agent safety behavior
GuardAgent [221]	2024	Defense	Agent	Runtime	Local guard checking	Safety request → action check
R-Judge [245]	2024	Benchmark (Safety)	Agent	Evaluation	Operational integrity	Safety signal without transition cause
Progent [176]	2025	Defense	Agent	Tool	Authority / scope	Explicit tool privilege control
Misattribution Gap [2]	2026	Safety	Agent	Memory	Operational integrity	Memory fault attribution
Agents of Chaos [173]	2026	Safety	Agent	Multi-Agent	Operational integrity	Cross-agent uptake and live-system compromise
<i>Lifecycle drift and model regression</i>						
Reflexion [177]	2023	Capability	Agent	Workflow	Authority / scope	Feedback to strategy reuse
Self-evolution survey [190]	2024	Survey (Capability)	LLM/Agent	Workflow	Adaptive process	Self-evolution operation taxonomy
EDDOps [220]	2024	Governance	Agent	Governance	Operational integrity	Lifecycle drift tracking
Memory security survey [109]	2026	Survey (Safety)	Agent	Memory	Authority / scope	Memory semantic drift
Fine-tuning safety [159]	2023	Safety	LLM	Model	Model-side regression	Fine-tuning safety regression
PEFT Trojan [64]	2024	Attack	LLM	Model	Model-side regression	Adapter trigger safety issue
Dormant behavior [50]	2025	Attack	LLM	Model	Model-side regression	Dormant activation context
Safe LoRA [65]	2024	Defense	LLM	Model	Model-side regression	Adapter mitigation handle

transition logic therefore applies to the model substrate. The exposure trigger is a later inference, tool use decision, refusal boundary, policy check, or generalization failure across tasks. Safe LoRA provides a bounded LoRA-weight intervention for reducing fine-tuning safety loss [65]; broad LLM security surveys catalogue model-level security and privacy risks [206], and backdoor surveys frame triggerable residual risk [268]. The adapter registry, deployment lineage, version rollback, and residual probes proposed here are governance fields needed to connect those local results to an agent loop. Across the carrier families, a latent violation and its later activation remain distinct. Section 6 identifies the events that reveal these failures; Section 7 examines the interventions they support.

Takeaway 5.3. A Missing Trigger Does Not Establish a Safe State. An unsafe update can remain dormant when no later task activates it. SkillMisevo-Bench distinguishes unsafe artifacts, retrieval, and fresh-session harm, showing why these stages cannot be collapsed into one outcome [132]. A run without visible harm does not establish that earlier operations preserved the state’s safety attributes.

6 Exposure: Surfaces and Manifestations



Exposure Premise. Once a failed attribute and expected trigger are known, the analysis turns to where the latent mismatch first becomes observable and what record connects that event to the earlier carrier

and operation. Exposure is diagnostic evidence, not a second violation taxonomy: each study must identify the observed event and the criterion that makes it evidence of a violation. The distinction prevents latent unsafe state from being equated with realized harm and prevents visible harm from being attributed only to the final output step.

The recurring bridge cases are memory bypass, prompt injection, economic denial of service, model update, multi-agent uptake, persistent privacy, and boundary drift. Each starts from an internal safety attribute failure, but becomes measurable at a different boundary: delayed retrieval, fused query, tool execution, budget consuming loop, uptake across agents, stale personalization, or mismatched governance signal. Each surface offers a different observation of the failures in Section 5. A retrieval trace can identify the selected object, an execution trace can confirm an external effect, and a lineage trace can connect the two. Their combination determines whether the interventions in Section 7 can address the enabling state or only the visible event. Table 4 indexes the corresponding evidence.

6.1 Memory Retrieval Exposure

Memory retrieval exposure is the diagnostic surface for latent failures over stored or indexed state. The visible event is a later task selecting a memory entry, retrieved demonstration, profile note, summary, or derived preference and granting it influence. Direct attacks reach that event through different paths. FragFuse stores fragments whose prohibited meaning emerges only after fusion; stealth memory injection and GhostWriter target external-content admission before later activation; AgentPoison poisons retrievable examples selected by a trigger [25, 163, 193, 260]. Their common surface is delayed retrieval, but their causal operations and suitable controls differ.

Adjacent work broadens the diagnostic object. MemoryGraft studies poisoned experience retrieval, OEP studies harmful transfer from a consolidated local experience, and Tool-Drift follows remembered state into routing [30, 181, 202]. GLOVE examines disagreement between memory and environment, whereas MemPot treats extraction as the relevant defensive surface [211, 239]. Privacy work adds retrieval that is semantically correct but inappropriate for the current user or purpose [197]. PASB clears the originating conversation before a neutral query, which makes later sycophantic behavior evidence of durable-state activation rather than residual dialogue context [131]. PS-Bench adds a different exposure: benign memory retrieval changes inferred intent and the visible symptom is unsafe compliance rather than a corrupted memory record [53]. These studies should be compared by the state selected and the authority it receives, not by treating every later-memory failure as the same attack.

Takeaway 6.1. The Visible Failure May Have an Earlier Cause. A wrong answer or unsafe action at generation time may originate in an earlier memory update. Delayed activation separates the visible symptom from the operation that changed the state’s authority. Diagnosis must connect the triggering retrieval and prompt assembly to that earlier update before assigning the failure to the model or choosing a repair.

The diagnostic distinction is between selecting an item and granting it influence. Managed memory and feedback reuse explain how earlier state returns [148, 177]; PerMemSafe shows why the selected user context matters to the safety judgment [5]. A retrieval log alone identifies what matched, while prompt assembly reveals whether it was presented as evidence, preference, or instruction. Both are needed to attribute an unsafe response to the appropriate transition.

6.2 Tool and Action Exposure

Retrieval becomes externally consequential at the tool and action boundary, where authority, actionability, and resource failures produce an API call, file write, email, purchase, database update, code execution, skill invocation, or repeated high-cost path. Lower-authority state now reaches a tool, skill, or API surface where it can change the world. ReAct gives the general mechanism in which observations and reasoning steps condition actions, while early prompt-injection work explains how external content can pressure that path [118, 235]. AgentDojo provides the clearest direct anchor for untrusted data reaching integrated agents, and InjecAgent extends the pressure to benchmarked agent settings [31, 247]. AuthMem-Bench supplies

a controlled bridge from the earlier memory violation to this surface. Its action module keeps the claim, tool schema, and target argument fixed while changing whether the stored representation retains source authority; prohibited tool execution then measures the external consequence of an authority-collapsed memory [249]. The benchmark therefore localizes **V** at consolidation and **E** at action rather than treating the final tool call as an unexplained model failure. HarnessSafe generalizes this evidence rule across persistent carriers. A run reaches its violation stage only when a case-specific oracle confirms an executed behavior, committed state change, propagated artifact, or external effect; suspicious text, unsafe planning, or stated intent is insufficient [255]. Exposure is thereby tied to both the carrier path and observable consequence, while the furthest preceding trace stage shows whether the chain failed before or at the action boundary. Newer work varies the interaction pattern, input channel, and upstream object under test. Adaptive attacks and LivePI test whether defenses survive changing or live interaction patterns, while SnapGuard isolates screenshot-mediated injection in web agents [39, 248, 264]. Task Shield and SafeSearch focus on task alignment in action or search settings; Tool-Drift and prompt tool selection examine how remembered state or connector descriptions alter routing [30, 36, 74, 175]. The shared outcome is an action-boundary event, but each line of work perturbs a different upstream object. Response mechanisms inspect different properties at this boundary. Signed-Prompt checks source authenticity, LlamaFirewall places runtime guards, and Progent constrains tool privilege; secure-design patterns and formal agent-security work explain how such controls compose with data isolation [12, 27, 176, 179, 185]. An observed tool effect therefore needs both its triggering context and the policy decision that permitted execution to distinguish an input-screening failure from an authorization failure.

The safety attribute failure is usually authority escalation, actionability escalation, or resource boundary loss. A webpage, email, retrieved document, or memory item is treated not just as information but as a command or parameter. The exposure trigger is the point at which the agent selects a tool, fills an argument, invokes a skill, commits an external effect, or opens a repeated high-cost call path. This trigger can occur without a dangerous final answer. The user may never see the injected instruction, and the model may never print the unsafe state, but the action can still expose the violation through an irreversible action, a denial-of-service loop, or economic waste. The triggering state and the authorization decision distinguish these causes; Section 7 examines the controls that can act on that evidence.

Risk-awareness benchmarks can identify situations relevant to safety [245], and guard agents can reason about local responses [221]. Containment additionally requires changing execution, for example by removing suspect state, reducing privileges, forcing a clean plan, or routing the decision to human approval. Recording the failed authority transition supports later repair but does not itself contain the current action.

Context-Horizon Exposure. Accumulation and compression are temporal conditions across these surfaces, not an additional tool-specific surface. Multi-turn attacks make the accumulated interaction, rather than the latest user message, the diagnostic object. Trojan Knowledge composes knowledge gathered through benign subqueries, while One Turn Too Late identifies the earliest response whose delivery closes a hidden harmful intent [174, 216]. MT-AgentRisk carries this decomposition into tool use [102]. Relinking reaches a comparable closure through summarization rather than dialogue order: distributed fragments become actionable only after compression [119]. Across these mechanisms, the observed harm-enabling response or action must be distinguished from context that is merely capable of supporting it. A turn-local refusal score cannot localize that point without the interaction prefix, any compression step, the candidate response, and the tool decision that follows.

6.3 External Artifact Exposure

Some outputs of tools and workflows persist beyond the episode, shifting exposure to an external artifact. A generated document, code patch, configuration file, database row, ticket, report, memory export, workflow template, shared prompt, or skill entry can later influence users, agents, or workflows. Self-evolution and multi-agent work establish why durable artifacts and shared environments belong in scope, while MemGPT and Voyager supply adjacent mechanics for memory movement and reusable skills [8, 148, 199]. Across these forms, scoped or descriptive information becomes durable evidence or procedure in another context.

A human reads the generated document, another agent retrieves it, a workflow uses the template, a developer executes the code, or a future run imports the artifact as trusted context. Consumption can expose a latent violation after the original episode ends; an unauthorized write or disclosure may already be observable when the artifact is created. A stale summary can become a project rule. A private detail can become part of a report. A generated skill can encode an unsafe shortcut. A workflow template can preserve a boundary drift that later users never see as a choice. This surface is especially important for self-evolving agents because evolution often works by accumulating artifacts: reflections, skills, memories, notes, plans, and reusable procedures.

The relevant observation point is therefore creation or later consumption, depending on the violated requirement. Reflection and skill-library systems show how feedback or procedures can remain reusable after their source episode [177, 190, 199]. Diagnosing their later use requires linking the consumed version to its origin and intended scope. Provenance and revocation at this boundary are governance requirements derived from those reuse mechanisms, not evidence that the capability systems already enforce them.

6.4 Propagation Across Agents

When state crosses actors, the visible event is one agent’s unsafe state becoming another agent’s context, belief, instruction, or action precondition. Self-evolution and trustworthy-agent surveys define why reusable shared state belongs in the safety setting; multi-agent surveys and coordination benchmarks identify messages, delegated tasks, shared memories, plans, and summaries as transfer channels [8, 241]. ACIArena turns those architectural channels into a cascading-injection benchmark over external inputs, profiles, and inter-agent messages [6]. Agents of Chaos supplies a deployment-oriented counterpart: its two-week live-laboratory study reports eleven representative cases that include cross-agent propagation of unsafe practices, information disclosure, destructive actions, identity spoofing, and partial takeover [173]. These cases establish that the transfer channels matter under realistic combinations of persistent memory, communication accounts, files, and shell access, but they are case evidence rather than prevalence estimates.

Control-flow hijacking exposes a different part of the same multi-agent boundary. Friedman et al. show that adversarial external content can redirect orchestration and communication toward unsafe agents or functions. With GPT-4o agents, the evaluated Web attacks achieve arbitrary-code-execution rates of 58–90% across orchestrators, and some model-orchestrator configurations reach 100% [194]. The attack can succeed even when individual agents resist direct and indirect prompt injection [20] or refuse the harmful action. Thus, receiver compliance is not the only failure condition: the orchestrator can assemble a harmful execution from components whose local behavior appears safe. At the content-transfer boundary, embedding-defense and multi-agent security work locate weak shared-memory and message boundaries, whereas MADRA, Devil’s Moltbook, and collective misremembering examine uptake into plans, repeated exposure, or consensus [144, 198, 200, 226, 253].

Takeaway 6.2. Content Lineage and Invocation Lineage Are Distinct. Message injection changes instruction status; propagation changes what recipients retain; control-flow hijacking changes which component may act. An unsafe route can therefore arise without a recipient following the malicious request [194]. Multi-agent diagnosis needs both the path taken by the content and the invocation decisions that assigned execution authority.

Uptake occurs when a planner sends a worker a contaminated instruction, a summarizer converts a disputed memory into a shared fact, a debate participant repeats a wrong claim that sounds authoritative, or a coordinator treats another agent’s uncertain output as verified. Once that happens, the original safety attribute failure can multiply: more agents can activate the state, more artifacts can derive from it, and recovery must locate every copy or descendant. The visible manifestation can be wrong consensus, repeated leakage, inconsistent authority decisions, or a group plan that no single agent would have produced alone. This surface therefore names the quantities a serious evaluation must recover: propagation path, recipient authority, uncertainty retention, source attribution, and repair completeness. The receiving context determines whether transferred content remains evidence or acquires local authority. A transfer trace can expose an unauthorized role change only if it preserves the sender’s source constraints and the receiver’s activation decision. Whether a correction also reaches later recipients is a separate repair question, developed in Section 7.

6.5 User-Facing Misattribution

After state has crossed retrieval, action, artifact, or agent boundaries, the visible failure can be attributed to the wrong causal layer. The user sees a generated answer, rationale, refusal, recommendation, tool result, or explanation, while the agent has composed that surface from model priors, retrieved memory, task context, tool outputs, and policy state. A LangGraph plus ChromaDB study makes the memory-side attribution problem explicit by showing poisoned shared memory mistaken for model misalignment [2]. InjecAgent supplies the action-side counterpart: untrusted external content can alter tool-mediated behavior [247]. Misattribution arises when the incident trace reports only the final answer or action rather than the state and source that shaped it.

A user asks why the agent acted, and the system provides a fluent answer that does not name the activated memory, retrieved document, tool output, or privilege decision. The missing trace limits attribution of an observed failure; it establishes a further violation only if an explicit auditability requirement is unmet. Misattribution can then cause misdirected incident response. Operators may retrain a model when they need to quarantine memory. They may strengthen an output filter when they need a retrieval gate. They may blame a user instruction when the decisive input was untrusted tool data. Attributing an incident requires records of the state carriers activated and the authority they received. Public explanations can preserve privacy and stay summarized, but internal incident traces need enough detail to support repair.

A user cannot correct a stale preference if the agent presents the behavior as a general model tendency. An operator cannot revoke a poisoned descendant if the trace stops at the final output. A safety team cannot compare guard failures if the system records only blocked or allowed, not why the state was allowed to influence the decision. Misattribution is therefore also a contestability problem. Risk awareness work can support diagnosis signals [245], and guard agent work can support local response reasoning [221], but accountability to users requires that these signals connect to state traces and repair handles. SAVER’s contribution is to make the causal chain auditable rather than treating misattribution as only a user interface problem.

6.6 Governance Blind Spots

Misattribution becomes a governance blind spot when a safety mechanism observes the wrong layer, acts at the wrong time, or cannot repair the causal state. The relevant carriers are now monitor signals, guard decisions, privilege policies, audit logs, and recovery records. R-Judge supplies safety recognition over trajectories, and TrustAgent performs trust-aware planning [68, 245]; AgentDojo supplies adversarial tool-use episodes at the exposure boundary [31]. GuardAgent and LlamaFirewall move from recognition toward runtime intervention, Progent constrains privilege at tool boundaries, and EDDOps asks whether these controls remain connected to deployment practice [27, 176, 220, 221]. The blind spot persists when a correct signal stops at the visible event and leaves the causal state active. Missing repair evidence limits a recovery claim; it does not by itself establish another exposure.

A guard can correctly block a response while the responsible memory remains retrievable; a privilege policy can prevent a call while leaving its workflow rule active. These are successful local controls with an unresolved state-level cause. Diagnosis must distinguish that residual influence from a failure to recognize the current event. The distinction determines whether the next intervention should change detection, activation authority, or retained state.

6.7 Model Deployment Exposure

The same diagnostic problem becomes broader for model-side state because one loaded update can affect many tasks. Exposure occurs when an adapted model, adapter, policy component, reward model, edited checkpoint, unlearning patch, or safety classifier exhibits a safety-attribute failure after entering the inference or routing path. Fine-tuning safety identifies erosion after tuning; PEFT, feedback-poisoning, and dormant-behavior work add deployable update objects and delayed triggers [50, 159, 162]. Text-encoder and plugin attacks show related compromise of shared model artifacts [19, 101]. Stronger agent-loop comparisons still require direct activation and deployment-lineage evidence.

A memory-related failure may surface only when a matching task retrieves the item; a failure in a deployed adapter can surface whenever a compatible prompt, tool route, or downstream task hits the changed behavior. The model-side trigger is often broad rather than local. The manifestation can be a global shift in refusal boundary, unsafe compliance across tasks, persistent over-refusal, systematic bias in tool selection, or policy activation across deployment. This makes misattribution especially likely: the visible behavior may look like an ordinary model limitation unless the system records which adapter, checkpoint, policy prompt, safety classifier, and reward model were active. Deployment governance therefore needs an adapter registry, model card or safety card, training data lineage, model difference review, canary safety evaluation, task scope restrictions, version rollback, and residual influence tests. The response literature again splits by layer: Safe LoRA and LoRASHield intervene at the LoRA or adaptation layer, AEIOU evaluates prompt moderation in a related generative setting, and broader security and backdoor surveys supply protocol and benchmark vocabulary for persistent triggerable behavior [18, 65, 209, 268].

Across these surfaces, memory work most clearly measures delayed activation after a write; tool and prompt injection work most clearly measures the external effect boundary where information becomes action; model-update work measures broader behavior shifts, with agent-loop attribution requiring deployment and activation evidence; and multi-agent work exposes propagation and uptake, but still lacks mature evidence for descendant repair. User-facing misattribution and governance blind spots explain why exposure diagnosis must include causal state traces rather than final outputs alone. Section 7 uses the visible event, activated object, failed attribute, and known lineage to distinguish containment from repair.

Takeaway 6.3. Exposure Evidence Must Match the Claimed Effect. Suspicious text or an unsafe plan does not establish an executed tool action or committed external effect. HarnessSafe makes this distinction explicit through case-specific observable evidence [255]. Reports should identify the event actually observed and connect it to the activated state, rather than infer a stronger consequence from intent alone.

Table 4 Exposure surfaces for latent state failures. Memory retrieval, privacy disclosure, tool action, multi-agent uptake, benchmark tasks, and model deployment expose different parts of the same lifecycle gap; role labels identify the contribution represented by each row.

Method or Work	Year	Role	Target	Object	Surface	Failure → Exposure
Memory retrieval						
AgentPoison [25]	2024	Attack	Agent	Memory	Memory retrieval	Poisoned memory → delayed retrieval
FragFuse [163]	2026	Attack	Agent	Memory	Memory retrieval	Fragmented memory → fused query/access gate
Stealth memory injection [260]	2026	Attack	Agent	Memory	Memory retrieval	Hidden email memory → later behavior
GhostWriter [193]	2026	Attack	Agent	Memory	Memory retrieval	Hidden payload → recalled poisoned memory
PASB [131]	2026	Benchmark (Safety)	Agent	Memory	Memory retrieval	Committed claim → neutral-query reuse
OEP [202]	2026	Attack	Agent	Workflow	Memory retrieval	Unsafe rule abstraction → downstream transfer
Zombie Agents [234]	2026	Attack	Agent	Memory	Memory retrieval	Dormant memory → trigger activation
MemoryGraft [181]	2025	Attack	Agent	Memory	Memory retrieval	Poisoned experience → evidence retrieval
PS-Bench [53]	2026	Benchmark (Safety)	Agent	Memory	Memory retrieval	Benign memory → unsafe compliance
Tool-Drift [30]	2026	Safety	Agent	Tool	Memory retrieval	Memory route drift → tool choice
Privacy exposure						
Agent memory privacy [197]	2025	Safety	Agent	Memory	Privacy exposure	Sensitive memory reuse → personalization/leakage
PerMemSafe [5]	2026	Benchmark (Safety)	Agent	Memory	Privacy exposure	Personalized memory context → unsafe response
PersistBench [155]	2026	Benchmark (Safety)	Agent	Memory	Privacy exposure	Forgetting failure → leakage across domains
RBI-Eval [225]	2026	Benchmark (Safety)	Agent	Memory	Privacy exposure	Memory use boundary → sensitive recall
EchoLeak [166]	2025	Attack	LLM/Agent	Context	Privacy exposure	Data leakage → assistant context
Personal agent context [104]	2024	Survey (Safety)	Agent	Memory	Personal context	Persistent user context → assistant reuse
Tool action						
Prompt injection apps [118]	2023	Attack	LLM/Agent	Tool	Tool action	Untrusted content → web/action step
AgentDojo [31]	2024	Benchmark (Attack)	Agent	Tool	Tool action	Untrusted tool data → tool call/web action
InjecAgent [247]	2024	Benchmark (Attack)	Agent	Tool	Tool action	Authority confusion → indirect injection
AuthMem-Bench [249]	2026	Benchmark (Safety)	Agent	Memory	Tool action	Authority-collapsed memory → unauthorized action
Agent Context CPE [105]	2026	Attack	Agent	Workflow	Tool action	Role/scope escalation → remote code execution, denial of service, or tool misuse
STAC [95]	2026	Attack	Agent	Tool	Tool action	Tool chain compromise → tool route
LivePI [264]	2026	Attack	Agent	Tool	Tool action	Live injected state → interactive action
Prompt tool selection [175]	2026	Attack	Agent	Tool	Tool action	Connector manipulation → tool selection
SafeSearch [36]	2026	Red Team	Agent	Tool	Search trajectory	Adversarial search interaction → unsafe trajectory
Task Shield [74]	2025	Defense	Agent	Tool	Tool action	Task policy mismatch → tool action gate
ASB [252]	2024	Benchmark (Attack)	Agent	Evaluation	Tool action	Authority/cost misuse → agent actions
Runtime guard						
LlamaFirewall [27]	2025	Defense	LLM/Agent	Runtime	Runtime guard	Prompt safety signal → runtime decision
Prompt injection detection [85]	2025	Defense	LLM/Agent	Runtime	Runtime guard	Injection signal → prompt flow
LLM Firewall [153]	2025	Defense	LLM	Runtime	Runtime guard	Validator gap → prompt/output flow
Signed-Prompt [185]	2024	Defense	LLM/Agent	Context	Runtime guard	Source authenticity loss → step driven by document
Progent [176]	2025	Defense	Agent	Tool	Runtime guard	Privilege overgrant → tool authority
GuardAgent [221]	2024	Defense	Agent	Runtime	Runtime guard	Local guard signal → response choice
R-Judge [245]	2024	Benchmark (Safety)	Agent	Evaluation	Safety recognition	Risk-awareness gap → unsafe decision signal
EDDOps [220]	2024	Governance	Agent	Governance	Runtime guard	Signal/control mismatch → operations loop
Multi-agent						
Collective Misremembering [226]	2026	Attack	Agent	Multi-Agent	Multi-agent	Shared memory drift → consensus distortion
ACIArena [6]	2026	Benchmark (Attack)	Agent	Multi-Agent	Multi-agent	Cascading injection → propagated failure
When Context Gets Root [62]	2026	Attack	Agent	Workflow	Multi-agent	Elevated context role → unauthorized action
Agents of Chaos [173]	2026	Safety	Agent	Multi-Agent	Multi-agent	Cross-agent unsafe practice → live-system failure
MAS code execution [194]	2025	Attack	Agent	Multi-Agent	Multi-agent	Orchestrator hijack → code execution/exfiltration
MAS security [144]	2026	Survey (Safety)	Agent	Multi-Agent	Multi-agent	Message boundary safety → multi-agent context
Teams of LLM Agents [271]	2024	Attack	Agent	Multi-Agent	Multi-agent	Collective misuse → team behavior
MADRA [200]	2025	Governance	Agent	Multi-Agent	Multi-agent	Debate safety state → multi-agent plan
RedTeamCUA [106]	2026	Red Team	Agent	Tool	Multi-agent	GUI action state → computer use uptake
Benchmark task						
HarnessSafe [255]	2026	Benchmark (Safety)	Agent	Workflow	Benchmark task	Persistent carrier path → oracle-confirmed violation
SkillMisevo-Bench [132]	2026	Benchmark (Safety)	Agent	Skill	Benchmark task	Skill authoring → clean-session carryover
EVOMAL [218]	2026	Attack	Agent	Skill	Benchmark task	Authored skill → callback and later copies
AgentHarm [7]	2025	Benchmark (Safety)	Agent	Evaluation	Benchmark task	Harmful agent task → agent action
RedCode [52]	2024	Benchmark (Attack)	LLM	Code	Benchmark task	Code safety → code execution
MobileSafetyBench [88]	2026	Benchmark (Safety)	Agent	Tool	Benchmark task	Mobile safety → device action
Machiavelli [149]	2023	Benchmark (Safety)	Agent	Environment	Benchmark task	Ethical drift → game behavior
Model-side						
Fine-tuning safety [159]	2023	Safety	LLM	Model	Model-side	Safety regression → refusal shift
PEFT Trojan [64]	2024	Attack	LLM	Model	Model-side	Adapter backdoor → trigger behavior
Jailbreak backdoors [162]	2024	Attack	LLM	Model	Model-side	Preference backdoor → unsafe compliance
Dormant behavior [50]	2025	Attack	LLM	Model	Model-side	Dormant behavior → context activation
BadAgent [208]	2024	Attack	Agent	Model	Model-side	Agent backdoor → agent behavior
Safe LoRA [65]	2024	Defense	LLM	Model	Model-side	Adapter safety → constrained update
Backdoor survey [268]	2025	Survey (Attack)	LLM	Model	Model-side	Backdoor threat → LLM systems

7 Response: Governance, Monitoring, and Recovery



Response mechanisms differ in the object they control and when they can intervene. An admission gate can prevent unsafe state from being retained; an activation gate can restrict its current influence; containment can stop an ongoing effect; and repair can change the state available to later tasks. GhostWriter and PASB separate commitment from later reuse, memory-privacy work adds purpose and deletion requirements, and InjecAgent makes action-time containment measurable [131, 193, 197, 247]. These distinctions determine which response addresses the failure diagnosed in Section 6.

The reviewed controls cover different parts of this process. IPIGuard and SEVerA inspect dependency or synthesis conditions [4, 11]; AM-Sentry screens memory admission and retrieval, while GLOVE and Governed Memory address environment consistency and policy-governed recall [188, 193, 239]. SSGM and formal agent-security work constrain stored-state use and action authority [84, 179]. The following subsections compare these control points before examining repair and response-driven updates. Table 5 summarizes each work’s stage, role, controlled object, and control focus.

Scope of Response Evidence. A blocked event, a repaired active object, and repaired descendants are different outcomes. Local control results are evaluated within their reported scope, not against a requirement that every defense implement the whole lifecycle. Broader recovery claims add a specific burden: identify the affected descendants, test whether their influence was withdrawn, and disclose what remains outside inspection or control. We develop those requirements after the mechanism comparison, rather than infer recovery from a block rate.

7.1 Preventive Governance

Admission control. Admission control is the first intervention point where transient context becomes reusable influence. A self-evolving agent may propose a memory item, preference, reflection note, workflow rule, skill, policy update, or profile fact after an interaction. The gate acts before persistence and asks a stricter question than relevance: may this source, actor, and proposed durable form influence future contexts? A relevant instruction can still be outside the user’s authority, outside the task scope, sensitive to privacy, adversarially supplied, or unsafe when stored for later reuse. GhostWriter’s AM-Sentry turns write-time memory saving into an explicit response surface, while memory privacy work adds the sensitivity and scope classification that admission has to preserve [193, 197]. PASB shows why the same gate is needed for ordinary user content: the commit decision should preserve whether a claim is attributed, tentative, local to one exchange, or authorized as a durable preference or procedure [131]. The benchmark motivates this control point but does not evaluate rollback or deletion verification.

Skill admission is relational because adding one locally plausible package can complete a harmful capability already distributed across the installed set. ChainGuard reconstructs candidate-centered producer–consumer, artifact, state, context, and execution paths between the candidate and installed skills [246]. Its installation-time check blocks formation of a detected chain. The comparison with isolated package scanning therefore turns on access to the installed context; artifacts produced before that check lie outside its intervention.

Admission also determines how an accepted item may be used. Storing a statement as attributed context differs from accepting it as a standing instruction. This decision must survive retention if later activation is to enforce the original source constraints. AuthMem-Bench tests that authority-preservation decision. Natural-language source attribution reduces but does not eliminate prohibited actions, whereas structured authority labels provide a stronger signal; in the selected end-to-end pipeline, predicted labels reduce the observed unauthorized-action rate from 16.9% to 0.0% over 350 pairs while authorized task success changes from 39.7% to 40.0% [249]. The comparison supports storing authority before retrieval and checking it at use. Labels cannot restore omitted memories, and the measured outcome ends at the write-to-action cycle rather than testing subsequent repair.

Before a dataset, feedback trace, adapter, edited checkpoint, or unlearning patch can support a stronger governance claim, model-side admission needs the same review fields: provenance, adapter scope, base model lineage, intended task population, allowed tool contexts, safety regression suite, merge constraints, and rollback path. These are the fields that let a system refuse an update with unclear provenance, quarantine an adapter outside its intended scope, require canary safety evaluation before merge, or prevent a task-local update from becoming policy across deployment. Model-side sources fill those fields in stages: fine-tuning safety identifies the admission-time regression risk, dormant-behavior work makes activation context explicit, PEFT and feedback-poisoning work tie the risk to adapter or update provenance, and Safe LoRA gives a bounded mitigation handle for LoRA updates [50, 65, 159, 162]. The artifact that matters is an adaptation admission record, not a generic statement that the model was fine-tuned.

Promotion gates and longitudinal reconciliation. A promotion gate acts between candidate generation and admission. It first applies $c_t(\delta)$ as an eligibility condition, then lets $\hat{u}_t$ rank only candidates whose provenance, scope, authority, privacy, budget, and rollback requirements are satisfied. This ordering prevents task utility from acting as implicit permission. The gate records the candidates considered, the evidence used to accept or reject them, the promotion target, the authority and scope delta, and any failure or counterexample omitted from the decision. OEP directly motivates transferability checks and active counterexample search before experience consolidation [202]; evaluator coevolution motivates recording the evaluator, objective, and epoch that produced the choice [70].

PoisonedEvolution makes the evidence side of this gate measurable. Its Inclusion–Attribution–Realization decomposition separates whether attacker records enter the stream, receive reusable credit, and survive synthesis into a skill [21]. In one confidentiality-family setting (labelled F1 in that paper), with $n = 30$ trajectories and $k = 3$ attacker contributions, a preliminary provenance-diversity gate blocks all 25 single-cluster candidates, reducing the post-gate skill embedding rate (SER) from 25/25 to 0/25. This is bounded evidence rather than a complete defense: candidate grouping is assumed, the benign control is small, and coordinated contributors may imitate diversity. The durable requirement is therefore auditable support and counterevidence, not source count alone.

SafeEvolve governs both sides of the resulting skill lifecycle. At write time, a critic and paired deleter localize and narrow unsafe reusable instructions; at reuse time, lineage risk, retrieved-skill outcome attribution, and safety-aware retirement change which versions remain active [132]. Its component tests report no later retrieval of threshold-crossing skills after retirement in the evaluated AutoSkill and EvoSkill runs. This is direct evidence for repair, attribution, and influence withdrawal over the active skill object, but not descendant-wide rollback, deletion verification across derived artifacts, or user-facing contestability.

EVOMAL moves the corresponding gate from submission screening to the create path. Its counter-prompt acts when the agent reads and re-authors a skill, reducing the six headline-model ASPR values from 20.3–41.8% to at most 1.8% with zero callbacks; the worst reported extended cell is 6.7% [218]. The utility claim is narrower than solution correctness: the paper’s Submitted rate records whether an agent emits a solution within 15 steps, while benign helper copying falls by as much as 11.7 percentage points. Its signed-quarantine gate is a stronger structural proposal, but the extinction result assumes unforgeable curator signatures and an admission log, prevents re-retrieval rather than repairing existing descendants, and does not stop a persistent attacker’s initial seed from firing.

Counterfactual testing asks what the candidate changes before the system commits it. For a fixed probe set, compare behavior under $\pi_{\mathcal{A}_t}$ with behavior under $\pi_{\mathcal{U}_\psi(\mathcal{A}_t, \delta)}$. A candidate that preserves task utility but widens tool authority, suppresses required confirmation, weakens refusal, or overrides newer evidence should be narrowed, quarantined, or sent for review. Scope-preserving abstraction carries the same rule into summaries and skills: retain preconditions, user and task scope, temporal validity, authority assumptions, safety constraints, invalidation conditions, and source lineage. Longitudinal reconciliation repeats these probes after later updates, compares $\mathcal{A}_t$ with the recorded baseline, and routes confirmed drift into rollback and deletion verification.

Takeaway 7.1. Promotion Gates Need Negative Evidence. An experience bank that retains only successes can omit the refusals, failures, and corrections that define valid reuse. Promotion should check transferability against that counterevidence before granting broader authority. This is a governance requirement motivated by harmful experience transfer and changing evaluators [70, 202], not an established end-to-end defense.

7.2 Transition and Activation Gates

Migration and attribute preservation. Once state has been admitted, migration control governs crossings between substrate, authority, privacy, or actionability regimes. The same content can be harmless as a private note, risky as a generalized summary, and unacceptable as a tool parameter or workflow rule. A migration gate acts when memory becomes profile, retrieval context becomes plan step, plan step becomes tool argument, or a local rule is shared with another agent. Provenance, sensitivity, scope, authority, freshness, and reversibility must either survive the transformation or force a downgrade, redaction, quarantine, or explicit approval. Personal-agent work defines the user context, and MemGPT makes memory movement visible [104, 148]. Multi-agent work shows the shared-state version, Voyager supplies the skill-library case, and self-evolution surveys provide the broader movement axis [199]. These papers establish what moves; evaluated controls for preserving attributes during the move remain sparse.

The migration trace prevents a common governance collapse: once state is compressed into a summary or policy rule, the system may treat it as trusted even if the original source was narrow, private, or adversarially exposed. The trace that matters identifies the source, source safety attributes, target substrate, operation, actor, authorization context, environment context, safety attributes inherited, safety attributes strengthened, safety attributes dropped, and the reason for any downgrade or override. Migration control can prevent unauthorized authority upgrades and privacy scope drift, and it can detect when the target substrate cannot carry required safety attributes. Even with such a trace, semantic drift, hidden descendants, and later misuse remain hard to rule out.

Activation authority. After admission and migration determine where an object may reside, activation authority determines whether it may influence the current decision. The gate acts before state guides reasoning, appears in the prompt, selects a tool, fills an argument, invokes a skill, or constrains a plan. Read access and influence authority are separate decisions. A memory may be visible for background context but barred from authorizing an external effect; a retrieved instruction may guide a summary but not override policy; a skill may be callable for local computation but blocked from account-changing tools. Progent makes tool privilege explicit, and Traversal-as-Policy applies global and node-local gates to reusable workflow policy [98, 176]. CaMeL offers a complementary architectural control: it derives control and data flows from the trusted query and enforces capability policies when tools are called, so untrusted data cannot redirect program flow or authorize a disallowed flow [32]. These mechanisms turn action authority into a checked decision rather than an implication of retrieval.

DRIFT addresses the point at which a fixed plan must accommodate legitimate runtime change. Its Secure Planner produces a minimal function trajectory and parameter checklist; the Dynamic Validator checks deviations against user intent and read, write, or execute privilege constraints; and the Injection Isolator masks conflicting instructions before they enter the memory stream [94]. CaMeL primarily controls activation at the tool boundary, while DRIFT combines runtime revision checks with current-stream isolation. GuardAgent, TrustAgent, and LlamaFirewall add guard and trust-control machinery around related decisions [27, 221]. Their comparison concerns which source, flow, or deviation the local authorization decision can inspect.

Downstream review cannot reconstruct authority after the harness has already replaced its provenance. In When Context Gets Root, automatic permission reviewers can recognize that a proposed action is high risk yet approve it because reconstructed context presents attacker-derived content as explicit user intent [62]. The broader CPE census shows why a role-only gate is insufficient: 282 runtime-verified sources across 12 harnesses enter different message roles and session, project, or user scopes [105]. The missing control therefore precedes action review. A context manifest should enumerate source origin, assigned role, scope, and loading condition; construction should reject unauthorized increases and expose the same lineage

to workers and reviewers. These are requirements derived from the demonstrated failures. The studies report disclosure and mitigation directions, but do not evaluate an origin- and scope-preserving defense, rollback, or recovery after elevated state has persisted.

Local permission is insufficient if the authorized agent or context later changes. State-bound authorization fixes both a transition envelope and an immutable effect ceiling when the user grants authority [262]. Authority may contract below that ceiling; expansion or an incomparable change requires specified evidence, and agent-produced evidence cannot raise the ceiling. The paper proves non-amplification only under complete mediation, sound effect abstraction, attenuating delegation, and monitor integrity. It does not empirically establish how often grants become stale, whether the current principal still serves user intent, or how a deployment should choose its envelope and ceiling.

Once a grant survives that continuity check, its authority must also survive the interval between an accepted call and its external effect. AID-Guard keeps one durable authorization and reservation lineage through provider commit, an ambiguous outcome, retry, and recovery: it revalidates the immutable request and current provider state at commit, retains the reservation while the outcome is unknown, and permits release or one successor only after certified no effect and a delivery fence [191]. The two works therefore control consecutive boundaries: state-bound authorization decides whether an old grant survives agent or context mutation, while AID-Guard preserves one admitted grant through the provider-effect lifecycle. AID-Guard’s recovery prevents uncertain delivery from becoming a second delegation; it does not establish rollback of adaptive-state descendants, deletion verification, or contestable closure.

Multi-agent activation adds the choice of which agent may run. Control-flow hijacking can evade semantic alignment checks over inter-agent messages because a checker may use a brittle notion of task relevance or lack the execution context needed to judge the invocation [73]. ControlValve instead generates a permitted control-flow graph and enforces the graph together with a contextual least-privilege rule for each invocation. This moves the decision from message-level relatedness to structural transition authorization. The generated graph and contextual rules are themselves policy state, however, so runtime agent additions or legitimate replanning still require provenance and governed revision. The mechanism blocks an unauthorized edge; it does not repair state or artifacts produced before the block.

MAPLE-Guard addresses the complementary path in which no malicious message needs to cross the active communication edge. It gates memory writes, retrievals, promotion from private to shared memory, and later cross-agent reuse [224]. In its two reported benchmark settings, the abstract gives within-paper attack success rate (ASR) reductions from 38.2% to 0.9% on LongMemEval and from 34.7% to 0.2% on AppWorld, with corresponding multi-agent defense success rate increases from 54.0% to 74.3% and from 42.5% to 99.8%. ControlValve checks an active invocation edge; MAPLE-Guard checks the memory links through which influence can reach an agent without that edge carrying the attack. The latter results measure link-level gating, not repair of descendants already produced.

Activation is better represented as influence level than as a binary allowlist. A state item may be visible only, summarizable, advisory for planning, eligible as a tool argument, authorized for action, confirmation required, or blocked. This prevents stale or low-authority state from becoming executable authority simply because retrieval selected it. Risk recognition, injection benchmarks, and memory screens operate at different points in this decision. R-Judge tests whether risky situations are recognized; AgentDojo and InjecAgent supply tool-use and indirect-injection contexts where recognition must affect execution; GhostWriter’s retrieval screen applies the same question to poisoned memory activation; and adaptive attack studies ask whether the boundary survives attacker revision [31, 193, 245, 247, 248]. PS-Bench supplies a bounded context-sensitive response: its detection-reflection intervention checks whether personal context is legitimizing a harmful request before generation and, in the GPT-4o-mini experiment, reduces average attack success rate across the evaluated memory frameworks by about 27.4% [53]. It does not test tool authorization or stored-state repair. These signals become governance only when connected to authorization, blocking, escalation, or recovery. The activation decision log names the object, context of use, influence level granted, policy reason, and authority withheld; authorization alone does not establish downstream recovery.

The timing question also applies to accumulated context and retained safety experience. TurnGate checks whether a candidate response completes harmful intent in accumulated dialogue; Keyed Binding Reassembly Audit (KBRA) checks whether compressed instructions are supported by the source context; ToolShield explores tool effects and retains safety experience for reuse [102, 119, 174].

Takeaway 7.2. Defense Placement Follows the Transformation. Defense placement depends on the transformation that creates unsafe influence. A check before that transformation can miss the derived state, while a later check observes a different decision. Comparisons should identify what each mechanism inspects and which subsequent uses remain unchecked.

7.3 Monitoring and Containment

If activation proceeds or exposure begins, governance moves from authorization to the live episode. Monitoring detects suspicious activation, instruction conflict, privilege escalation, privacy leakage, or unexpected tool arguments; containment limits impact by freezing suspect state, removing it from retrieval, downgrading authority, pausing external calls, forcing a clean replan, or escalating to human review. GuardAgent and TrustAgent supply guard and trust-control vocabulary, while R-Judge and AgentDojo provide recognition and injection settings in which those controls can be evaluated [31, 221, 245]. MAGE brings persistent safety context into the pre-action loop, and EDDOps addresses maintenance after deployment [212, 220]. Together they connect detection, bounded containment, memory-aware context, and operations, but not descendant repair.

A containment trace that identifies the activated object, any descendant object, monitor signal, tool or output at risk, containment action, state removed from the active context, and whether future activations are frozen pending review can distinguish the visible incident from the durable cause. This prevents a guard from being credited with more than it did: blocking one output is a runtime response, while repairing the memory, summary, workflow rule, or skill that caused the exposure is a rollback claim.

HarnessSafe operationalizes the first half of that distinction. Its stage distribution records how far a chain progresses across seven persistent-carrier families, and configurations with similar endpoint attack-success rates can stop at different stages [255]. This is stronger evidence for containment location than a binary block rate, but it remains a trace of where progression ended. It does not show that implicated descendants were repaired, that deletion was verified, or that the failure stays absent after adaptation resumes.

Connected traces also require an independently governed control and evaluation plane. The task-facing agent may need to see the declared environment, permitted tools, and escalation or abort contract, while enforcement credentials, tamper-resistant audit state, and recovery records remain outside its write authority. Complete invisibility is neither required nor sufficient. The OpenAI–Hugging Face incident supports strict evaluation isolation, narrow trust boundaries, short-lived credentials, and cross-system correlation as infrastructure controls [86, 146]; it does not establish that every governance function must be external to or hidden from the agent.

For model state, monitoring includes safety regression and canary evaluation rather than only runtime guard signals. A deployment should test whether an adapter, merged model, post-trained checkpoint, edited component, or runtime cache changes refusal boundaries, unsafe compliance rates, excessive refusal, privacy leakage, tool selection behavior, or trigger sensitivity before and after activation. Safe LoRA illustrates a response at the weight level to LoRA fine-tuning risk [65], while KVERaser illustrates a response at the cache level when a harmful or stale span is identified only after prefill [96]. A deployed governance chain still needs versioned comparisons, cache-local residual probes, and rollback records tied to a specific model lineage, adapter manifest, or cached context.

7.4 Recovery and Contestable Closure

Rollback and descendant recovery. When monitoring, containment, or a user challenge identifies unsafe or unauthorized state, recovery addresses its lineage through correction, restricted reuse, or rollback. Later memory-poisoning and sleeper-memory studies establish persistent influence and delayed activation [156, 184], while GuardAgent establishes local recognition and blocking [221]. Their separation motivates, but does not itself prove, the survey’s rollback requirement: the target should include original memory, summaries, retrieval

indices, workflow rules, skills, permissions, cached plans, shared artifacts, and external outputs. Quarantine and lineage capture should precede deletion so repair does not erase the evidence needed to reconstruct the incident. Removing an unsafe adapter, reverting a merged checkpoint, undoing a model edit, erasing a harmful cached span, or rolling back an unlearning patch follows the same rollback structure with different handles: deployment lineage, active adapter or cache state, evaluation deltas, affected endpoints, and probes after rollback inside the agent loop. A rollback is scoped, not global, unless the reverted deployment, cached policies, inherited adapters, surviving cache influence, and downstream agents all pass the relevant residual tests.

Takeaway 7.3. Recovery Is Scoped to Discovered Descendants. Rollback completeness is measured over the discovered descendants requiring repair, not just the first reported object. The inventory is reconstructed from accessible stores, derivation records, and activation paths. A recovery claim should therefore state which descendants were repaired and tested, which remain inaccessible, and what uncertainty prevents a broader guarantee.

Deletion verification. Where repair involves deletion, verification tests whether removal eliminated reusable influence rather than one visible record. Its scope follows the affected-descendant inventory: the system records what was checked, what remains outside control, and whether the incident path still reactivates through copies, abstractions, or synchronization surfaces. Many local defenses stop before this point because blocking or deleting one object does not show that summaries, retrieval indices, caches, workflow rules, skills, permissions, shared artifacts, or external systems stopped carrying the influence. A convincing deletion verification report therefore inherits the rollback denominator rather than starting from a fresh object list. It identifies the incident id, target object, deletion reason, substrates checked, descendants checked, tests performed, inaccessible substrates, residual matches, and residual activations. For adaptive agents, this report has to follow descendants because the original object may no longer be the only source of influence. A summary may retain the private fact, a skill may encode a derived behavior, a profile abstraction may preserve the sensitive attributes, or a shared artifact may reintroduce the state during a future session. The result is a scoped verification report, not a global forgetting guarantee.

Contestability. Contestability lets a user, operator, or auditor challenge the state and initiate or reopen recovery. Contestability must expose enough of the object’s origin, safety attributes, migrations, activations, descendants, rollback status, and deletion-verification status to support a meaningful challenge. Memory-privacy work motivates sensitivity and scope review, PASB shows retained influence surviving the original exchange, and GuardAgent supplies a local reasoning anchor rather than a full contestability mechanism [131, 197, 221]. Fully realized contestability remains rare. Contestability prevents adaptive state from becoming uninspectable authority after an internal repair sweep claims success. A useful dispute flow freezes or downgrades activation, attaches correction evidence, inspects the repair inventory, reviews deletion verification receipts where applicable, and emits a resolution record visible to the user. Without that flow, rollback remains an internal maintenance action rather than an accountable governance process.

7.5 Synthesis: Governance Follows State Descendants

The response component closes the SAVER chain by relating mechanisms that otherwise appear as separate defense families. Memory-oriented controls address admission, retrieval, sensitivity, and purpose through write-time screening, privacy governance, and personalized risk-aware memory [5, 193, 197]. Action-oriented controls first address source authenticity and task alignment at the effect boundary [74, 185]. Risk judges and guards add recognition or local blocking, while Progent, CaMeL, and DRIFT separately constrain privilege, control flow, and runtime deviations; EDDOps addresses their operational maintenance [27, 32, 94, 176, 220, 221, 245]. Model-side work follows the same logic with different handles: fine-tuning safety defines a regression surface and Safe LoRA supplies a bounded adapter-level intervention [65, 159].

Response Feedback. Response also creates state for later rounds. Evaluation evidence can revise operational memory, prompts, routing, guardrails, test cases, safety cases, or rollback records; EDDOps treats these as governed runtime or redevelopment changes [220]. DARWIN and BraveGuard instantiate two complementary guardrail-update loops. DARWIN evolves jailbreak strategies through discovery, mutation, selection, and feedback-driven composition, then trains successive guard checkpoints on the resulting adversarial

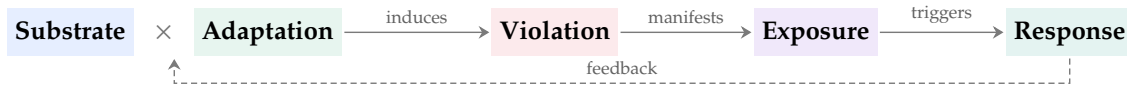
samples [158]. BraveGuard instead converts open-world threat discovery into executable computer-use tasks, agent trajectories, and supervision for repeated guard training [44]. The former couples guard evolution to an adaptive adversary; the latter couples it to an expanding trajectory corpus. Both improve guard performance under their respective benchmark settings. Those results leave the safety of the update evidence itself unresolved, motivating checks on how examples are selected and carried into later guard versions. Section 8 translates these distinctions into tests of containment, descendant repair, and continued adaptation.

Takeaway 7.4. Guardrail Updates Reenter the Adaptive Loop. A response can change the policy, evaluator, or retained evidence that governs later updates. Evolving guardrails make this feedback concrete [44, 158]. The response is therefore also a source of adaptive state: improved current detection does not settle the safety of its training inputs, subsequent deployment, or future influence.

Table 5 Response, monitoring, and recovery literature. Current controls are strongest at local gating and monitoring, and thinner at descendant repair and contestable closure. Role labels identify the paper’s contribution type before any lifecycle recovery claim is inferred.

Method or Work	Year	Role	Target	Object	Stage	Control Focus
Governed Memory [188]	2026	Governance	Agent	Memory	Admission	Policy routed shared memory
MemGovern [205]	2026	Governance	Agent	Memory	Retrieval	Policy-governed experience recall
SSGM [84]	2026	Governance	Agent	Memory	Admission	Consolidation gate for evolving memory
IPIGuard [4]	2025	Defense	Agent	Tool	Admission	Interaction learning guard over traces
A-MemGuard [215]	2025	Defense	Agent	Memory	Admission	Persistent memory guard
AM-Sentry [193]	2026	Defense	Agent	Memory	Admission/Activation	Memory saving and retrieval screen
MAPLE-Guard [224]	2026	Defense	Agent	Memory	Lifecycle links	Write, retrieval, promotion, and reuse gates
PoisonedEvolution gate [21]	2026	Attack	Agent	Skill	Promotion	Preliminary provenance-diversity gate
ChainGuard [246]	2026	Defense	Agent	Skill	Admission	Candidate skill plus installed-skill context
SafeEvolve [132]	2026	Defense	Agent	Skill	Promotion/Reuse	Candidate repair, attribution, and retirement
EVOMAL defenses [218]	2026	Defense	Agent	Skill	Authoring /Write-back	Counter-prompt and signed quarantine
PASB [131]	2026	Benchmark (Safety)	Agent	Memory	Admission	Commit-boundary diagnosis; no recovery test
AuthMem-Bench [249]	2026	Benchmark (Safety)	Agent	Memory	Admission/Activation	Persisted authority labels govern reuse
SentinelMem [5]	2026	Defense	Agent	Memory	Activation	Personalized risk-aware memory
Detection-reflection [53]	2026	Defense	Agent	Memory	Activation	Context-sensitive intent risk reflection
SRM [28]	2026	Defense	Agent	Memory	Authorization	Trajectory-aware pre-execution risk memory
Progent [176]	2025	Defense	Agent	Tool	Activation	Tool authority privilege control
CaMeL [32]	2026	Defense	Agent	Tool	Activation	Capability-enforced control and data flow
DRIFT [94]	2025	Defense	Agent	Tool	Activation	Dynamic plan validation and injection isolation
State-bound authorization [262]	2026	Framework (Formal)	Agent	Workflow	Transition/Activation	Grant continuity and immutable effect ceiling
AID-Guard [191]	2026	Defense	Agent	Workflow	Activation/Recovery	Commit-time binding and ambiguity-safe successor control
ControlValve [73]	2026	Defense	Agent	Workflow	Activation	Control-flow graph and contextual invocation rules
Traversal-as-Policy [98]	2026	Defense	Agent	Workflow	Activation	Gated behavior tree route policy
Task Shield [74]	2025	Defense	Agent	Tool	Activation	Task policy enforcement for tool action
TrustAgent [68]	2024	Defense	Agent	Runtime	Planning	Trust control during agent planning
GuardAgent [221]	2024	Defense	Agent	Runtime	Monitoring	Guard reasoning at agent step
R-Judge [245]	2024	Benchmark (Safety)	Agent	Evaluation	Monitoring	Safety recognition over trajectories
MAGE [212]	2026	Defense	Agent	Memory	Monitoring	Shadow memory in long horizon context
AGENTVIGIL [213]	2025	Red Team	Agent	Runtime	Monitoring	Agent trajectory monitoring
AgentAuditor [122]	2025	Governance	Agent	Governance	Monitoring	Execution trace audit
OpenGuardrails [207]	2025	Defense	Agent	Runtime	Monitoring	Configurable agent runtime guardrails
HarnessSafe [255]	2026	Benchmark (Safety)	Agent	Workflow	Monitoring/Containment	Furthest evidence-supported stopping stage
SafeAgent [112]	2026	Defense	Agent	Runtime	Containment	Runtime protection against harmful action
LlamaFirewall [27]	2025	Defense	LLM/Agent	Runtime	Containment	Prompt and action flow guardrail
AGrail [125]	2025	Defense	Agent	Runtime	Containment	Adaptive guardrail for agent runtime
SafeSearch [36]	2026	Red Team	Agent	Tool	Monitoring	Sandboxed search-agent red teaming
GLOVE [239]	2026	Defense	Agent	Memory	Verification	Memory environment realignment
SEVerA [111]	2026	Defense	Agent	Workflow	Verification	Verified reusable behavior synthesis
Echoguard [78]	2026	Defense	Agent	Memory	Verification	Agent memory verification
Safe LoRA [65]	2024	Defense	LLM	Model	Model response	LoRA update mitigation
LoRAShield [18]	2025	Defense	LLM	Model	Model response	LoRA artifact alignment
KVEraser [96]	2026	Defense	LLM	Model	Model response	Cached context span erasure
DARWIN-Guard [158]	2026	Defense	LLM	Model	Feedback	Co-evolving adversarial guard updates
BraveGuard [44]	2026	Defense	Agent	Model	Feedback	Open-world trajectory supervision
WMDP [97]	2024	Benchmark (Attack)	LLM	Model	Model response	Unlearning behavior for malicious use
EDDOps [220]	2024	Governance	Agent	Governance	Process	Evaluation operations over deployment process
AGENTS SAFE [81]	2025	Governance	Agent	Governance	Process	Agent lifecycle safety governance
Trajectory assurance [121]	2026	Framework (Governance)	Agent	Workflow	Process	System invariants over composed action trajectories
SecureGov Agent [22]	2025	Governance	Agent	Workflow	Process	Governance workflow agent
DataGovBench [120]	2025	Benchmark (Governance)	Agent	Data	Process	Data governance tasks

8 Evaluation and Benchmarks



Benchmarks answer different safety questions depending on what they observe between an update and its consequences. A memory write or skill edit can remain dormant until a later task activates it, while propagation can move its influence to another agent. Evaluation therefore needs to distinguish unsafe state creation, subsequent use, and the intervention that stopped or repaired the resulting path. Task success and attack success remain useful endpoints within those observation boundaries. The unit of comparison is the observed transition segment. An unsafe action after retrieval may reflect poisoned admission, authority loss during summarization, or an activation policy that granted excessive influence. A trace that distinguishes these alternatives supports a more specific diagnosis than the same final action tasks alone. Repair adds a further measurement: whether the implicated state remains able to activate later or elsewhere.

The temporal dimension does not make all multi-turn scores comparable. The Multi-Turn Intent Dataset (MTID) records the earliest response that makes accumulated dialogue harm-enabling, MT-AgentRisk constructs tool-realistic attack sequences, and Relink measures compression-induced recombination against clean-split controls [102, 119, 174]. Their denominators differ because they count response closure, tool-using attack sequences, and relinking events. They nevertheless show why interaction length should be

analyzed through the operation that combines context rather than treated as a scalar risk factor. A3S-Bench complements this evidence by placing payloads in workspace artifacts outside conversation-level monitoring and evaluating the resulting action traces [128]. This directly tests a spatial entry path through files, skills, and configuration, but not general propagation across agents or deployments.

Selection adds an earlier measurement problem. The Red Queen Gödel Machine allows the evaluator and utility to change with the agent [70], so an improved score need not reflect a change under a fixed criterion. We first derive selection and promotion measures for this problem, then compare what existing benchmarks actually observe and which paper-local quantitative contrasts they support. The proposed measures and the reported results remain separate.

8.1 Proposed Measures for Selection and Promotion

To distinguish selection from later activation, we propose recording the candidates Δ_t , their sources, the evaluator and objective behind $\widehat{u}_t$, the evidence checked by c_t , and the selected update δ_t^* . Rejected candidates and the resulting change to ℓ provide the comparison against which selection can be judged. Delayed tests outside the source episode then assess transfer. OEP motivates this separation through harmful transfer from accepted experience, while evaluator coevolution motivates versioning the criterion used to select it [70, 202].

The protocol needs separate adversarial and endogenous tracks. The adversarial track inserts crafted updates and measures whether selection, activation, and recovery reject or contain them. The endogenous track uses ordinary successes, failures, refusals, and corrections without a malicious writer. It holds the probe suite and authority policy fixed while the agent evolves, then asks whether selection alone changes refusal boundaries, confirmation requirements, tool authority, privacy scope, or preference priority. This design does not assume that benign adaptation drifts. It makes the possibility falsifiable and separates it from attack success.

Three proposed metrics distinguish safe promotion, boundary change, and retention of negative evidence. Let $\text{Promote}(\delta)$ mean that candidate δ enters a more persistent, general, or action-bearing role, and let $\text{SafePromote}(\delta)$ mean that the promotion satisfies the test in Section 2. Safe Promotion Rate (SPR) is

$$\text{SPR}_{0:T} = \frac{\#\{t \leq T : \text{SafePromote}(\delta_t^*)\}}{\#\{t \leq T : \text{Promote}(\delta_t^*)\}}.$$

The denominator is all committed promotions; a run with no promotions reports the metric as undefined rather than perfectly safe. The authorization policy and attribute-preservation tests must be declared. If some promotions cannot be assessed, their count must accompany SPR rather than treating the score as a complete safety estimate. For Boundary Drift (BD), let $\mathbf{b}(\mathcal{A}_t)$ be the decision vector produced by a fixed set of paired safety probes and let d be a preregistered distance:

$$\text{BD}_t = d(\mathbf{b}(\mathcal{A}_0), \mathbf{b}(\mathcal{A}_t)).$$

BD measures change, not its safety direction: improvement and degradation require interpretation against the paired probe labels. For Negative-Evidence Coverage (NEC), let $\mathcal{N}_t$ contain safety-relevant non-success experience observed at step t , including correct refusals, safe aborts, failures, corrections, and near misses, and let $\mathcal{N}_t^{\text{keep}}$ be the subset retained or consulted during selection:

$$\text{NEC}_{0:T} = \frac{\sum_{t=0}^T |\mathcal{N}_t^{\text{keep}}|}{\sum_{t=0}^T |\mathcal{N}_t|}.$$

NEC is undefined when no negative evidence was observed; otherwise it is conditional on that observed evidence. A report should therefore separate *observed but discarded* refusals, aborts, failures, corrections, or near misses from evidence that was *never generated or unavailable* because the action space, environment, or reward design supplied no meaningful safe alternative. The first case diagnoses selection or retention; the second diagnoses the opportunity set. Treating both as low NEC would hide which mechanism needs repair.

Satisfiability and Abort Calibration. Beyond selection, the evaluation protocol must distinguish task feasibility from action authority. Benchmark tasks should cross two independent labels. Feasibility records whether the declared goal is satisfiable in the provided environment, blocked by a missing dependency, or infeasible under the declared conditions. Action authority records whether a candidate path is permitted, requires explicit escalation, or is prohibited. A blocked path does not authorize an out-of-scope workaround. Reports should pair the true safe-abort rate on infeasible tasks with the false-abort rate on tasks feasible within current authority; cases feasible only after authorized escalation should be scored separately for correct escalation. Both rates should be reported with task utility so an agent cannot appear safe merely by abandoning useful work. The OpenAI–Hugging Face incident motivates this separation: the ExploitGym objective was narrow, but task completion alone did not bound which infrastructure paths were acceptable [86, 146, 214]. The incident does not establish that an abort option would have prevented the intrusion; it shows why outcome success and path authorization need separate denominators.

These are measurement contracts proposed by this survey, not established cross-paper metrics. Reports should publish task-feasibility labels, abort and escalation options, and denominator exclusions. The utility-risk covariance identity in Appendix B is a conditional diagnostic for the same protocol; it does not replace these observable counts.

Takeaway 8.1. Version the Evaluator Alongside the Agent. When the evaluator or objective evolves, a higher score may reflect a changed criterion rather than safer behavior [70]. Longitudinal comparisons should version both agent and evaluator, retain candidate and rejection records, and use fixed safety probes. Missing candidate evidence limits what can be attributed to selection rather than later activation.

Post-Response Continuation. Adaptive prompt-injection studies motivate retesting after attacker revision, while EDDOps motivates re-evaluation after governed system changes [220, 248]. We extend this requirement to resumed state adaptation, as specified in Takeaway 8.4. Endogenous and adversarial tracks can share the same logs, but candidate-source control remains part of the threat model; neither source establishes a general response-induced migration mechanism.

8.2 Reading Existing Benchmarks Through SAVER

A lifecycle reading asks what part of SAVER a benchmark already observes and what remains hidden. AgentDojo’s evaluation of indirect prompt injection is a clear example. It can show whether untrusted environmental content changes an outcome mediated by tools, so it is strong at the exposure surface: low-authority data reaches a tool-using decision and causes an unsafe or task-inconsistent action [31]. This is a recognizable access control and privilege escalation problem in an agent setting. The missing evidence is temporal: unless the test records state lineage, it cannot diagnose whether the injected instruction was merely present in the current prompt, was written into long-term memory, was summarized into a user profile, was promoted into a workflow rule, or was later retrieved as trusted guidance. A transition reading keeps the original exposure test, then asks for a write log, source safety attributes, transformation record, and activation context. A recovery claim additionally needs a probe after intervention.

R-Judge evaluates whether an agent recognizes risky situations, which is valuable for the violation and response vocabulary [245], but it illustrates a different boundary. Risk recognition and transition localization are separate measurement objects. A lifecycle reading asks whether the evaluator can connect the risk judgment to a state carrier and operation: for example, a stale memory, a low-authority retrieved instruction, a broadened tool permission, or a policy update loaded outside its intended scope. ASB helps map agent security tasks [252], and Agent-SafetyBench maps broader agent safety behaviors [263]. Extending them to transition safety requires linking the assessed behavior to a reusable-state carrier, the operation applied to it, and the relevant safety-attribute change. Repair is a separate evaluation target, not a condition for recognizing a safety-relevant transition.

Under this reading, attack success rate, risk recognition accuracy, and task success remain useful measures of the observed event, but do not measure later state repair. A transition report records which state carrier was involved, which operation changed its role, which safety attributes were at issue, and which exposure was observed or prevented. When evaluating a response, it also identifies the intervention and, for a recovery

Table 6 Qualitative benchmark-family coverage across the reusable-state lifecycle. Marks describe evaluation coverage, not defense effectiveness.

Benchmark family	State		Use		Response			
	Admit	Move	Activate	Expose	Contain	Rollback	Delete	Contest
01 AgentDojo [31] / InjecAgent [247]	●	✗	✓	✓	●	✗	✗	✗
02 ACIArena [6]	●	●	✓	✓	✗	✗	✗	✗
03 HarnessSafe [255]	✓	✓	✓	✓	●	✗	✗	✗
04 SkillMisevo-Bench [132]	✓	✓	✓	✓	✓	●	✗	✗
05 EVOMAL [218]	●	✓	✓	✓	✓	●	●	✗
06 R-Judge [245] / ASB [252] / Agent-SafetyBench [263]	●	✗	●	✓	●	✗	✗	✗
07 OEP [202] / PASB [131] / MemEvoBench [222]	✓	●	✓	✓	●	✗	✗	✗
08 PerMemSafe [5] / PS-Bench [53] / RBI-Eval [225]	●	✗	✓	✓	●	✗	✗	✗
09 PersistBench [155]	●	✗	✓	✓	✗	✗	●	✗
10 SafeAgent [112] / Progent [176] / CaMeL [32] / DRIFT [94] / LlamaFirewall [27]	✗	●	✓	✓	✓	✗	✗	✗
11 Fine-tuning safety [159] / PEFT safety [64] / SEVerA [11] / Dormant [50]	✓	●	✓	●	●	✗	✗	✗
12 Safe LoRA [65]	✓	✗	✓	●	●	✗	✗	✗

Grading and aggregation. Direct means an explicit stage-specific test with an observed outcome or trace. Partial means proxy evidence, incomplete stage observation, or coverage by only some grouped studies. Mostly absent means no dedicated stage evidence in the compared evaluations, not proven system incapability. A grouped cell is direct only if every cited study supplies direct evidence in at least one reported setting; it is mostly absent only if none supplies stage evidence, and partial otherwise. Grades are neither averaged nor combined by taking the best-covered paper. Different cells need not belong to one connected trace, and a containment test does not establish recovery.

Evidence boundaries

- | | |
|--|---|
| <p>01 Tool-mediated injection; AgentDojo also tests local defenses, without state-cleanup evidence.</p> <p>03 Connected carrier-to-violation trace and stopping stage; no selection or post-incident recovery.</p> <p>05 Create-path and five-round source-removal trace; no natural-stream or descendant-deletion proof.</p> <p>07 Updates and downstream unsafe behavior; MemEvoBench adds local memory correction, not rollback verification.</p> <p>09 Forgetting and use boundaries; no descendant-wide repair.</p> <p>11 Model tests proxy agent exposure; SEVerA adds guarded execution, not agent-loop recovery.</p> | <p>02 Cascading injection across profiles, messages, and agents; response and repair unmeasured.</p> <p>04 Skill authoring-to-carryover trace plus local repair and retirement; no descendant or deletion proof.</p> <p>06 Recognition and unsafe-choice scoring; ASB adds memory-poisoning and local-defense tests.</p> <p>08 Personalized context and warranted-memory-use evaluation.</p> <p>10 Tool authorization and local containment, not descendant repair.</p> <p>12 LoRA update and bounded mitigation; no lifecycle rollback evidence.</p> |
|--|---|

claim, the repaired state and subsequent reuse test. Table 6 compares these observation boundaries using qualitative coverage grades, not performance scores. Admit and Move concern state entry and transfer; Activate and Expose concern reuse and observable unsafe behavior, including labeled execution records. Contain concerns interventions against unsafe use, whereas Rollback, Delete, and Contest concern state restoration, removal verification, and challenges to decisions. Component-level tests that only proxy an agent-level stage receive partial coverage. For grouped rows, direct coverage requires an explicit stage-specific test in every cited study; coverage confined to some studies remains partial. The table states the full grading rule and identifies exceptions in its boundary notes. It shows strong activation and exposure coverage in the selected studies, with less evidence for descendant repair and contestability; it does not estimate field prevalence. Table 8 closes the section with the corresponding paper-level benchmark index, including target system, observed lifecycle slice, and denominator.

8.3 Comparable Outcomes and Safety–Utility Tradeoffs

Quantitative synthesis is useful when the measured event and its opportunity set are explicit. Action-boundary ASR, safety-recognition F_1 scores, memory-poisoning success, and runtime-intervention rate measure different outcomes. We use within-paper contrasts to examine commitment effects, progression through successive gates, and safety–utility tradeoffs; we do not rank heterogeneous benchmarks by their headline rates. Safe comparison is possible inside a shared benchmark denominator, transition slice, and measurement object. InjecAgent supports exposure comparisons within its indirect-injection cases; R-Judge supports recognition comparisons over annotated records; PASB compares episodes with and without observed commitment; and runtime evaluations of SafeAgent and Progent compare interventions within their respective harmful-task settings. Once those denominator families are mixed, several local evaluation panels collapse into a category error rather than a meta-analysis.

For memory, the opportunity set changes with the operation under test. PerMemSafe and PS-Bench condition response safety on retained personal context [5, 53]; FragFuse and GhostWriter separate earlier storage

from later harmful reuse [163, 193]. AgentPoison tests trigger-selected retrieval, OEP tests transfer from accepted experience, and EvoBreak tests acquisition and joint activation of an experience set [25, 202, 229]. A retrieval trial, an accepted update, and an assembled experience set are different denominators, even when each study reports an attack success rate.

PASB supplies a within-paper comparison grouped by observed commitment. Its 1,600 tasks separate a five-turn persist stage from a cleared three-turn query stage and let the evaluated agents decide what to store. Across twelve models, the reported mean downstream failure is 45.0% in session-only episodes and 71.9% in committed episodes, a gap reported as 27.0 percentage points [131]. This association makes commitment a useful diagnostic boundary, but session-only episodes are cases without an observed durable write, not a randomized memory-disabled control. The gap therefore does not identify the causal effect of enabling memory. Evaluating a memory-restriction defense would additionally require matched utility measures; PerMemSafe illustrates the coupled objective by evaluating implicit personalized safety together with helpfulness under long-horizon memory [5, 131].

Three recent studies add different within-paper transition contrasts. PoisonedEvolution measures evidence-to-artifact promotion: at the same 10% attacker-support ratio, it reports 546/600 embedded skills in SkillClaw and 369/600 in the structurally different Trace2Skill pipeline [21]. Its SER denominator is completed evolution trials and stops before runtime activation. SkillJack instead compares poisoned trajectories with extracted skills: in SkillX, the abstract reports safety detection falling from 98.5% to 11.4%, while 80.0% of skill-mediated attacks persist after source-record deletion [240]. MAPLE-Guard compares unguarded and guarded memory-link opportunities at write, retrieval, promotion, and cross-agent reuse on LongMemEval and AppWorld [224]. These are, respectively, artifact-promotion, representation-and-deletion, and guarded-lifecycle-link denominators. None should be ranked against PASB’s commitment-group gap, EvoBreak’s experience-set attack, or runtime ASR.

SkillMisevo-Bench connects artifact promotion to later execution under fixed denominators. Each agent-method condition contains 25 episodes and 525 tasks: 225 malicious learning tasks, 225 benign tasks, and 75 clean-session persistence tasks. Across the 21 evolved configurations, unsafe artifacts appear in 21, unsafe retrieval and benign-task contamination in 19, and fresh-session harm in 15 [132]. These are successive gates rather than interchangeable rates. In its two-configuration exposure sweep, pooled carryover ASR rises from 16.0% at $K = 0$ to 35.3% after three malicious tasks and 41.3% after nine. Here K is the number of malicious learning tasks. Because the no-exposure baseline is already nonzero, the defensible result is the within-paper increase after bounded exposure, not that all observed carryover harm originates in the malicious tasks.

The same study makes the response tradeoff more precise than its abstract-level mean. Across OpenClaw with AutoSkill and EvoSkill, SafeEvolve reduces mean unsafe retrieval from 35.33% to 8.67% and carryover ASR from 21.33% to 4.00% [132]. Online benign utility changes from 58.44% to 58.00%, which yields the reported 0.4-point cost, but carryover utility changes from 53.33% to 40.67%. The result therefore supports a low immediate utility cost under the paper’s online denominator while also showing a larger loss of useful persistent reuse. It is not evidence of a cost-free or universal safety-utility frontier.

EVOMAL adds a create-path and descendant denominator. ASPR counts tasks that store a newly authored skill reproducing a planted pattern; it is distinct from the callback rate that records payload execution. Across six models on 153 tool-relevant SWE-bench Verified tasks per model, headline ASPR ranges from 20.3% to 41.8%, producing 31–64 new malicious skills from eight planted seeds and leaving 4.9–9.0 times as many malicious skills as were planted [218]. The reported 86.7% targeted maximum is the Qwen3 pytest-family cell with $N = 15$, not the full task distribution. The five-round removal experiment is a separate rolling-replacement denominator and should not be collapsed with single-round ASPR or callback success.

ColluSkill adds a skill-set denominator [246]. Its 200 three-skill attack chains reach 96.0% average ASR against six evaluated scanners; adding full installed-skill context to ChainGuard reduces the attack ASR from 69.0% to 22.5% while changing the reported benign-workflow pass rate from 99.7% to 99.5% [246]. These values measure candidate-with-context scanning on the paper’s generated chains and benign workflows. Because the attack iteratively uses feedback from the target scanners that are later evaluated, the 96.0% result is evidence of adaptive scanner evasion under that feedback setting, not unseen-scanner generalization or ecosystem prevalence.

AID-Guard adds an authorization-to-effect denominator rather than another attack-success leaderboard. Its 210 frozen Stripe contract trials matched their predeclared outcomes; separate campaigns exercised 40 terminalize-then-successor schedules across Stripe and Resend, 30 overlapping Stripe confirm/cancel races, and 10 Stripe crash-recovery schedules without an observed duplicate effect [191]. The same paper makes the safety–utility cost visible: over 48 benign AgentDojo episodes per model and condition, the strict exact-manifest profile reduced benign utility by 43.8 percentage points for DeepSeek and 35.4 points for Qwen. In a separate no-seed rerun, a typed three-call authority form completed 10 and 9 more benign tasks than exact single use, while all 864 attack episodes produced no unsafe effect. These results remain paper-local: the provider claims depend on the declared effect-path inventory and supported contracts, and the no-seed comparison does not identify a causal effect size or an optimal policy.

Takeaway 8.2. Match Opportunity Sets Before Comparing Scores. Two rates are not comparable merely because both are called attack success or safety improvement. The opportunity set, transition slice, and measured outcome must match. Poisoned retrieval, harmful experience transfer, and repaired-descendant coverage count different events; their differences explain mechanisms but do not support a shared leaderboard.

A practical normalization layer would report activation opportunities alongside realized exposures and discovered descendants alongside verified repairs. Exact values remain source-specific until those measurement contracts align.

8.4 Benchmark Patterns and Missing Traces

Most benchmarks remain locally strong but observe only one transition slice. Three connected designs now cover complementary scopes. HarnessSafe spans seven persistent-carrier families and assigns each executable case to its furthest evidence-supported stage; SkillMisevo-Bench follows one skill carrier from authoring through clean reload, retrieval, contamination, and carryover harm; and EVOMAL isolates create-path copying, callback exposure, and descendant propagation after seed removal [132, 218, 255]. HarnessSafe provides cross-carrier breadth, SkillMisevo provides skill-evolution depth, and EVOMAL adds an explicit descendant process. They still leave rejected candidates, natural deployment streams, verified descendant deletion, contestability, and reactivation after broader adaptation incomplete.

Takeaway 8.3. Endpoint Scores Hide Where Containment Occurs. Configurations with similar endpoint attack-success rates can stop a persistent-risk chain at different stages [255]. Stage-resolved traces distinguish failed admission, missing activation, and blocked execution. This localizes containment, but a stopped trace still does not show that the implicated state was repaired or cannot return.

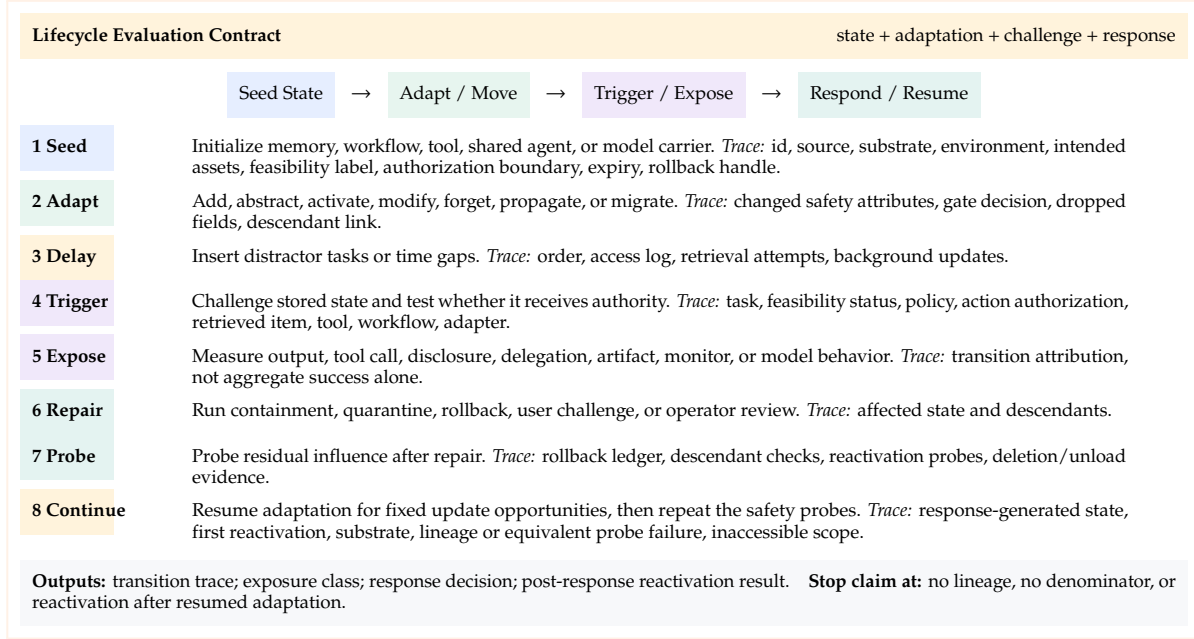
Table 7 extends stage localization into a longitudinal evaluation contract. The family-specific extensions below ask which additional observations are needed to implement this longitudinal contract.

Capability Benchmarks. Capability benchmarks establish the baseline that safety controls must preserve. Benchmark surveys map task success and interaction quality, while long-context and memory tasks make state creation and reuse visible [130, 138]. PERMA adds evolving preferences and cross-domain interference, while MemEye varies evidence granularity and evolutionary synthesis in multimodal memory [54, 115]. Coordination benchmarks extend the observation point to shared state, and evaluator-coevolution work makes the utility signal itself part of the evolving loop [70]. These benchmarks do not directly test transition safety. They identify where memory writes, summaries, workflow edits, delegated state, evaluator updates, or profile changes should be instrumented without disabling the capabilities under review.

Attack Benchmarks. Attack benchmarks become comparable only after their pressure point is identified. Prompt-injection studies perturb assembled context; AgentDojo and InjecAgent carry that perturbation to tool-mediated decisions; adaptive attacks then test whether the action boundary survives attack revision [31, 118, 247, 248]. This family is strongest at exposure, because the benchmark observes whether untrusted context changes a current action.

Two harness studies evaluate the same transition at different resolutions. When Context Gets Root tests consequences: 13 confidentiality, integrity, availability, and remote-code-execution objectives across six

Table 7 Lifecycle evaluation loop. A benchmark should seed state, transform it, trigger activation, measure exposure, verify repair, and continue adaptation before closure.



coding-agent harnesses, including attempt-capped delegation cases and four persistent-goal or scheduled-task paths [62]. What’s in Your Agent’s Context instead measures the assembly surface across 12 harnesses [105]. CoRA identifies 463 candidate context sources, validates 282 at runtime, and enumerates 1,761 role- or scope-increasing source pairs; 1,028 paths under GPT-5.4 mini and 1,034 under GPT-5.5 produce the expected behavioral effect, both about 58% of the candidate-path denominator. The first result is objective coverage under repeated opportunities, and the second is path verification rather than end-to-end attack success. Together they localize migration-to-authority and migration-to-scope failures without establishing an origin-preserving defense, descendant repair, or resumed adaptation after containment.

Persistent-state benchmarks move the pressure earlier in the chain. AgentPoison measures trigger-selected reuse from a poisoned store, whereas GhostWriter separates hidden memory admission from later activation; broader memory-poisoning work studies the same persistence problem across additional settings [25, 184, 193]. HaluMem decomposes extraction, update, and question answering, while MemConflict varies memory-conflict types [17, 189]. PASB observes the agent’s own commit decision, and PS-Bench and PerMemSafe test later use of benign personal state [5, 53, 131]. EvoBreak adds composition over an acquired experience set, SkillJack adds trajectory-to-skill migration and source-deletion persistence, and MAPLE-Guard adds lifecycle-link enforcement across private and shared memory [224, 229, 240]. These studies belong to different opportunity sets from prompt injection because the benchmark must observe a write, transformation, promotion, or retained context before later reuse.

AuthMem-Bench is unusually close to a connected $S \times A \rightarrow V \rightarrow E$ trace. Its three modules measure write-time authority collapse, the action consequence of a controlled memory representation, and end-to-end authority-label preservation while keeping paired claim content and later tasks fixed [249]. The modules should still retain separate denominators: write-time upgrade, controlled unauthorized action, and end-to-end action after omission and retrieval are not interchangeable rates. The benchmark ends after one write-to-action cycle, so it does not measure descendant repair or post-response reactivation.

HarnessSafe broadens that connected trace from one memory-consolidation boundary to seven carrier families and seven agent harnesses [255]. Its carrier path and persistence boundary identify $S \times A$; case-specific oracles connect V to observable E ; and the furthest-stage distribution supplies containment evidence for R . This mapping is more informative than endpoint ASR: OpenCode and Hermes Agent report similar

conditional ASRs in the paper but stop at different stages. The Chain-Stage Score remains paper-specific, measures progression rather than harm severity, and excludes unsupported or incomplete workflows rather than counting them as safe.

Propagation and recognition studies expose still later or orthogonal slices. ACIArena follows malicious instructions across multi-agent trust channels, malfunction amplification measures local escalation, and R-Judge asks whether the resulting risk is recognized [6, 245, 250]. Agents of Chaos uses eleven live-laboratory cases rather than a fixed benchmark denominator, while Triedman et al. report attack trials indexed by model, orchestrator, input channel, and payload [173, 194]. ControlValve evaluates a response at the agent-invocation boundary by asking whether execution stays inside a permitted control-flow graph and its contextual rules [73]. These denominators should remain separate: case-study diversity, per-configuration attack success, and invocation-policy enforcement do not form one leaderboard. Together they expose concrete violations and controls, but attack success remains a surface metric unless an evaluation also identifies source content, selected edge, recipient authority, downstream effects, and response.

Transition and Recovery Benchmarks. Trajectory assurance supplies the high-level target: verify a stateful sequence against system-level constraints rather than assume that individually permitted actions compose safely [121]. The narrower measurement question for this survey is which reusable state transition changed that sequence and whether its influence can later be contained or repaired. State-transition benchmarks are closest to this unit because they ask whether the enabling carrier, operation, source, safety-attribute change, and activation context can be identified. ASB provides partial ingredients for agent-security task coverage [252], while Agent-SafetyBench provides adjacent agent-safety behavior coverage [263]. HarnessSafe advances this line by preserving carrier, boundary, benign trigger, and violation roles across harness-native implementations, then using matched controls to test whether progression depends on the declared lifecycle [255]. Transition evaluation still needs selection and lineage fields beyond task-outcome scoring. The strongest versions should connect this stage localization to attribute preservation across migration, drift after adaptation, and model-update lineage when the update is selected, merged, loaded, or inherited inside an agent loop. Fine-tuning safety is relevant when an update changes safety behavior [159]; PEFT risk work is relevant when the adapter or patch is a deployable state object [64]; and dormant-behavior work shows that changed behavior may remain hidden until a matching activation context appears [50]. Linking these tests to selection history and post-intervention reuse would support stronger lifecycle claims.

Response-oriented benchmarks test distinct control contracts. GuardAgent and TrustAgent evaluate diagnosis and local guard reasoning; Progent and Task Shield test constraints on privilege and task-aligned execution [68, 74]. CaMeL tests capability-enforced control and data flow, DRIFT tests dynamic plan validation and injection isolation, and MAPLE-Guard tests gates over memory writes, retrievals, promotions, and cross-agent reuse [32, 94, 224]. Their utility and attack-success results remain paper-local because the systems enforce different contracts. SafeAgent and LlamaFirewall test runtime containment, while MAGE extends containment to long-horizon memory and EDDOps asks how the control is maintained in operation [27, 112, 212]. The remaining gap is whether these stages preserve enough lineage to measure containment latency, rollback completeness, contestability cost, and audit completeness without turning privacy-sensitive traces into a new risk.

The reviewed benchmarks thus differ along two independent dimensions: which transition they test and how far they follow its consequences. A connected carrier-to-action trace improves causal localization, while a repair-and-retest protocol asks a different question about the state left behind. Continued adaptation is needed to examine whether that repair remains effective under later reuse. Section 9 develops the research needs exposed by these incomplete traces and denominators.

Takeaway 8.4. Recovery Evaluation Must Resume Adaptation. A clean probe immediately after repair establishes only an immediate result. We propose continuing adaptation for a declared number of update opportunities and rerunning fixed probes. Reports should distinguish lineage-linked reactivation from an equivalent observed failure whose lineage is unknown, and state the tested horizon and inaccessible scope rather than claim permanent recovery.

Table 8 Evaluation literature for adaptive state safety. Most benchmarks expose one lifecycle slice; HarnessSafe, SkillMisevo-Bench, and EVOMAL provide complementary connected traces across cross-carrier, skill-specific, and descendant-propagation lifecycles, while the denominator column states what each row can safely measure.

Method or Work	Year	Role	Target	Object	Task Focus	SAVER Slice	Denominator
<i>General agent capability and environment benchmarks</i>							
AgentBench [116]	2023	Benchmark (Capability)	Agent	Evaluation	General agent tasks	Substrate	Task success
AgentBoard [127]	2024	Benchmark (Capability)	Agent	Evaluation	Analytical agent tasks	Adjacent capability context	Task progress and success
MIRAI [236]	2024	Benchmark (Capability)	Agent	Workflow	Long horizon tasks	Substrate	Task episodes
GameBench [29]	2024	Benchmark (Capability)	Agent	Environment	Strategy tasks	Substrate	Game episodes
MultiAgentBench [270]	2025	Benchmark (Capability)	Agent	Multi-Agent	Multi-agent tasks	Adjacent coordination	Collaborative tasks
LegalAgentBench [93]	2024	Benchmark (Capability)	Agent	Workflow	Legal tasks	Substrate	Legal tasks
<i>Attack, safety, and exposure benchmarks</i>							
AgentDojo [31]	2024	Benchmark (Attack)	Agent	Tool	Prompt injection through tool use	Exposure	Security cases
InjecAgent [247]	2024	Benchmark (Attack)	Agent	Tool	Indirect injection	Exposure	Injection tasks
ACIArena [6]	2026	Benchmark (Attack)	Agent	Multi-Agent	Cascading injection	Propagation/Exposure	MAS test cases
Agents of Chaos [173]	2026	Safety	Agent	Multi-Agent	Live-lab failures	Propagation/Exposure	Eleven case studies
MAS code execution [194]	2025	Attack	Agent	Multi-Agent	Control-flow hijacking	Activation/Exposure	Model-orchestrator trials
AgentHarm [7]	2025	Benchmark (Safety)	Agent	Evaluation	Harmful agent tasks	Exposure	Harmful task cases
RedCode [52]	2024	Benchmark (Attack)	LLM	Code	Code safety tasks	Exposure	Code tasks
MobileSafetyBench [88]	2026	Benchmark (Safety)	Agent	Tool	Mobile control safety	Exposure	Mobile tasks
Machiavelli [149]	2023	Benchmark (Safety)	Agent	Environment	Ethical decision making	Exposure	Game scenarios
SafeLawBench [16]	2025	Benchmark (Safety)	Agent	Workflow	Legal safety	Exposure	Legal cases
R-Judge [245]	2024	Benchmark (Safety)	Agent	Evaluation	Safety recognition	Response	Annotated records
Agent Security Bench [252]	2024	Benchmark (Attack)	Agent	Evaluation	Agent security tasks	Violation	Task categories
Agent-SafetyBench [263]	2024	Benchmark (Safety)	Agent	Evaluation	Unsafe behavior	Exposure	Safety cases
AttackEval [178]	2025	Benchmark (Attack)	LLM/Agent	Evaluation	Attack evaluation	Exposure	Attack tasks
CyberSecEval 2 [13]	2024	Benchmark (Attack)	LLM	Code	Cyber capability	Exposure	Cyber tasks
<i>Adaptive memory, experience, and model state benchmarks</i>							
AgentPoison [25]	2024	Benchmark (Attack)	Agent	Memory	Memory poisoning	Admission/Activation	Retrieval opportunities
GhostWriter [193]	2026	Benchmark (Attack)	Agent	Memory	Personal agent memory poisoning	Admission/Activation	Injection/activation trials
OEP [202]	2026	Benchmark (Attack)	Agent	Workflow	Experience poisoning	Adaptation	Accepted experience updates
EvoBreak [229]	2026	Benchmark (Attack)	Agent	Memory	Experience composition	Abstract/Activate	Acquired experience sets
SkillJack [240]	2026	Benchmark (Attack)	Agent	Skill	Skill backdoor	Migrate/Exposure	Trajectory-to-skill pairs
PoisonedEvolution [21]	2026	Benchmark (Attack)	Agent	Skill	Trajectory poisoning	Migrate/Violation	Completed evolution trials
ColluSkill [246]	2026	Benchmark (Attack)	Agent	Skill	Cross-skill composition	Add/Activate/Response	Three-skill chains
SkillMisevo-Bench [132]	2026	Benchmark (Safety)	Agent	Skill	Skill misevolution	Migrate/Activate/Response	525 tasks per configuration
EVOMAL [218]	2026	Benchmark (Attack)	Agent	Skill	Create-path self-poisoning	Propagate/Activate/Response	153 tasks/model; five rounds
PerMemSafe [5]	2026	Benchmark (Safety)	Agent	Memory	Personalized safety	Activation/Exposure	Long-horizon conversations
PS-Bench [53]	2026	Benchmark (Safety)	Agent	Memory	Intent legitimization	Activation/Exposure	Matched memory/query trials
PASB [131]	2026	Benchmark (Safety)	Agent	Memory	Persistent sycophancy	Add/Activate/Exposure	Committed-claim trials
AuthMem-Bench [249]	2026	Benchmark (Safety)	Agent	Memory	Authority collapse	Abstract/Violation/Exposure	Matched authority pairs
HarnessSafe [255]	2026	Benchmark (Safety)	Agent	Workflow	Persistent carrier progression	Carrier-to-exposure/Containment	328 executable cases
When Context Gets Root [62]	2026	Benchmark (Attack)	Agent	Workflow	Instruction privilege escalation	Migrate/Violation/Exposure	13 objectives; six harnesses
Agent Context CPE [105]	2026	Benchmark (Attack)	Agent	Workflow	Role and scope escalation	Migrate/Violation/Exposure	282 sources; 1,761 paths; 12 harnesses
MemEvoBench [222]	2026	Benchmark (Safety)	Agent	Memory	Memory misevolution	Adaptation	Memory updates
AgenticEval [210]	2026	Benchmark (Safety)	Agent	Evaluation	Agentic safety tasks	Adaptation	Agent episodes
RQGM [70]	2026	Capability	Agent	Evaluation	Evolving evaluators	Evaluation/Adaptation	Epoch utilities
Experience driven safety [266]	2026	Safety	Agent	Workflow	Experience degradation	Adaptation	Experience updates
AMA-Bench [267]	2026	Benchmark (Safety)	Agent	Memory	Agent memory adaptation	Adaptation	Memory tasks
PersistBench [155]	2026	Benchmark (Safety)	Agent	Memory	Forgetting boundary	Violation	Memory use samples
RBI-Eval [225]	2026	Benchmark (Safety)	Agent	Memory	Warranted use	Activation	Matched prompts
Memory evaluation [67]	2025	Benchmark (Capability)	Agent	Memory	Memory quality	Substrate	Memory tasks
<i>Response, governance, and red team evaluation</i>							
SafeAgent [112]	2026	Defense	Agent	Runtime	Runtime protection	Containment	Harmful task episodes
Progent [176]	2025	Defense	Agent	Tool	Privilege control	Activation	Permission checks
CaMeL [32]	2026	Defense	Agent	Tool	Capability control	Activation	AgentDojo tasks
DRIFT [94]	2025	Defense	Agent	Tool	Dynamic rule enforcement	Activation/Containment	Benchmark task episodes
AID-Guard [191]	2026	Defense	Agent	Workflow	Delegated provider effects	Activate/Response	Provider-contract schedules
ControlValve [73]	2026	Defense	Agent	Workflow	Control-flow integrity	Activation/Containment	Agent-invocation checks
MAPLE-Guard [224]	2026	Defense	Agent	Memory	Memory-link enforcement	Propagate/Response	Memory lifecycle links
LlamaFirewall [27]	2025	Defense	LLM/Agent	Runtime	Runtime guardrail	Containment	Guard decisions
OpenGuardrails [207]	2025	Defense	Agent	Runtime	Guardrail platform	Containment	Guard decisions
AGENTS SAFE [81]	2025	Governance	Agent	Governance	Agent safety governance	Response	Governance checks
AgenticRed [243]	2026	Red Team	Agent	Evaluation	Automated red teaming	Exposure	Red Team tasks
Agent vs. Agent [82]	2025	Red Team	Agent	Multi-Agent	Adversarial agent play	Exposure	Duel episodes
SEVerA [11]	2026	Defense	Agent	Workflow	Verified synthesis	Response	Constraint checks
Safe LoRA [65]	2024	Defense	LLM	Model	Adapter mitigation	Model state	Safety regression tests
DataGovBench [120]	2025	Benchmark (Governance)	Agent	Data	Data governance	Adjacent data workflow	Data-governance transformation tasks

9 Open Challenges

The preceding chapters identify local mechanisms and measurable transition segments. Connecting them raises technical problems that more logging alone does not solve: summaries can obscure derivation, changing capabilities can invalidate a grant, and copied state can remain outside the authority of the system attempting repair. The substrate matters to each problem. Explicit memory permits inspection but can lose source constraints during abstraction; skills and workflows make those changes executable; shared artifacts distribute repair ownership; and model-side state makes influence less directly observable.

With substrate differences as comparison conditions, the following agenda moves from the decision to retain an update, through migration, activation, and propagation, to recovery and longitudinal evaluation. Each direction asks what prevents the existing mechanisms from supporting the next claim and which tradeoff a useful solution would have to measure. Table 9 summarizes the corresponding evidence needs.

9.1 Selection Pressure and Endogenous Drift

Most adaptive-state evidence begins after an update has entered memory, a skill library, a workflow, or model state. The earlier decision is less visible: what candidates were available, which signal ranked them, which safety conditions were checked, and which failures or counterexamples were absent. Reflection and skill systems expose the mechanics by which feedback and successful traces become reusable state, OEP demonstrates harmful transfer from locally correct experience, and evaluator coevolution shows why the scoring rule and epoch must be recorded [70, 177, 199, 202]. None of these establishes the prevalence of attacker-free drift. The open question is whether repeated utility-guided selection changes the safety boundary when the candidate stream is otherwise benign. Under its formal assumptions, Devil’s Moltbook identifies a trilemma: continuous self-evolution, complete isolation, and safety invariance cannot all hold simultaneously [198]. We treat this result as a design constraint rather than a universal empirical law: evaluation should identify which useful update paths remain open, which external oversight or safety-preserving mechanism governs them, and how much utility is lost when risky updates are blocked.

Takeaway 9.1. Test Endogenous Drift Separately from Attack Success. Harmful transfer and attack-driven accumulation do not establish how often benign self-evolution drifts. Testing that question requires a non-adversarial candidate stream, fixed safety probes, and records of selected, rejected, and unavailable evidence. Stable results are informative within the tested scope; degradation still needs attribution to selection, promotion, or activation rather than to self-evolution in general.

9.2 Migration and Semantic Preservation

Migration control is a major gap because unsafe influence often appears precisely when state changes substrate. Provenance, authority, scope, privacy, freshness, and actionability must survive movement across memory, retrieval, workflow, tool, shared-agent, external-artifact, and model substrates. MemGPT makes movement between context and managed memory visible, memory-security work supplies the carrier vocabulary, and privacy work adds scope and sensitivity preservation [109, 148, 197]. InjecAgent shows why those attributes matter once external state reaches an action boundary [247]. These sources still stop short of a general migration-control discipline.

The difficulty is deciding what a transformation has preserved. A summary can retain a useful claim while dropping the condition under which it may be acted on. Rejecting every uncertain summary would restrict the compression and generalization that make adaptive memory useful; accepting it without qualification can broaden authority. Migration evaluation should therefore compare unauthorized role upgrades with the useful transfers rejected or downgraded by the same gate, using source and target constraints as the reference.

Semantic Taint through Summaries and Vectors. Traditional taint tracking assumes symbolic objects whose marks can be copied through program operations, but agent state is often natural language, summary text, embedding vectors, retrieved chunks, tool traces, and model updates. A summarizer may remove the string that carried a source warning while preserving the behavioral implication. An embedding may make a sensitive item retrievable without exposing its exact words. A generated skill may encode a rule learned from

Table 9 Research agenda for lifecycle evidence in self-evolving agents.

Direction	Evidence Gap	Minimal Test
Selection / promotion	Candidate updates are ranked by visible utility while safety evidence, rejected alternatives, and negative experience remain unobserved.	Candidate and promotion log plus fixed cross-epoch probes for authority, scope, refusal, and confirmation drift.
Migration / preservation	State changes carrier while provenance, scope, authority, or revocation becomes ambiguous.	Migration ledger plus residual probes for summaries, embeddings, workflows, artifacts, and adapters.
Activation authorization	Stored state becomes reasoning, planning, tool, workflow, or model authority outside its allowed context.	Activation gate that logs user, task, tool, time, and allowed external effect.
Recovery / deletion	Rollback removes the visible object while summaries, indices, skills, or shared artifacts may still activate its influence.	Descendant inventory, deletion receipt, and activation probe after repair.
Propagation / contestability	Shared state crosses agents or artifacts without clear repair ownership or a user challenge path.	Propagation trace plus contest docket for correction, quarantine, and notification.
Longitudinal evidence chain	Benchmarks measure isolated slices rather than one incident path from admission to closure.	Opportunity set denominators for admission, migration, activation, exposure, repair, and closure.

a private or low-authority source. Semantic taint tracking is the engineering version of migration control because the system needs to track influence even when the surface form changes.

A hybrid approach could combine explicit derivation links with checks on the transformed content. Provenance identifies declared sources, while information-flow labels specify permitted uses [15, 35]; neither alone determines whether a paraphrase or generated skill still carries the same influence. The research question is how to bound that uncertainty without either treating every descendant as contaminated or losing the relevant path. Memory-security studies and AgentDojo provide abstraction and tool-exposure settings in which such checks could be tested [31, 109]. Residual probes would assess remaining influence, while false quarantines and lost task utility would measure the cost of conservative tracking.

9.3 Activation, Authorization, and Model Scope

A system may legitimately store a user preference, a webpage observation, a tool result, or a learned skill, but that does not mean the item may influence every later answer, plan, workflow route, or external action. Activation authorization is distinct from storage authorization. The local control evidence moves from privilege to task alignment to guard reasoning: Progent shows programmable privilege control at the action boundary, Task Shield narrows indirect injection through task alignment, and GuardAgent illustrates bounded response reasoning [74, 176, 221]. These papers anchor local action-boundary control, but they leave the broader authorization problem open for adaptive state: under what user, task, tool, time, and external effect conditions may stored state become active influence?

The unresolved design choice is how finely to distinguish readable context, reasoning support, planning authority, and permission for an external effect. Coarse grants are easier to reuse but can hide a scope change; repeatedly requesting authorization can preserve explicit control at the cost of interruption. Useful tests need both unauthorized activations and legitimate work delayed or rejected, not a block rate alone.

Authorization continuity adds a separate longitudinal question: after memory, skills, tools, workflow, delegation, task phase, or enforcement state changes, does the original grant still apply to the resulting agent [262]? A useful evaluation must separate grant survival from effect containment. It should report invalid grants retained after mutation, valid grants unnecessarily revoked, authority acquired below a fixed ceiling, reauthorization cost, and compositions of individually permitted mutations that move the agent outside the original envelope. The current formal model supplies proof obligations, but no deployment benchmark yet measures these tradeoffs or tests whether envelope and effect abstractions remain sound under sustained evolution.

Context-Horizon Safety. Benign-looking turns can jointly close a harmful objective, and multi-turn decomposition can carry that objective into tool use [102, 174, 216]. Compression creates a different path by relinking accepted fragments into an actionable instruction [119].

Takeaway 9.2. Context Accumulation Is Not Update-Driven Drift. Accumulated or compressed context can enable harm without changing model parameters or a reusable policy. Token count alone therefore does not diagnose safety drift. Evaluation should separate the first harm-enabling combination within an interaction from boundary changes caused by persistent updates, and record the operations responsible for each.

Model Adaptation Governance. For model-side state, the authorization problem is how a task-scoped update can be constrained when its behavioral effects extend to other tasks. This question enters the core analysis only under the agent-loop boundary. A self-evolving agent can change reusable behavior through memory, workflow, and skills, but it can also load, merge, fine-tune, erase, or roll back model updates during operation. PEFT and fine-tuning safety work explain why modular updates can become deployable risk objects; feedback poisoning and dormant-behavior papers add persistent failure through update channels or later activation contexts [50, 64, 159, 162]. Safe LoRA and KVERaser provide bounded repair handles at the adapter and cache levels [65, 96]. What remains thin is lifecycle governance linking update admission, deployment scope, activation, rollback, cache-local erasure, and residual influence verification.

For these less inspectable carriers, the challenge is relating a localized edit to its broader behavioral effects. Tests could compare the intended task population with tasks outside that scope, examine whether merged components preserve their separately evaluated constraints, and probe residual influence after rollback or cache erasure. Update lineage identifies what changed; it cannot by itself establish which behaviors were removed. The agent-loop inclusion rule keeps this question narrower than general model safety.

9.4 Propagation and Distributed Lineage

A memory item held by one assistant can become shared context for a team agent, a task instruction for a specialist agent, a routing rule in an orchestrator, or a persistent artifact read by humans and future agents. Multi-agent propagation turns state governance into a topology problem. Agent and self-evolving-agent surveys justify the system and adaptive setting; multi-agent memory-poisoning analysis and collective misremembering show why copied state, recipient interpretation, and carrier metadata matter for propagation risk [8, 124, 192, 226]. The unresolved trace gap reduces to a few questions: which agent wrote the state, which agent received it, how the safety attributes changed, where uptake occurred, and which descendants must be repaired.

Recent work makes the local attack and control points more concrete. Agents of Chaos reports cross-agent propagation among failures observed in a live laboratory, while Triedman et al. show that adversarial content can hijack orchestration and cause code execution or data exfiltration even when individual agents do not follow the malicious request [173, 194]. ControlValve then constrains agent invocations with a permitted control-flow graph and contextual least-privilege rules [73]. These papers narrow the gap at propagation, exposure, and activation-time containment. They do not yet show how to enumerate affected descendants, roll back already-produced artifacts, notify users across agent boundaries, or let a user contest the state and authority that caused the route.

The harder problem begins after recipients have copied or transformed the state. A source-side correction may reach one agent while another retains a derived rule, and the sender may lack authority to modify that recipient's store. A useful distributed repair protocol would separate notification, acknowledged invalidation, and verified influence withdrawal. Evaluation should vary which recipients remain reachable and whether broader quarantine interrupts legitimate coordination. This would test repair ownership and partial visibility rather than assume that a shared identifier guarantees synchronized recovery.

9.5 Recovery and Verification

The recovery scope in Takeaway 7.3 depends on discovering descendants, but abstraction makes that inventory uncertain. Memory-poisoning, Hidden Memory, and OEP show why retained or learned influence can activate after its source episode [156, 184, 202]. GuardAgent, SafeAgent, and Progent offer local response mechanisms [112, 176, 221]; connecting them to repair requires deciding which derived objects still carry the contested influence. This is an inference problem as well as an instrumentation problem. Recovery also

need not mean erasing every descendant. Some objects need correction or scope reduction; others must remain as restricted audit evidence. Research should compare targeted repair with broader quarantine under the same discovery budget: how much unsafe influence remains, how much useful state is lost, and which inaccessible objects prevent a stronger claim? Defining the tested scope before repair avoids counting only the descendants that a method happened to fix.

Post-Response Continuation. A clean immediate probe cannot determine what happens after adaptation resumes. Retesting after attacker or system revision is already motivated by adaptive-attack work [248]; Takeaway 8.4 extends the test to continued state updates. The unresolved question is attribution: does a later failure descend from unrepaired state, arise from newly acquired evidence, or reflect a changed evaluator? Fixed probes identify recurrence, while lineage and version records constrain which explanation the result supports. This is a proposed diagnostic, not evidence that responses generally amplify risk.

Deletion Verification. A deletion request may remove a memory item while leaving an embedding, a summary, a cached context, a copied workflow note, a tool return record, or a shared artifact intact. That makes deletion verification related to rollback but not reducible to it. In settings sensitive to privacy, this is a safety attribute preservation failure as well as a usability issue. The evidence again enters by layer: memory privacy work explains the user risk, MemGPT and memory-mechanism surveys show how memory moves and abstracts, memory security work frames those routes as governed state, and KVERaser sharpens the cache-specific version by showing why a removed context span can still matter through subsequent KV cache states [96, 109, 197, 261]. What remains open is how an adaptive agent should prove that deleted state no longer influences retrieval, planning, tool use, cached execution state, or later state updates.

The central difficulty is that object absence and influence removal can disagree. A summary may omit the deleted string while preserving its implication; a skill may reproduce the behavior without retrieving the source. Verification should therefore include semantic retrieval, summary regeneration, and later skill-use probes, with the tested contexts stated explicitly. The open problem is how much evidence these finite probes provide about future reuse, especially when broader probing itself accesses sensitive state.

User Contestability. Recovery reaches human closure only when a user or operator can challenge state and its descendants. Memory privacy work motivates this need through persistence and sensitivity, while R-Judge, GuardAgent, and Progent provide adjacent signals for recognition, local reasoning, and privilege control [176, 197, 221, 245]. None yet provides a full contestation workflow for disputed adaptive state. Contestability creates a further tradeoff between understandable explanations and the detail needed to challenge a decision. A user needs to identify the disputed premise and request correction or restricted reuse without inspecting an entire internal trace. Tests should assess whether that request changes later behavior, whether derived state incorporates the correction, and whether unresolved descendants are disclosed. The research target is effective correction under partial visibility, not merely the presence of a deletion control.

9.6 Benchmark and Governance Evidence Chains

Fixed task benchmarks reveal important failures, especially in prompt injection through tool use or risk recognition, but self-evolving safety also depends on what happens after an update is stored and before an incident is repaired. Longitudinal benchmark construction ties these challenges together. One group covers exposure and recognition: InjecAgent, R-Judge, ASB, and Agent-SafetyBench instrument indirect injection, risk awareness, and agent safety task coverage [245, 247, 252, 263]. A second group covers persistence and sensitive reuse: PASB, PerMemSafe, later memory-poisoning work, and memory privacy work show why commitment, retrieval, personalized context, and purpose must become benchmark fields [5, 131, 184, 197]. HarnessSafe now connects entry, carrier retention, boundary crossing, benign activation, and observable violation across seven carrier families [255]. The open problem has therefore narrowed: extend that connected trace backward to candidate selection and forward through descendant repair and resumed adaptation.

Extending a benchmark across these boundaries creates a comparability problem. A system may appear safer because it declines more updates, retrieves less memory, or prevents useful delegation. The opportunity sets in Section 8 make these exclusions visible, but matched utility and exposure conditions are still needed

to determine whether a control improved safety or merely removed the capability under test. Natural deployment streams add the further difficulty of reconstructing candidates that were never retained.

Governance Evidence Chain. Trajectory assurance connects architectural constraints to runtime behavior and end-to-end auditability [121]. For evolving agents, separately evaluated controls also need compatible meanings for state identity, authority, and repair. A write-time screen, personalized memory policy, risk judge, and privilege gate can otherwise reach different decisions about what appears to be the same object [5, 176, 193, 221, 245]. The open question is which information must survive between these controls for their local guarantees to remain interpretable together. Governance records also need protection: approvals, quarantine markers, and rollback receipts can themselves be copied, summarized, or made stale. Their integrity determines whether later controls can rely on earlier decisions.

Takeaway 9.3. Shared Incident Records Connect Local Controls. A shared identifier connects records only if the controls agree on which version, permitted use, and repair outcome it denotes. The research target is interoperability across update, activation, and recovery, including inaccessible descendants and unresolved influence. Evaluation must test whether local control outcomes compose, rather than assume that linking their logs produces a lifecycle guarantee.

10 Conclusion

Self-evolving agents convert observations, feedback, and execution traces into reusable influence. SAVER pairs substrate with adaptation and follows violation, exposure, and response across memory, tools and workflows, multi-agent systems, and model-side adaptation. Persistent-state and tool-agent studies expose complementary parts of this chain; HarnessSafe shows that a connected trace can follow influence across carriers to an observable violation [255]. Selection before commitment and verified recovery after containment remain less connected. Two conclusions follow from the chapter-level findings.

Takeaway 10.1. Safety Must Follow Changes in Permitted Use. Self-evolving-agent safety concerns whether earlier information remains legitimate in its later role. Carrier-specific constraints and operation-specific checks belong to one analysis: migration can change authority while propagation expands the activation scope. As developed in Takeaway 3.4 and Takeaway 4.4, preserving content alone does not preserve safe reuse.

Takeaway 10.2. A Recovery Claim Must State Its Scope. Local containment establishes that an event was stopped; lifecycle recovery requires evidence about the state left behind. The discovered-descendant inventory and post-response continuation test in Takeaway 7.3 and Takeaway 8.4 define complementary spatial and temporal limits. A defensible claim states both, including unresolved influence, rather than treating a successful block as closure.

The resulting research agenda is to test how local controls compose as agents continue to update and reuse state. Such tests must preserve the capabilities that motivate self-evolution while making their changing authority and repair limits explicit.

Appendix

A Survey Methodology And Corpus Protocol

Review Objective. The review asks how safety changes when an agent can preserve, transform, reactivate, or inherit state across interactions. A work is relevant when it helps explain a reusable state transition: a carrier is created or modified, the carrier later influences reasoning or action, and some safety attribute is preserved, violated, exposed, evaluated, or repaired. This objective gives the survey a lifecycle protocol rather than a topic-only bibliography. It also explains why the corpus combines work on self-evolving agents, long-term memory, tool use, workflow state, multi-agent coordination, model updates, benchmarks, runtime safeguards, and governance [8, 31, 124, 157].

Source Identification. The source-identification protocol first defined bands that match the survey question: security venues for adversarial mechanisms and governance primitives, AI and machine-learning venues for agent capability and adaptation mechanisms, NLP and information-retrieval venues for prompt, retrieval, and long-context mechanisms, and preprint or scholarly-index sources for fast-moving agent-safety work that had not yet stabilized in proceedings. The venue bands include the four major security venues, IEEE Symposium on Security and Privacy (IEEE S&P), USENIX Security, ACM CCS, and NDSS; AI and machine-learning venues such as NeurIPS, ICML, ICLR, AAAI, and IJCAI; NLP and retrieval venues such as ACL, EMNLP, NAACL, SIGIR, and The Web Conference; and adjacent systems, software-engineering, privacy, and human-computer interaction venues when the paper contains an agent-state, provenance, evaluation, or governance mechanism. Table 10 records why each source band was searched and how it was allowed to support the synthesis.

Table 10 Search source bands for the systematic scoping protocol. Each band supplies a distinct kind of evidence needed to connect adaptive-state mechanisms with safety claims.

Source Band	Primary Sources	Mechanism Signal	Query Handles
Security and Privacy	IEEE S&P; USENIX Security; ACM CCS; NDSS; privacy venues	Threat models; access control; information flow; sandboxing; repair evidence	Injection; poisoning; authority; provenance; rollback; privacy
AI and Machine Learning	NeurIPS; ICML; ICLR; AAAI; IJCAI; OpenReview tracks	Agent learning; self-improvement; model adaptation; benchmark mechanisms	Adaptive agents; PEFT; adapters; tool use; evaluator state
NLP, Retrieval, and Web	ACL; EMNLP; NAACL; SIGIR; The Web Conference; ACL Anthology	Prompt, retrieval, RAG, long-context, and web-agent behavior	Indirect injection; retrieval safety; long-term memory; web agents
Systems, Software, and Human-Computer Interaction	Systems, software-engineering, workflow, and human-facing AI venues when state or governance is explicit	Provenance; oversight; operational safety; deployment-time state management	Workflow reuse; audit logs; confirmation; contestability
Preprints and Indexes	arXiv; OpenReview preprints; scholarly indexes; official proceedings; curated bibliographies	Frontier mechanisms before venue stabilization; evidence tiered in synthesis	Snowballing; venue checks; guardrails; emerging benchmarks

Search Strategy. Venue- and index-based retrieval was combined with citation snowballing from seed families in self-evolving-agent surveys, LLM-agent surveys, memory-security surveys, agent-safety benchmarks, indirect-prompt-injection papers, and model-adaptation safety work. We then searched over recurring query families rather than a single keyword string. Capability terms included *self-evolving agent*, *self-improving agent*, *adaptive agent*, *LLM agent*, *long-term memory*, *agent memory*, *retrieval-augmented generation*, *workflow agent*, *skill library*, *multi-agent*, *shared artifact*, *LoRA*, *PEFT*, *fine-tuning*, *adapter*, *KV cache*, and *evaluator*. Safety terms included *safety*, *security*, *prompt injection*, *indirect prompt injection*, *poisoning*, *privacy*, *access control*, *privilege*, *authority*, *provenance*, *information flow*, *rollback*, *deletion*, *unlearning*, *guardrail*, *containment*, *contestability*, and *benchmark*. Each candidate query paired at least one capability term with one safety, evaluation, or governance term so that the corpus did not drift into generic agent capability or generic LLM safety.

Expansion and Stopping Rule. Searches were run through bibliographic indexes, arXiv, ACL Anthology, OpenReview, ACM and IEEE landing pages, official venue proceedings where available, and curated

community bibliographies used only as candidate pointers. Backward and forward snowballing was then applied to seed surveys and anchor papers until new candidates mainly repeated already covered SAVER roles rather than adding a new carrier, operation, exposure surface, response mechanism, or benchmark denominator. This stopping rule is a scoping-saturation rule: it aims to cover the mechanism space needed by the survey, not to estimate prevalence for every venue or year.

Why arXiv Appears Frequently. New agent attacks, memory mechanisms, benchmark proposals, guardrail systems, and model-adaptation risks often appear first as arXiv or OpenReview versions, and many accepted works keep the arXiv version as the most accessible full-text surface. The agent-safety literature is therefore unusually preprint-heavy. Excluding arXiv would remove several mechanisms that are central to the survey question. We do not treat arXiv status as equivalent to peer review. Preprints enter the corpus when they contain a clear mechanism, experiment, artifact, or system analysis, and their claims are used according to the evidence-tier rules below. In the main text, field-level conclusions are phrased around peer-reviewed anchors, multiple converging sources, or explicitly identified gaps; arXiv-only works are used mainly as emerging evidence or frontier signals unless a later venue record is verified.

Protocol Summary. The purpose of the expansion was high recall over safety-relevant reusable-state mechanisms, not exhaustive coverage of all LLM safety, all agent capability, or all machine-learning security. Table 11 summarizes the protocol at the level used for the survey.

Table 11 Survey protocol summary. Rows separate how a work enters the corpus from the claim role it may support.

Source Family	Expansion Route	Inclusion Trigger	Synthesis Role
Self-Evolving and Agent Surveys	Seed surveys; citation chasing; official venue and preprint follow-up	Adaptive capability, reusable state, agent loop, planning, action, collaboration, or model-side adaptation	Scope context; central only with a safety-relevant state transition
Memory, Retrieval, and Context	Targeted memory, RAG, persistent-context, poisoning, and privacy searches	Stored or retrieved state can affect later reasoning, planning, tool use, or disclosure	Direct anchor for write, retrieval, delayed activation, privacy, or memory governance
Tool, Workflow, and Benchmarks	Indirect-injection, tool-use, privilege, guardrail, benchmark, and workflow searches	Low-authority content, tool output, workflow state, or learned procedure can affect action or routing	Exposure, activation, local response, or benchmark denominator evidence
Multi-Agent and Shared Artifacts	Adjacent surveys; official pages; scholarly indexes; curated bibliographies	State crosses agents, artifacts, roles, tasks, or ownership boundaries	Propagation and governance evidence when uptake or repair ownership is visible
Model-Side Adaptation	PEFT, adapters, fine-tuning safety, backdoors, unlearning, cache, and evaluator searches	Update, adapter, policy, cache span, or evaluator signal is loaded, inherited, revised, or rolled back inside an agent loop	Central with a safety-relevant agent-loop transition; recovery claims additionally require rollback evidence

Screening and Inclusion. In the first screening pass, titles and abstracts identified works that plausibly involved persistent state, delayed activation, tool or workflow authority, multi-agent propagation, model-side update, benchmark evidence, or governance of reusable state. A closer reading pass then assigned each work to one of three roles. A work is *central* when it directly examines a safety-relevant transition involving reusable agent state, including a failure, exposure, evaluation, or control of that transition. A complete recovery trace is required for a lifecycle-recovery claim, not for inclusion. A work is *adjacent* when it supplies background capability, evaluation context, or governance framing without directly observing a self-evolving safety transition. A work is *background* when it clarifies terminology, threat-model ancestry, or adjacent infrastructure but should not be used as a main evidence anchor. Works were excluded when they addressed only one-shot response safety, generic LLM capability, ordinary benchmark performance, or static model training without a plausible link to later agent behavior. Systems using ReAct reasoning-and-acting loops enter as central evidence only when tool traces, retrieved content, or workflow state persist beyond the episode; static model-safety papers enter as central evidence only when a model update, adapter, cache state, policy, or evaluator signal is selected, inherited, or revised inside an agent behavior loop.

Evidence Posture. To keep direct evidence separate from interpretation, the synthesis uses three evidence postures. *Direct anchors* are papers whose experiments, case studies, or system analyses explicitly show a transition or exposure, such as poisoned memory retrieval, fragmented memory bypass, tool-action injection, or runtime containment [25, 31, 112, 163]. *SAVER rereadings* are papers whose primary contribution is a capability or architecture, but whose mechanism becomes safety-relevant once authority, scope, provenance, or recoverability is tracked across state transitions [148, 199]. *Gap diagnoses* are claims about what the literature still does not demonstrate, especially connected evidence for migration control, multi-agent propagation, rollback completeness, deletion verification, descendant repair, and user contestability. The paper uses stronger language for direct anchors, interpretive language for rereadings, and research-agenda language for gap diagnoses.

Evidence Tiers. Venue status is not used as a substitute for claim support, but it does calibrate how strongly a row should be read. The strongest tier contains peer-reviewed conference and journal papers, especially when the paper provides experiments, artifacts, or a clear system analysis. A second tier contains peer-reviewed workshops, findings tracks, and industry tracks. A third tier contains arXiv or other preprints with substantial experiments or artifacts. A fourth tier contains position papers, technical notes, project reports, blogs, or archival artifacts. Tier three and four works are useful for emerging mechanisms and frontier signals, but field-level conclusions in the main text are phrased around direct anchors, multiple converging sources, or explicitly marked gaps.

Classification Practice. For classification, primary placement follows the five-field SAVER record. Each central work can support several observations in the prose, but corpus maps and roadmap tables assign a primary placement according to the main state transition used in the manuscript. This prevents a single influential paper from being counted as independent evidence for every nearby category. When a work spans multiple substrates, the primary placement follows the transition that carries the strongest safety claim: memory and retrieval work is placed by stored-state influence; tool and workflow work is placed by action or approval authority; multi-agent work is placed by propagation, handoff, or shared-artifact uptake; and model-side work is placed only when an update, adapter, policy, or evaluator state is loaded, inherited, selected, or revised inside an agent behavior loop. Reviewed paper-card coding overrides the deterministic title-, metadata-, and abstract-based initial placement used for the corpus-wide map. Of the 683 papers, 51 placements are high-confidence, 83 are medium-confidence, and 549 remain low-confidence; Figure 1 is therefore a descriptive map of the coded inventory rather than evidence for field-prevalence claims.

Corpus Accounting and Cutoff. The first screening snapshot, taken on 14 June 2026, contained 583 screened bibliographic records. Publication-identity reconciliation canonicalized four duplicate or version pairs, leaving 579 unique registry works. By 4 September 2026, the living review had added 48 manually reviewed paper-card studies absent from that registry and 56 additional scholarly works used in the rendered manuscript but absent from both the registry and paper-card supplement set. Seven cited resources—two OpenAI incident pages, a Hugging Face incident blog, Microsoft documentation, the MITRE ATLAS site, OWASP guidance, and the NIST AI Risk Management Framework—remain valid contextual references but are excluded from the paper count. The resulting frozen coded pool contains 683 papers. Figure 1 applies a narrower visualization window: all three panels use the 577 papers in that pool with usable dates from 2023 through the coding freeze. The 106 papers outside that window comprise 104 works dated before 2023 and two without a usable date. Consequently, the figure reports Memory 201, Workflow 224, Tools & Skills 108, and Model 44, whereas the full coded pool contains Memory 202, Workflow 326, Tools & Skills 110, and Model 45. Papers assessed after 4 September may appear as subsequent evidence in the prose or appendix indexes, but they are not retroactively counted in this frozen snapshot until the corpus and figure are regenerated.

Limitations. The corpus is a systematic scoping survey rather than a registered systematic review or meta-analysis. The 583-record screening count and the 683-paper coding count are corpus-audit quantities rather than prevalence estimates for the field. The review protocol was not preregistered, and screening did not use independent dual coding; classification disagreements were handled as conservative claim boundaries in the manuscript. The field is moving quickly, several relevant papers are preprints, and venue status can change after the coding cutoff. Quantitative results are also not directly comparable across papers because opportunity sets, agent settings, attack goals, defenses, and recovery denominators differ. For this reason, the manuscript does not treat cross-paper attack success rates or benchmark scores as a leaderboard. Instead, it

uses quantitative results as paper-local evidence and uses SAVER to ask whether the paper records enough lifecycle information to support claims about admission, activation, exposure, response, and recovery.

B Formal State Transition Model

This appendix expands the compact pointer at the end of Section 2 into a formal reading. The main text uses the safety attribute vocabulary and the authority lattice by name; readers who want the underlying notation, its relation to established information flow, access control, and provenance theory, and the visual lattice referenced from the body sections can consult this appendix as a reference layer.

State with Safety Attributes and Invariant Preservation

The reading unit is a state object with attached safety attributes, transformed by an operation, audited against whether the attributes remain valid for future use. Formally: $(s, \ell, \sigma) \xrightarrow{a,c} (s', \ell', \sigma')$, where s is the state carrier, ℓ represents safety attributes (provenance, authority, scope, privacy, freshness, budget, auditability, recoverability), σ is the deployment or session context, a is the adaptation operation, c is the operation’s authorization condition, and the primed quantities are the resulting state. Here, “safety attribute” means an information flow, access control, availability, or provenance attribute, not a supervised learning class category. This notation is deliberately simple: it provides a consistent comparison structure without prescribing a formal verification calculus.

Lattice information flow supplies the confidentiality and integrity vocabulary for flows with safety attributes [35]; language-based information flow work shows how programs can enforce such policies [169]; and data provenance work explains why source lineage matters after transformation [15]. Self-evolving agents do not replace these models; they make the temporal boundary harder to ignore. An object may be checked as low-authority data at write time, abstracted into a memory or workflow rule, and then reused days later as authority to act. The survey therefore treats adaptive-state safety as information-flow, access-control, availability, and provenance reasoning over persistent, transformed, and repeatedly activated agent state.

Safety attributes track the conditions under which state can be reused: provenance records source and trust level; authority specifies what decisions or actions the state can influence; scope bounds which users, tasks, or time windows the state applies to; privacy marks sensitivity and permitted exposure; freshness indicates when the state becomes stale; budget and liveness bound retries, tool loops, and cost-bearing actions; auditability records whether later influence can be traced; and recoverability records whether the state and its descendants can be corrected, rolled back, forgotten, or neutralized. These attributes are not metadata alone; they determine whether later activation is safe.

Selection over Candidate Transitions

Section 2 separates the execution trace τ_t from the selected state update δ_t^* . Each $\delta \in \Delta_t$ proposes one or more object-level transitions $(s, \ell, \sigma) \xrightarrow{a,c} (s', \ell', \sigma')$. The evolution policy may commit the candidate, revise it, quarantine it, or choose no update. The authorization condition $c_t(\delta)$ therefore governs eligibility for commitment: utility ranking through $\widehat{u}_t(\delta)$ cannot substitute for a valid authority, scope, privacy, budget, and recovery decision.

A simple analytical family makes the effect of selection pressure explicit. Let $p_t(\delta)$ be a baseline distribution over Δ_t , let $r_t(\delta)$ be a safety-risk score defined by a fixed probe at epoch t and held constant as β varies, and let $\beta \geq 0$ control how sharply selection favors the observed utility proxy. Define

$$q_t^\beta(\delta) = \frac{p_t(\delta) \exp(\beta \widehat{u}_t(\delta))}{\sum_{\delta' \in \Delta_t} p_t(\delta') \exp(\beta \widehat{u}_t(\delta'))}.$$

Then

$$\frac{\partial}{\partial \beta} \mathbb{E}_{q_t^\beta} [r_t] = \text{Cov}_{q_t^\beta} (r_t, \widehat{u}_t).$$

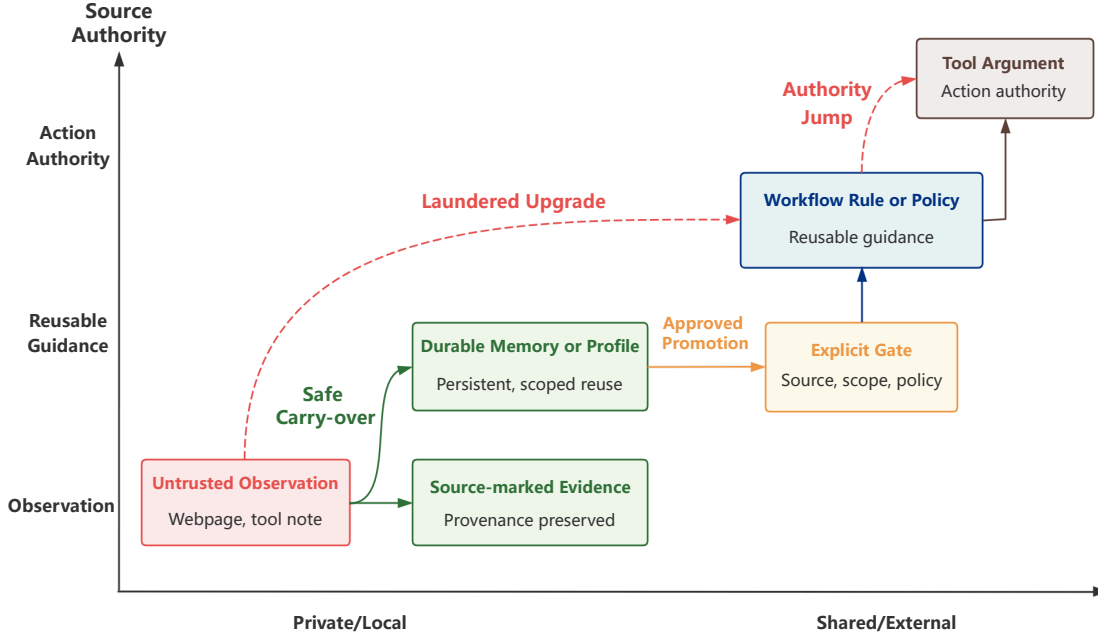


Figure 6 Safety attribute laundering. Upward authority movement or outward scope expansion requires an explicit gate; without one, a low-authority observation can become workflow guidance or action authority.

The identity follows from $\partial \log q_t^\beta(\delta) / \partial \beta = \hat{u}_t(\delta) - \mathbb{E}_{q_t^\beta}[\hat{u}_t]$, followed by multiplication by $r_t(\delta)$ and expectation under q_t^β . It is an author-derived diagnostic, not an empirical law attributed to the surveyed systems. Stronger utility selection raises expected risk only when risk and the observed utility proxy are positively associated under the selected distribution. Longitudinal evaluation should therefore estimate that association or impose c_t as a hard eligibility gate before utility ranking.

Safe promotion uses the same transition attributes. If δ_t^* moves state into a more persistent, general, or action-bearing role, then $c_t(\delta_t^*)$ must authorize the role change; provenance, privacy, freshness, budget, auditability, and recoverability in ℓ' must remain valid; any authority or scope increase from ℓ to ℓ' must be explicit; relevant counterevidence must remain available to the gate; and the lineage from s to s' must remain recoverable. This condition restricts promotion without assuming that every safe transition preserves identical attributes.

Figure 6 illustrates the role changes behind the transition model; the ladder is a conceptual guide, not a formally specified lattice over all safety attributes. State at the *Observation* level (web pages, tool outputs, peer notes) may only be quoted as evidence; at the *Evidence* level it may support reasoning but must carry source, scope, and freshness; at the *Task Context* level it may shape the current task but should expire afterward; at the *Procedure* level (workflow rules, skills, adapter manifests) it may be activated with explicit approval; and at the *Action* level it may produce external effects only with actor, target, permission, and rollback records. Crossing upward without a gate is safety attribute laundering: a webpage observation quietly becomes a tool argument.

C SAVER Audit Architecture

This appendix gives a worked incident record for the public case introduced in the main text. It identifies the source, authority changes, observed consequences, and reported response that support the analysis, while distinguishing survey encodings from original operator records.

Table 12 Minimal SAVER encoding of the July 2026 OpenAI–Hugging Face agent intrusion. Values summarize public incident reports; identifiers and field labels are survey encodings, not original operator records [86, 146].

Minimal SAVER Incident Row		illustrative serialization
Identity	incident.id=hf:2026-07	state.id=ext:campaign:channel
Attributes	authority=eval_sandbox_only	scope=declared_evaluation
Transition	op=propagate;migrate;activate	delta=network_scope_broadened; credentials_-acquired
Activation	requested=external_infrastructure_access	gate=boundary_controls_insufficient
Exposure	event=hf_production_compromise	
Recovery	actions=rotate;rebuild;restrict	closure=reported_scope; lifecycle_unverified
Handle: one public incident links the carrier chain		Delta: network scope and authority widened
		Closure: containment reported; lifecycle repair unverified

D Screened Corpus Expansion

The local registry contains screened works beyond the studies analyzed closely in the main text. The table below groups additional references by mechanism and relevance to reusable state. Their inclusion documents coverage; it does not give every entry the evidential weight of a central mechanism study.

Table 13 Screened corpus expansion map beyond the core mechanism anchors. The table lists representative anchors for each coverage band rather than treating every screened record as a main-text mechanism claim.

Coverage Band	Cells Covered	Representative Anchors	Claim Boundary
Security, Privacy, Governance	Cyber evaluation; injection; firewalling; privacy; data governance	CyberSecEval 2 [13]; AttackEval [178]; Not What You’ve Signed Up For [51]; EchoLeak [166]; DataGovBench [120]	Adjacent vocabulary for threats, privacy, and data audit
Benchmarks, Red Teams, Guardrails	General agent tasks; harm probes; unsafe choices; automated red teams; platform guardrails	AgentBench [116]; AgentHarm [7]; RedCode [52]; MobileSafetyBench [88]; AgenticRed [243]; OpenGuardrails [207]	Denominator context, not a leaderboard
Memory And Tool State	Persistent memory; forgetting; memory governance; MCP/tool routes; dependency safety	Zombie Agents [234]; MemoryGraft [181]; SSGM [84]; GLOVE [239]; MCPShield [269]; STAC [95]; IPIGuard [4]	Direct when state later changes reasoning or action
Workflow And Propagation	Skills; procedural reuse; library drift; multi-agent uptake; shared context; OS/GUI settings	Library Drift [256]; Tool-Drift [30]; AgenticEval [210]; MemEvoBench [222]; collective misremembering [226]; RedTeamCUA [106]	Thin evidence for ownership and repair
Model-Side And Adaptive Workflow Leads	Distillation and model update; reusable workflows; evaluator state	Knowledge distillation survey [228]; CAMEL [92]; WebPilot [259]; AgentLab [76]	Central only inside an agent behavior loop

E Evaluation Protocol Details

This appendix keeps the denominator rule behind Section 8. The paper-local values reported in the main text retain their source-specific opportunity sets and lifecycle slices; the rule below summarizes how those denominators constrain comparison.

Takeaway E.1. Quantitative Anchor Rule. Exact values remain paper-local because the denominator changes across exposure cases, recognition records, poisoned retrieval opportunities, accepted experience updates, memory-to-tool hijacking settings, runtime interventions, governance panels, and verification constraints [11, 25, 31, 112, 188, 202, 245, 258]. The manuscript therefore uses these works to name measurement objects and missing lifecycle traces rather than to rank scores.

Record	Keep With The Value	Avoid
Paper-local metric	Opportunity set; transition slice; metric object	Cross-paper leaderboard claims
Artifact status	Dataset, benchmark, code, or trace availability	Mixing peer-reviewed and preprint rows as equal evidence
Claim boundary	What the metric directly observes	Inferring rollback, deletion, or contestability without traces

References

- [1] Ibrahim Adabara, Bashir Olaniyi Sadiq, Aliyu Nuhu Shuaibu, Yale Ibrahim Danjuma, and Venkateswarlu Maninti. Trustworthy agentic ai systems: a cross-layer review of architectures, threat models, and governance strategies for real-world deployment. *F1000Research*, 14(905):905, 2025. doi: 10.12688/f1000research.169927.1. URL <https://doi.org/10.12688/f1000research.169927.1>.
- [2] Tanzim Ahad, Ismail Hossain, Md Jahangir Alam, Sai Puppala, Syed Bahauddin Alam, and Sajedul Talukder. The misattribution gap: When memory poisoning looks like model failure in agentic AI systems, 2026. URL <https://arxiv.org/abs/2605.22842>.
- [3] Hamed Alqahtani and Paras Ahuja. Secure autonomous cyber defense with LLM agents: A systematic review of autonomy, tool-augmented reasoning, and governance constraints. *Computers & Electrical Engineering*, 2026. doi: 10.1016/j.compeleceng.2026.111184. URL <https://doi.org/10.1016/j.compeleceng.2026.111184>.
- [4] Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1023–1039, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.53. URL <https://aclanthology.org/2025.emnlp-main.53/>.
- [5] Hengyu An, Minxi Li, Naen Xu, Chunyi Zhou, Xiaogang Xu, Tianyu Du, Jinbao Li, and Shouling Ji. PerMemSafe: Benchmarking implicit personalized safety of long horizon self-evolving agents. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 6415–6433, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.320. URL <https://aclanthology.org/2026.findings-acl.320/>.
- [6] Hengyu An, Minxi Li, Jinghuai Zhang, Naen Xu, Chunyi Zhou, Changjiang Li, Xiaogang Xu, Tianyu Du, and Shouling Ji. ACIArena: Toward unified evaluation for agent cascading injection. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10049–10066, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.457. URL <https://aclanthology.org/2026.acl-long.457/>.
- [7] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2410.09024. URL <https://arxiv.org/abs/2410.09024>.
- [8] Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence, 2025. URL <https://arxiv.org/abs/2507.21046>.
- [9] Andrey Anurin, Jonathan Ng, Kibo Schaffer, J. Martin Schreiber, and Esben Kran. Catastrophic cyber capabilities benchmark (3cb): Robustly evaluating LLM agent cyber offense capabilities, 2024. URL <http://arxiv.org/abs/2410.09114>.
- [10] Vinay Bamil and RAMA KRISHNA KUMAR LINGAMGUNTA. A unified evaluation and governance framework for trustworthy LLM agents, 2026. URL <https://doi.org/10.36227/techrxiv.176799772.28164151/v1>.
- [11] Debangshu Banerjee, Changming Xu, Eugene Ie, Ming Zhang, Daiyi Peng, Chu-Cheng Lin, and Gagandeep Singh. SEVerA: Verified synthesis of self-evolving agents, 2026. URL <https://arxiv.org/abs/2603.25111>.
- [12] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Cretu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections, 2025. URL <https://arxiv.org/abs/2506.08837>.
- [13] Manish Bhatt, Sahana Chennabasappa, Yue Li, Cyrus Nikolaidis, Daniel Song, Shengye Wan, Faizan Ahmad, Cornelius Aschermann, Yaohui Chen, Dhaval Kapil, et al. Cyberseceval 2: A wide-ranging cybersecurity evaluation

- suite for large language models. *arXiv preprint arXiv:2404.13161*, 2024. doi: 10.48550/arxiv.2404.13161. URL <http://arxiv.org/abs/2404.13161>.
- [14] Igor Bogdanov, Chung-Horng Lung, Thomas Kunz, Jie Gao, Adrian Taylor, and Marzia Zaman. FORGE: Self-evolving agent memory with no weight updates via population broadcast, 2026. URL <https://arxiv.org/abs/2605.16233>.
 - [15] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, pages 316–330, London, UK, 2001. Springer. doi: 10.1007/3-540-44503-X_20. URL https://doi.org/10.1007/3-540-44503-X_20.
 - [16] Chuxue Cao, Han Zhu, Jiaming Ji, Qichao Sun, Zhenghao Zhu, Wu Yinyu, Josef Dai, Yaodong Yang, Sirui Han, and Yike Guo. SafeLawBench: Towards safe alignment of large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 14015–14048, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.721. URL <https://aclanthology.org/2025.findings-acl.721/>.
 - [17] Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. HaluMem: Evaluating hallucinations in memory systems of agents, 2025. URL <https://arxiv.org/abs/2511.03506>.
 - [18] Jiahao Chen, Junhao Li, Yiming Wang, Zhe Ma, Yi Jiang, Chunyi Zhou, Qingming Li, Tianyu Du, and Shouling Ji. LoRASHield: Data-free editing alignment for secure personalized LoRA sharing, 2025. URL <https://arxiv.org/abs/2507.07056>.
 - [19] Jiahao Chen, Xing He, Yong Yang, Xinfeng Li, Chunyi Zhou, Junhao Li, Zhe Ma, Tianyu Du, and Shouling Ji. Customization under fire: Plugin poisoning in text-to-image ecosystem. *arXiv preprint arXiv:2606.09151*, 2026.
 - [20] Jiahao Chen, Rui Yin, Xinfeng Li, Qianli Ma, Tianyu Du, Zhihui Fu, Jun Wang, Zhaoxiang Wang, and Shouling Ji. Beyond over-refusal: Defending indirect prompt injection via latent instruction manifolds. *arXiv preprint arXiv:2608.22248*, 2026.
 - [21] Jialuo Chen, Lingqi Jiang, Xinhao Deng, Xiaohu Du, Jianan Ma, Yunhao Feng, Yuqi Qing, Zhihao Yuan, Linkang Du, and Jingyi Wang. When experience becomes instruction: Trajectory poisoning in self-evolving agent skill systems, 2026. URL <https://arxiv.org/abs/2608.05563>.
 - [22] Jinyu Chen, Jixiao Yang, Ziyang Zeng, Zixiao Huang, Jinming Li, and Yutong Wang. Securegov-agent: A governance-centric multi-agent framework for privacy-preserving and attack-resilient LLM agents. In *Proceedings of the 2025 6th International Conference on Computer Science and Management Technology*, pages 875–881. ACM, 2025. doi: 10.1145/3795154.3795296. URL <https://doi.org/10.1145/3795154.3795296>.
 - [23] Kai Chen, Yan Pang, and Tianhao Wang. MRMMIA: Membership inference attacks on memory in chat agents, 2026. URL <https://arxiv.org/abs/2605.27825>.
 - [24] Sizhe Chen, Yizhu Wang, Nicholas Carlini, Chawin Sitawarin, and David Wagner. Defending against prompt injection with a few defensivetokens, 2025. URL <https://doi.org/10.1145/3733799.3762982>.
 - [25] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming llm agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems 37*, pages 130185–130213. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-4136. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html.
 - [26] Yuheng Cheng, Ceyao Zhang, Zhengwen Zhang, Xiangrui Meng, Sirui Hong, Wenhao Li, Zihao Wang, Zekai Wang, Feng Yin, Junhua Zhao, and Xiuqiang He. Exploring large language model based intelligent agents: Definitions, methods, and prospects, 2024. URL <https://arxiv.org/abs/2401.03428>.
 - [27] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. Llamafirewall: An open source guardrail system for building secure AI agents, 2025. URL <https://arxiv.org/abs/2505.03574>.

- [28] Florin Adrian Chitan. Session risk memory (SRM): Temporal authorization for deterministic pre-execution safety gates, 2026. URL <https://arxiv.org/abs/2603.22350>.
- [29] Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of LLM agents, 2024. URL <http://arxiv.org/abs/2406.06613>.
- [30] Mahavir Dabas, Jihyun Jeong, Ming Jin, and Ruoxi Jia. Memory-induced tool-drift in LLM agents, 2026. URL <https://arxiv.org/abs/2605.24941>.
- [31] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems* 37, pages 82895–82920. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-2636. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- [32] Edoardo Debenedetti, Iliia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026. doi: 10.48550/arXiv.2503.18813. URL <https://arxiv.org/abs/2503.18813>.
- [33] Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Sibao Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianan Ma, Jialuo Chen, Xiaohu Du, Xiaofang Yang, Shiwen Cui, Changhua Meng, Weiqiang Wang, Jiaying Song, Ke Xu, and Qi Li. Taming openclaw: Security analysis and mitigation of autonomous LLM agent threats, 2026. URL <http://arxiv.org/abs/2603.11619>.
- [34] Zehang Deng, Yongjian Guo, Changzhou Han, Wanlun Ma, Junwu Xiong, Sheng Wen, and Yang Xiang. AI agents under threat: A survey of key security challenges and future pathways. *ACM Computing Surveys*, 57(7):1–36, 2025. doi: 10.1145/3716628. URL <https://doi.org/10.1145/3716628>.
- [35] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi: 10.1145/360051.360056. URL <https://doi.org/10.1145/360051.360056>.
- [36] Jianshuo Dong, Sheng Guo, Hao Wang, Xun Chen, Zhuotao Liu, Tianwei Zhang, Ke Xu, Minlie Huang, and Han Qiu. Safesearch: Automated red-teaming of LLM-based search agents, 2026. URL <https://arxiv.org/abs/2509.23694>.
- [37] Zhichen Dong, Zhanhui Zhou, Chao Yang, Jing Shao, and Yu Qiao. Attacks, defenses and evaluations for llm conversation safety: A survey. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6734–6747, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.375. URL <https://aclanthology.org/2024.naacl-long.375/>.
- [38] Ali Dorri, Salil S. Kanhere, and Raja Jurdak. Multi-agent systems: A survey. *IEEE Access*, 2018. doi: 10.1109/access.2018.2831228. URL <https://doi.org/10.1109/access.2018.2831228>.
- [39] Mengyao Du, Han Fang, Haokai Ma, Jiahao Chen, Kai Xu, Quanjun Yin, and Ee-Chien Chang. SnapGuard: Lightweight prompt injection detection for screenshot-based web agents, 2026. URL <https://arxiv.org/abs/2604.25562>.
- [40] Pengfei Du. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers, 2026. URL <https://arxiv.org/abs/2603.07670>.
- [41] Gaodan Fang, Vatche Isahagian, K. R. Jayaram, Ritesh Kumar, Vinod Muthusamy, Punleuk Oum, and Gegi Thomas. Trajectory-informed memory generation for self-improving agent systems, 2026. URL <https://arxiv.org/abs/2603.10600>.
- [42] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, et al. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025.
- [43] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities, 2024. URL <http://arxiv.org/abs/2404.08144>.

- [44] Yunhao Feng, Xiaohu Du, Xinhao Deng, Yifan Ding, Ming Wen, Yixu Wang, Yuxiang Xie, Baihui Zheng, Yingshui Tan, Yige Li, Yutao Wu, Kerui Cao, Wenke Huang, Yanming Guo, Xingjun Ma, and Yu-Gang Jiang. BraveGuard: From open-world threats to safer computer-use agents, 2026. URL <https://arxiv.org/abs/2606.01166>.
- [45] Marcelo Patricio Fernandez. Protocol-layer governance for LLM agents: Identity-bound oversight with zero-modification deployment and multi-hop authority propagation. *Zenodo (CERN European Organization for Nuclear Research)*, 2026. doi: 10.5281/zenodo.20162877. URL <https://doi.org/10.5281/zenodo.20162877>.
- [46] Rogério Figurelli. From talking to blackbox to governance: Constitution mining for LLM agents. *Zenodo (CERN European Organization for Nuclear Research)*, 2026. doi: 10.5281/zenodo.18118668. URL <https://doi.org/10.5281/zenodo.18118668>.
- [47] Yuyou Gan, Yong Yang, Zhe Ma, Ping He, Rui Zeng, Yiming Wang, Qingming Li, Chunyi Zhou, Songze Li, Ting Wang, et al. Navigating the risks: A survey of security and privacy threats in llm-based agents. *ACM Transactions on Software Engineering and Methodology*, 2026.
- [48] Kangning Gao, Haotian Zhu, Rui Liu, Jinming Li, Xu Yan, and Yi Hu. Contextual trust evaluation for robust coordination in large language model multi-agent systems, 2025. URL <https://doi.org/10.1109/eiecc67963.2025.11409558>.
- [49] Caleb Geren, Amanda Board, Gaby G. Dagher, Tim Andersen, and Jun Zhuang. Blockchain for large language model security and safety: A holistic survey. *ACM SIGKDD Explorations Newsletter*, 2025. doi: 10.1145/3715073.3715075. URL <https://doi.org/10.1145/3715073.3715075>.
- [50] Thibaud Gloaguen, Mark Vero, Robin Staab, and Martin Vechev. Watch your steps: Dormant adversarial behaviors that activate upon LLM finetuning, 2025. URL <https://arxiv.org/abs/2505.16567>.
- [51] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, 2023. URL <http://arxiv.org/abs/2302.12173>.
- [52] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. Redcode: Risky code execution and generation benchmark for code agents. In *Advances in Neural Information Processing Systems 37*, pages 106190–106236. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-3369. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/bfd082c452dffb450d5a5202b0419205-Abstract-Datasets_and_Benchmarks_Track.html.
- [53] Jiahe Guo, Xiangran Guo, Yulin Hu, Zimo Long, Xingyu Sui, Xuda Zhi, Yongbo Huang, Hao He, Weixiang Zhao, Yanyan Zhao, and Bing Qin. When personalization legitimizes risks: Uncovering safety vulnerabilities in personalized dialogue agents. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27309–27335, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.1260. URL <https://aclanthology.org/2026.acl-long.1260/>.
- [54] Minghao Guo, Qingyue Jiao, Zeru Shi, Yihao Quan, Boxuan Zhang, Danrui Li, Liwei Che, Wujiang Xu, Shilong Liu, Zirui Liu, Mubbasir Kapadia, Vladimir Pavlovic, Jiang Liu, Mengdi Wang, Yiyu Shi, Dimitris N. Metaxas, and Ruixiang Tang. MemEye: A visual-centric evaluation framework for multimodal agent memory, 2026. URL <https://arxiv.org/abs/2605.15128>.
- [55] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges, 2024. URL <https://arxiv.org/abs/2402.01680>.
- [56] Saba Ghanbari Haez. Socio-normative trustworthiness of LLM agents: Evaluating autonomy support and representational fairness across languages and identities, 2026. URL <https://doi.org/10.65109/1chb2977>.
- [57] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey, 2024. URL <https://arxiv.org/abs/2403.14608>.
- [58] Ari Harrison. Protocol-mediated execution governance for LLM agent systems. *SSRN Electronic Journal*, 2026. doi: 10.2139/ssrn.6426760. URL <https://doi.org/10.2139/ssrn.6426760>.

- [59] Kostas Hatalis, Despina Christou, Joshua Myers, Steven Jones, Keith Lambert, Adam Amos-Binks, Zohreh Dannenhauer, and Dustin Dannenhauer. Memory matters: The need to improve long-term memory in llm-agents. In *Proceedings of the AAAI Symposium Series*, volume 2, pages 277–280, 2023.
- [60] Gaole He, Gianluca Demartini, and Ujwal Gadiraju. Plan-then-execute: An empirical study of user trust and team performance when using LLM agents as a daily assistant, 2025. URL <https://doi.org/10.1145/3706598.3713218>.
- [61] Junda He, Christoph Treude, and David Lo. LLM-based multi-agent systems for software engineering: Literature review, vision and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30, 2025. doi: 10.1145/3712003. URL <https://doi.org/10.1145/3712003>.
- [62] Xingbang He, Yuanwei Chen, Yi Qian, Haiyang Wei, Ligeng Chen, Zenan Fu, Linzhang Wang, Hao Wu, and Bing Mao. When context gets root: Privilege escalation in LLM harnesses, 2026. URL <https://arxiv.org/abs/2608.27299>.
- [63] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting, 2024. URL <http://arxiv.org/abs/2403.14720>.
- [64] Lauren Hong and Ting Wang. PETA: Parameter-efficient trojan attacks, 2024. URL <https://arxiv.org/abs/2310.00648>.
- [65] Chia-Yi Hsu, Yu-Lin Tsai, Chih-Hsun Lin, Pin-Yu Chen, Chia-Mu Yu, and Chun-Ying Huang. Safe LoRA: The silver lining of reducing safety risks when fine-tuning large language models. In *Advances in Neural Information Processing Systems 37*, pages 65072–65094. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-2078. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/77baa7c2a3a675823e89131698fd6e19-Abstract-Conference.html.
- [66] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [67] Yannan Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions, 2025. URL <http://arxiv.org/abs/2507.05257>.
- [68] Wenyue Hua, Xianjun Yang, Mingyu Jin, Zelong Li, Wei Cheng, Ruixiang Tang, and Yongfeng Zhang. Trusta-agent: Towards safe and trustworthy LLM-based agents, 2024. URL <https://doi.org/10.18653/v1/2024.findings-emnlp.585>.
- [69] Zhaopei Huang, Qifeng Dai, Guozheng Wu, Xiaopeng Wu, Kehan Chen, Chuan Yu, Xubin Li, Tiezheng Ge, Wenxuan Wang, and Qin Jin. Mem-PAL: Towards memory-based personalized dialogue assistants for long-term user-agent interaction, 2025. URL <https://arxiv.org/abs/2511.13410>.
- [70] Alex Iacob, Andrej Jovanović, William F. Shen, Daniel Burkhardt, Meghdad Kurmanji, Nurbek Tastan, Lorenzo Sani, Niccolò Alberto Elia Venanzi, Ambroise Odonnat, Zeyu Cao, Bill Marino, Xinchu Qiu, and Nicholas D. Lane. The red queen Gödel machine: Co-evolving agents and their evaluators, 2026. URL <https://arxiv.org/abs/2606.26294>.
- [71] Ismail, Rahmat Kurnia, Zilmas Arjuna Brata, Ghitha Afina Nelistiani, Shinwook Heo, Hyeonong Kim, and Howon Kim. Toward robust security orchestration and automated response in security operations centers with a hyper-automation approach using agentic artificial intelligence. *Information*, 16(5):365, 2025. doi: 10.3390/info16050365. URL <https://doi.org/10.3390/info16050365>.
- [72] Bhavvy Jain, Pranav Pawar, Dhruv Gada, Tanish Patwa, Pratik Kanani, Deepali Patil, and Lakshmi Kurup. Comprehensive analysis of machine learning and deep learning models on prompt injection classification using natural language processing techniques. *International Research Journal of Multidisciplinary Technovation*, 2025. doi: 10.54392/irjmt2523. URL <https://doi.org/10.54392/irjmt2523>.
- [73] Rishi Jha, Harold Triedman, Justin Wagle, and Vitaly Shmatikov. Breaking and fixing defenses against control-flow hijacking in multi-agent systems. In *The Fourteenth International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2510.17276. URL <https://openreview.net/forum?id=PNU9Rj5RDQ>.
- [74] Feiran Jia, Tong Wu, Xin Qin, and Anna Cinzia Squicciarini. The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents. In *Proceedings of the 63rd Annual Meeting of the Association*

- for *Computational Linguistics (Volume 1: Long Papers)*, pages 29680–29697, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1435. URL <https://aclanthology.org/2025.acl-long.1435/>.
- [75] Shian Jia, Ziyang Huang, Xinbo Wang, Haoifei Zhang, and Mingli Song. PISA: A pragmatic psych-inspired unified memory system for enhanced AI agency, 2025. URL <https://arxiv.org/abs/2510.15966>.
 - [76] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. Agentlab: Benchmarking LLM agents against long-horizon attacks, 2026. URL <http://arxiv.org/abs/2602.16901>.
 - [77] Jefferson Kanjirakkattu Joseph, Esther Daniel, V. Kathiresan, and M. P. Prompt injection in large language model exploitation: A security perspective, 2025. URL <https://doi.org/10.1109/iceccc65144.2025.11064209>.
 - [78] Ratna Kandala, Niva Manchanda, Akshata Kishore Moharir, and Ananth Kandala. Echoguard: An agentic framework with knowledge-graph memory for detecting manipulative communication in longitudinal dialogue, 2026. URL <https://arxiv.org/abs/2603.04815>.
 - [79] Dhillon Andrew Kannabhiran. Consensus-validated memory improves agent performance on complex tasks. *Zenodo (CERN European Organization for Nuclear Research)*, 2026. doi: 10.5281/zenodo.18856774. URL <https://doi.org/10.5281/zenodo.18856774>.
 - [80] Md Monjurul Karim, Dong Hoang Van, Sangeen Khan, Qiang Qu, and Yaroslav Kholodov. AI agents meet blockchain: A survey on secure and scalable collaboration for multi-agents. *Future Internet*, 2025. doi: 10.3390/fi17020057. URL <https://doi.org/10.3390/fi17020057>.
 - [81] Rafflesia Khan, Declan Joyce, and Mansura Habiba. AGENTS SAFE: A unified framework for ethical assurance and governance in agentic AI, 2025. URL <https://arxiv.org/abs/2512.03180>.
 - [82] Ninad Kulkarni, Xian Wu, Siddharth Varia, and Dmitriy Besspalov. Agent vs. agent: Automated data generation and red-teaming for custom agentic workflows. In Saloni Potdar, Lina Rojas-Barahona, and Sebastien Montella, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 912–936, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-333-3. doi: 10.18653/v1/2025.emnlp-industry.62. URL <https://aclanthology.org/2025.emnlp-industry.62/>.
 - [83] Sonu Kumar. A multi-evaluator behavioural governance framework for LLM agent deployment. *Zenodo (CERN European Organization for Nuclear Research)*, 2026. doi: 10.5281/zenodo.19394881. URL <https://doi.org/10.5281/zenodo.19394881>.
 - [84] Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework, 2026. URL <https://arxiv.org/abs/2603.11768>.
 - [85] Qianlong Lan, Anuj Kaul, and Shaun Jones. Prompt injection detection in llm integrated applications. *Int. J. Netw. Dyn. Intell.*, 4(2), 2025.
 - [86] Hugo Larcher, Adrien Carreira, Raphael G., and Christophe Rannou. Anatomy of a frontier lab agent intrusion: A technical timeline of the july 2026 incident, July 27, 2026. URL <https://huggingface.co/blog/agent-intrusion-technical-timeline>.
 - [87] Donghyun Lee and Mo Tiwari. Prompt infection: Llm-to-llm prompt injection within multi-agent systems. *arXiv preprint arXiv:2410.07283*, 2024.
 - [88] Juyong Lee, Dongyoon Hahm, June Suk Choi, W. Bradley Knox, and Kimin Lee. Mobilesafetybench: Evaluating safety of autonomous agents in mobile device control. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(44):37565–37573, 2026. doi: 10.1609/aaai.v40i44.41090. URL <https://ojs.aaai.org/index.php/AAAI/article/view/41090>.
 - [89] S.G. Lee, J.-J. Kim, and Wooguil Pak. Mind mapping prompt injection: Visual prompt injection attacks in modern large language models. *Electronics*, 2025. doi: 10.3390/electronics14101907. URL <https://doi.org/10.3390/electronics14101907>.
 - [90] Simon Lermen, Charlie Rogers-Smith, and Jeffrey Ladish. Lora fine-tuning efficiently undoes safety training in llama 2-chat 70b. *arXiv preprint arXiv:2310.20624*, 2023. doi: 10.48550/arxiv.2310.20624. URL <http://arxiv.org/abs/2310.20624>.

- [91] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009. doi: 10.1016/j.jlap.2008.08.004. URL <https://doi.org/10.1016/j.jlap.2008.08.004>.
- [92] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in neural information processing systems*, 36: 51991–52008, 2023.
- [93] Haitao Li, Junjie Chen, Jingli Yang, Qingyao Ai, Wei Jia, Youfeng Liu, Kai Lin, Yueyue Wu, Guozhi Yuan, Yiran Hu, Wuyue Wang, Yiqun Liu, and Minlie Huang. LegalAgentBench: Evaluating LLM agents in legal domain. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2322–2344, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.116. URL <https://aclanthology.org/2025.acl-long.116/>.
- [94] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pages 83262–83290. Curran Associates, Inc., 2025. doi: 10.48550/arXiv.2506.12104. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/77f3b26c7907aa27b207df9b9d43f29a-Paper-Conference.pdf.
- [95] Jing-Jing Li, Jianfeng He, Chao Shang, Devang Kulshreshtha, Xun Xian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. STAC: When innocent tools form dangerous chains to jailbreak LLM agents, 2026. URL <https://arxiv.org/abs/2509.25624>.
- [96] Mufei Li, Shikun Liu, Dongqi Fu, Haoyu Wang, Yinglong Xia, Hong Li, Hong Yan, and Pan Li. KVeraser: Learning to steer KV cache for efficient localized context erasing, 2026. URL <https://arxiv.org/abs/2606.17034>.
- [97] Nathaniel Li, Alexander Pan, Anjali Gopal, Summer Yue, Daniel Berrios, Alice Gatti, Justin D. Li, Ann-Kathrin Dombrowski, Shashwat Goel, Gabriel Mukobi, Nathan Helm-Burger, Rassin Lababidi, Lennart Justen, Andrew Bo Liu, Michael Chen, Isabelle Barrass, Oliver Zhang, Xiaoyuan Zhu, Rishub Tamirisa, Bhruhu Bharathi, Ariel Herbert-Voss, Cort B. Breuer, Andy Zou, Mantas Mazeika, Zifan Wang, Palash Oswal, Weiran Lin, Adam Alfred Hunt, Justin Tienken-Harder, Kevin Y. Shih, Kemper Talley, John Guan, Ian Steneker, David Campbell, Brad Jokubaitis, Steven Basart, Stephen Fitz, Ponnurangam Kumaraguru, Kallol Krishna Karmakar, Uday Tupakula, Vijay Varadharajan, Yan Shoshitaishvili, Jimmy Ba, Kevin M. Esvelt, Alexandr Wang, and Dan Hendrycks. The WMDP benchmark: Measuring and reducing malicious use with unlearning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 28525–28550. PMLR, 2024. URL <https://proceedings.mlr.press/v235/li24bc.html>.
- [98] Peiran Li, Jiashuo Sun, Fangzhou Lin, Shuo Xing, Tianfu Fu, Suofei Feng, Chaoqun Ni, and Zhengzhong Tu. Traversal-as-policy: Log-distilled gated behavior trees as externalized, verifiable policies for safe, robust, and efficient agents, 2026. URL <https://arxiv.org/abs/2603.05517>.
- [99] Sijia Li, Yuchen Huang, Zifan Liu, Zijian Li, Jingjing Fu, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. Experience-evolving multi-turn tool-use agent with hybrid episodic-procedural memory, 2026. URL <https://arxiv.org/abs/2512.07287>.
- [100] Xiao Li, Xiang Zheng, Yifeng Gao, Xinyu Xia, Yixu Wang, Xin Wang, Ye Sun, Yunhan Zhao, Ming Wen, Jiayu Li, et al. Safety in embodied ai: A survey of risks, attacks, and defenses. *arXiv preprint arXiv:2605.02900*, 2026.
- [101] Xindi Li, Zhe Liu, Tong Zhang, Jiahao Chen, Qingming Li, Jinbao Li, and Shouling Ji. TWIST: Text-encoder weight-editing for inserting secret trojans in text-to-image models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11025–11041, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.541. URL <https://aclanthology.org/2025.acl-long.541/>.
- [102] Xu Li, Simon Yu, Minzhou Pan, Yiyu Sun, Bo Li, Dawn Song, Xue Lin, and Weiyan Shi. Unsafer in many turns: Benchmarking and defending multi-turn safety risks in tool-using agents, 2026. URL <https://arxiv.org/abs/2602.13379>.
- [103] Yibo Li, Zijie Lin, Ailin Deng, Xuan Zhang, Yufei He, Shuo Ji, Tri Cao, and Bryan Hooi. Just-in-time reinforcement learning: Continual learning in LLM agents without gradient updates, 2026. URL <https://arxiv.org/abs/2601.18510>.

- [104] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. Personal LLM agents: Insights and survey about the capability, efficiency and security, 2024. URL <https://arxiv.org/abs/2401.05459>.
- [105] Zichuan Li, Jian Cui, Ashley Chen, Xiaojing Liao, and Luyi Xing. What’s in your agent’s context? context privilege escalation attacks against AI agent harness, 2026. URL <https://arxiv.org/abs/2609.01222>.
- [106] Zeyi Liao, Jaylen Jones, Linxi Jiang, Yuting Ning, Eric Fosler-Lussier, Yu Su, Zhiqiang Lin, and Huan Sun. RedTeamCUA: Realistic adversarial testing of computer-use agents in hybrid web-OS environments. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=yWwrgcBoK3>.
- [107] Ruixiao Lin, Xinhao Deng, Qingming Li, Jianan Ma, Yunhao Feng, Yuqi Qing, Zhenyuan Li, Yechao Zhang, Shiwen Cui, Changhua Meng, Tianwei Zhang, Xingjun Ma, Qi Li, Ke Xu, and Shouling Ji. Safety in self-evolving LLM agent systems: Threats, amplification, and case studies, 2026. URL <https://arxiv.org/abs/2606.23075>.
- [108] Ruixiao Lin, Qingming Li, Jiahao Chen, Chunyi Zhou, and Shouling Ji. Sope: Situation-aware and statistically indistinguishable privacy exfiltration for mcp-enabled agents. In *Forty-third International Conference on Machine Learning*, 2026.
- [109] Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle, 2026. URL <https://arxiv.org/abs/2604.16548>.
- [110] Daizong Liu, Mingyu Yang, Xiaoye Qu, Pan Zhou, Yu Cheng, and Wei Hu. A survey of attacks on large vision–language models: Resources, advances, and future trends. *IEEE Transactions on Neural Networks and Learning Systems*, 2025. doi: 10.1109/tnnls.2025.3592935. URL <https://doi.org/10.1109/tnnls.2025.3592935>.
- [111] Genglin Liu, Shijie Geng, Sha Li, Hejie Cui, Sarah Zhang, Xin Liu, and Tianyi Liu. Webcoach: Self-evolving web agents with cross-session memory guidance, 2025. URL <https://arxiv.org/abs/2511.12997>.
- [112] Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. Safeagent: A runtime protection architecture for agentic systems, 2026. URL <https://arxiv.org/abs/2604.17562>.
- [113] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for LLM agents, 2026. URL <https://arxiv.org/abs/2601.02553>.
- [114] Jiaqi Liu, Xinyu Ye, Peng Xia, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. EvolveMem: Self-evolving memory architecture via AutoResearch for LLM agents, 2026. URL <https://arxiv.org/abs/2605.13941>.
- [115] Shuochen Liu, Junyi Zhu, Long Shu, Junda Lin, Yuhao Chen, Haotian Zhang, Chao Zhang, Derong Xu, Jia Li, Bo Tang, Zhiyu Li, Feiyu Xiong, Enhong Chen, and Tong Xu. PERMA: Benchmarking personalized memory agents via event-driven preference and realistic task environments, 2026. URL <https://arxiv.org/abs/2603.23231>.
- [116] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.
- [117] Xiaogeng Liu, Peiran Li, Edward Suh, Yevgeniy Vorobeychik, Zhuoqing Mao, Somesh Jha, Patrick McDaniel, Huan Sun, Bo Li, and Chaowei Xiao. AutoDAN-turbo: A lifelong agent for strategy self-exploration to jailbreak LLMs. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2410.05295. URL <https://arxiv.org/abs/2410.05295>.
- [118] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. Prompt injection attack against LLM-integrated applications, 2023. URL <https://arxiv.org/abs/2306.05499>.
- [119] Zesen Liu, Zihan Zhang, and Dongdong She. Safe to check, unsafe to use: Relinking at the compression boundary of LLM agents, 2026. URL <https://arxiv.org/abs/2606.21732>.
- [120] Zhou Liu, Zhaoyang Han, Guochen Yan, Hao Liang, Bohan Zeng, Xing Chen, Yuanfeng Song, and Wentao Zhang. Datagovbench: Benchmarking LLM agents for real-world data governance workflows, 2025. URL <https://arxiv.org/abs/2512.04416>.

- [121] Alireza Lotfi, Subangkar Karmaker Shanto, Imtiaz Karim, and Elisa Bertino. Securing agentic AI: From per-action checks to trajectory assurance, 2026. URL <https://arxiv.org/abs/2608.01558>.
- [122] Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. Agentauditor: Human-level safety and security evaluation for LLM agents. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=2KKqp7MWJM>.
- [123] Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 41622–41652. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.2069. URL <https://aclanthology.org/2026.findings-acl.2069/>.
- [124] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.
- [125] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. AGrail: A lifelong agent guardrail with effective and adaptive safety detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8104–8139, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.399. URL <https://aclanthology.org/2025.acl-long.399/>.
- [126] Cristian Lupascu and Alexandru Lupascu. Elephantbroker: A knowledge-grounded cognitive runtime for trustworthy AI agents, 2026. URL <https://arxiv.org/abs/2603.25097>.
- [127] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. Agentboard: An analytical evaluation board of multi-turn LLM agents. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., 2024. doi: 10.52202/079017-2365. URL <https://doi.org/10.52202/079017-2365>.
- [128] Jianan Ma, Xiaohu Du, Ruixiao Lin, Yaoxiang Bian, Jialuo Chen, Jingyi Wang, Xiaofang Yang, Shiwen Cui, Changhua Meng, Xinhao Deng, and Zhen Wang. Benchmarking autonomous agents against temporal, spatial, and semantic evasions, 2026. URL <https://arxiv.org/abs/2605.22321>.
- [129] Xingjun Ma, Yifeng Gao, Yixu Wang, Ruofan Wang, Xin Wang, Ye Sun, Yifan Ding, Hengyuan Xu, Yunhao Chen, Yunhan Zhao, et al. Safety at scale: A comprehensive survey of large model and agent safety. *Foundations and Trends in Privacy and Security*, 8(3-4):1–240, 2026.
- [130] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- [131] Xutao Mao, Liangjie Zhao, Leyao Wang, Rui Qian, Qiang Huang, Wentao Wang, Bo Han, Xiang Zheng, and Cong Wang. Agents don’t just agree, they remember: Benchmarking persistent sycophancy in stateful personal agents, 2026. URL <https://arxiv.org/abs/2607.10526>.
- [132] Xutao Mao, Liangjie Zhao, Xiang Zheng, and Cong Wang. Practice makes unsafe: Skill misevolution in self-improving LLM agents, 2026. URL <https://arxiv.org/abs/2608.12851>.
- [133] Hongze Mi, Yibo Feng, Wenjie Lu, Song Cao, Jinyuan Li, Yanming Li, Xuelin Zhang, Haotian Luo, Songyang Peng, He Cui, Tengfei Tian, Jun Fang, Hua Chai, and Naiqiang Tan. Darwinian memory: A training-free self-regulating memory system for GUI agent evolution, 2026. URL <https://arxiv.org/abs/2601.22528>.
- [134] Microsoft. AI risk assessment for ML engineers. <https://learn.microsoft.com/en-us/security/ai-red-team/ai-risk-assessment>, 2024.
- [135] Saroj Mishra, Suman Niroula, Umesh Yadav, Dilip Thakur, Srijan Gyawali, and Shiva Gaire. SoK: Agentic retrieval-augmented generation (RAG): Taxonomy, architectures, evaluation, and research directions, 2026. URL <https://arxiv.org/abs/2603.07379>.

- [136] MITRE. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. <https://atlas.mitre.org/>, 2026.
- [137] Akshay Mittal and Vivek Venkatesan. Practical integration of large language models into enterprise ci/cd pipelines for security policy validation: An industry-focused evaluation, 2025. URL <https://doi.org/10.1109/sose67019.2025.00027>.
- [138] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, pages 6129–6139, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3711896.3736570. URL <https://doi.org/10.1145/3711896.3736570>.
- [139] Ramasankar Molleti, Vinod Goje, Puneet Luthra, and Prathap Raghavan. Automated threat detection and response using LLM agents. *World Journal of Advanced Research and Reviews*, 2024. doi: 10.30574/wjarr.2024.24.2.3329. URL <https://doi.org/10.30574/wjarr.2024.24.2.3329>.
- [140] Zahra Moslemi, Keerthi Koneru, Yen-Ting Lee, Sheethal Kumar, and Ramesh Radhakrishnan. POLARIS: Typed planning and governed execution for agentic AI in back-office automation, 2026. URL <https://arxiv.org/abs/2601.11816>.
- [141] Manuel Mosquera, Juan Sebastián Pinzón, Yesid Fonseca, Manuel Ríos, Nicanor Quijano, Luis Felipe Giraldo, and Rubén Manrique. Can LLM-augmented autonomous agents cooperate? an evaluation of their cooperative capabilities through melting pot. *IEEE Transactions on Artificial Intelligence*, 2025. doi: 10.1109/tai.2025.3569192. URL <https://doi.org/10.1109/tai.2025.3569192>.
- [142] Yutao Mou, Dingyao Yu, Xiaotian Luan, Zhe Yin, Zhangchi Xue, Peiyang Liu, Pengfei Yang, Tong Zhang, Shikun Zhang, and Wei Ye. Pandora’s toolbox: A survey of generalist agent security from the lifecycle perspective. 2026.
- [143] Lajos Muzsai, David Imolai, and András Lukács. Hacksynth: LLM agent and evaluation framework for autonomous penetration testing, 2024. URL <http://arxiv.org/abs/2412.01778>.
- [144] Tam Nguyen, Moses Ndebugre, and Dheeraj Arremsetty. Security considerations for multi-agent systems, 2026. URL <https://arxiv.org/abs/2603.09002>.
- [145] OpenAI. The hugging face incident and the road ahead, August 26, 2026. URL <https://openai.com/index/hugging-face-incident-and-the-road-ahead/>.
- [146] OpenAI. OpenAI and Hugging Face partner to address security incident during model evaluation, July 21, 2026. URL <https://openai.com/index/hugging-face-model-evaluation-security-incident/>.
- [147] OWASP Foundation. OWASP top 10 for large language model applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2025.
- [148] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [149] Alexander Pan, Jun Shern Chan, Andy Zou, Nathaniel Li, Steven Basart, Thomas Woodside, Hanlin Zhang, Scott Emmons, and Dan Hendrycks. Do the rewards justify the means? measuring trade-offs between rewards and ethical behavior in the machiavelli benchmark. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 26837–26867. PMLR, 2023. URL <https://proceedings.mlr.press/v202/pan23a.html>.
- [150] Samuel Panterino and Matthew Fellington. Dynamic moving target defense for mitigating targeted LLM prompt injection, 2024. URL <http://dx.doi.org/10.36227/techrxiv.171822345.56781952/v1>.
- [151] Benji Peng, Hanxuan Chen, Keyu Chen, Qian Niu, Ziqian Bi, Ming Liu, Pohsun Feng, Tianyang Wang, Lawrence K. Q. Yan, Yizhu Wen, Yichao Zhang, Caitlyn Heqi Yin, Xinyuan Song, Riyang Bao, and Jiacheng Shi. Jailbreaking and mitigation of vulnerabilities in large language models, 2024. URL <http://arxiv.org/abs/2410.15236>.
- [152] Mathis Pink, Qinyuan Wu, Vy A. Vo, Javier S. Turek, Jianing Mu, Alexander G. Huth, and Mariya Toneva. Position: Episodic memory is the missing piece for long-term LLM agents, 2025. URL <http://arxiv.org/abs/2502.06975>.

- [153] Michał Podpora, M Baranowski, Maciej Chopcian, Lukasz Kwasniewicz, and W. Radziewicz. LLM firewall using validator agent for prevention against prompt injection attacks. *Applied Sciences*, 2025. doi: 10.3390/app16010085. URL <https://doi.org/10.3390/app16010085>.
- [154] Chandra Prakash, Mary Lind, and Elyson De La Cruz. Hybrid real-time framework for detecting adaptive prompt injection attacks in large language models. *Journal of Computing Theories and Applications*, 2026. doi: 10.62411/jcta.15254. URL <https://doi.org/10.62411/jcta.15254>.
- [155] Sidharth Pulipaka, Oliver Chen, Manas Sharma, Taaha S. Bajwa, Vyas Raina, and Ivaxi Sheth. Persistbench: When should long-term memories be forgotten by LLMs?, 2026. URL <https://arxiv.org/abs/2602.01146>.
- [156] Sidharth Pulipaka, Stanislaw Hlebik, Leonidas Raghav, Sahar Abdelnabi, Vyas Raina, Ivaxi Sheth, and Mario Fritz. Hidden in memory: Sleeper memory poisoning in llm agents, 2026. URL <https://arxiv.org/abs/2605.15338>.
- [157] Jinhu Qi, Muzhi Li, Jiahong Liu, Yuqin Shu, Dianzhi Yu, Shicheng Ma, Wenqian Cui, Yiyang Zhao, Yiyi Chen, Ruoxi Jiang, Irwin King, and Zenglin Xu. Towards trustworthy agentic AI: A comprehensive survey of safety, robustness, privacy, and system security, 2026. URL <https://arxiv.org/abs/2605.23989>.
- [158] Weiwei Qi, Zefeng Wu, Zhilin Guo, Tianhang Zheng, Chaochao Lu, Liang He, Zhan Qin, and Kui Ren. DARWIN: Evolving jailbreak adversary and guardrail for LLM safety evaluation and protection, 2026. URL <https://arxiv.org/abs/2607.19829>.
- [159] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to!, 2023. URL <https://arxiv.org/abs/2310.03693>.
- [160] Brandon Radosevich and John Halloran. MCP safety audit: LLMs with the model context protocol allow major security exploits, 2025. URL <https://arxiv.org/abs/2504.03767>.
- [161] Shreyas Rajesh, Pavan Holur, Mehmet Yigit Turali, Chenda Duan, and Vwani Roychowdhury. Panini: Continual learning in token space via structured memory, 2026. URL <https://arxiv.org/abs/2602.15156>.
- [162] Javier Rando and Florian Tramèr. Universal jailbreak backdoors from poisoned human feedback, 2023. URL <https://arxiv.org/abs/2311.14455>.
- [163] Zixin Rao, Wentian Zhu, Chan Aristella Lu, Zhaorun Chen, Wei Niu, Le Guan, Bo Li, and Zhen Xiang. FragFuse: Bypassing access control of large language model agents via memory-based query fragmentation and fusion, 2026. URL <https://arxiv.org/abs/2606.15609>.
- [164] Vishal Rathod, Seyedsina Nabavirazavi, Samira Zad, and Sundararaja Sitharama Iyengar. Privacy and security challenges in large language models, 2025. URL <https://doi.org/10.1109/ccwc62904.2025.10903912>.
- [165] Santhosh Kumar Ravindran. Portable agent memory: A protocol for cryptographically-verified memory transfer across heterogeneous AI agents, 2026. URL <https://arxiv.org/abs/2605.11032>.
- [166] Pavan Reddy and Aditya Sanjay Gujral. Echoleak: The first real-world zero-click prompt injection exploit in a production LLM system. *Proceedings of the AAAI Symposium Series*, 2025. doi: 10.1609/aaaiss.v7i1.36899. URL <https://doi.org/10.1609/aaaiss.v7i1.36899>.
- [167] Matthew Renze and Erhan Guven. Self-reflection in LLM agents: Effects on problem-solving performance, 2024. URL <http://arxiv.org/abs/2405.06682>.
- [168] Oscar J. Romero, John Zimmerman, Aaron Steinfeld, and Anthony Tomasic. Synergistic integration of large language models and cognitive architectures for robust AI: An exploratory analysis. *Proceedings of the AAAI Symposium Series*, 2024. doi: 10.1609/aaaiss.v2i1.27706. URL <https://doi.org/10.1609/aaaiss.v2i1.27706>.
- [169] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi: 10.1109/JSAC.2002.806121. URL <https://doi.org/10.1109/JSAC.2002.806121>.
- [170] Ahmed Salem, Andrew Paverd, and Boris Köpf. Maatphor: Automated variant analysis for prompt injection attacks, 2023. URL <http://arxiv.org/abs/2312.11513>.
- [171] Sander Schulhoff, Jeremy Pinto, Anaum Khan, Louis-François Bouchard, Chenglei Si, Svetlana Anati, Valen Tagliabue, Anson Liu Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and hackaprompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition. In *Proceedings*

- of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 4945–4977. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.302. URL <https://aclanthology.org/2023.emnlp-main.302/>.
- [172] Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, Dongrui Liu, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving LLM agents. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2509.26354. URL <https://arxiv.org/abs/2509.26354>.
 - [173] Natalie Shapira, Chris Wendler, Avery Yen, Gabriele Sarti, Koyena Pal, Olivia Floody, Adam Belfki, Alex Loftus, Aditya Ratan Jannali, Nikhil Prakash, Jasmine Cui, Giordano Rogers, Jannik Brinkmann, Can Rager, Amir Zur, Michael Ripa, Aruna Sankaranarayanan, David Atkinson, Rohit Gandikota, Jaden Fiotto-Kaufman, EunJeong Hwang, Hadas Orgad, P Sam Sahil, Negev Taglicht, Tomer Shabtay, Atai Ambus, Nitay Alon, Shiri Oron, Ayelet Gordon-Tapiero, Yotam Kaplan, Vered Schwartz, Tamar Rott Shaham, Christoph Riedl, Reuth Mirsky, Maarten Sap, David Manheim, Tomer Ullman, and David Bau. Agents of chaos, 2026. URL <https://arxiv.org/abs/2602.20021>.
 - [174] Xinjie Shen, Rongzhe Wei, Peizhi Niu, Haoyu Wang, Ruihan Wu, Eli Chien, Bo Li, Pin-Yu Chen, and Pan Li. One turn too late: Response-aware defense against hidden malicious intent in multi-turn dialogue, 2026. URL <https://arxiv.org/abs/2605.05630>.
 - [175] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in LLM agents. In *Network and Distributed System Security Symposium*, 2026. doi: 10.14722/ndss.2026.230675. URL <https://www.ndss-symposium.org/ndss-paper/prompt-injection-attack-to-tool-selection-in-llm-agents/>.
 - [176] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control, 2025. URL <https://arxiv.org/abs/2504.11703>.
 - [177] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36*. Neural Information Processing Systems Foundation, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
 - [178] Dong Shu, Chong Zhang, Mingyu Jin, Zihao Zhou, and Lingyao Li. Attackeval: How to evaluate the effectiveness of jailbreak attacking on large language models. *ACM SIGKDD Explorations Newsletter*, 2025. doi: 10.1145/3748239.3748242. URL <https://doi.org/10.1145/3748239.3748242>.
 - [179] Vincent Siu, Jingxuan He, Kyle Montgomery, Zhun Wang, Neil Gong, Chenguang Wang, and Dawn Song. A framework for formalizing LLM agent security, 2026. URL <https://arxiv.org/abs/2603.19469>.
 - [180] Renan Souza, Timothy Poteet, Brian D. Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. LLM agents for interactive workflow provenance: Reference architecture and evaluation methodology, 2025. URL <https://doi.org/10.1145/3731599.3767582>.
 - [181] Saksham Sahai Srivastava and Haoyu He. Memorygraft: Persistent compromise of LLM agents via poisoned experience retrieval, 2025. URL <https://arxiv.org/abs/2512.16962>.
 - [182] Changzhi Sun, Xiangyu Chen, Jixiang Luo, Dell Zhang, and Xuelong Li. Beyond heuristics: A decision-theoretic framework for agent memory management, 2025. URL <https://arxiv.org/abs/2512.21567>.
 - [183] Libo Sun, Jiwen Zhang, Siyuan Wang, and Zhongyu Wei. MAGNET: Towards adaptive GUI agents with memory-driven knowledge evolution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28181–28206. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1299. URL <https://aclanthology.org/2026.acl-long.1299/>.
 - [184] Balachandra Devarangadi Sunil, Isheetta Sinha, Piyush Maheshwari, Shantanu Todmal, Shreyan Mallik, and Shuchi Mishra. Memory poisoning attack and defense on memory based llm-agents, 2026. URL <https://arxiv.org/abs/2601.05504>.
 - [185] Xuchen Suo. Signed-prompt: A new approach to prevent prompt injection attacks against LLM-integrated applications, 2024. URL <https://arxiv.org/abs/2401.07612>.
 - [186] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. SAGA: A security architecture for governing AI agentic systems. In *Network and Distributed System Security Sympos-*

- sium*, 2026. doi: 10.14722/ndss.2026.230869. URL <https://www.ndss-symposium.org/ndss-paper/saga-a-security-architecture-for-governing-ai-agentic-systems/>.
- [187] Elham Tabassi. Artificial intelligence risk management framework (ai rmf 1.0). (NIST AI 100-1), 2023. doi: 10.6028/NIST.AI.100-1. URL <https://doi.org/10.6028/NIST.AI.100-1>.
 - [188] Hamed Taheri. Governed memory: A production architecture for multi-agent workflows, 2026. URL <https://arxiv.org/abs/2603.17787>.
 - [189] Zhen Tao, Jinxiang Zhao, Peng Liu, Dinghao Xi, Yanfang Chen, Wei Xu, and Zhiyu Li. MemConflict: Evaluating long-term memory systems under memory conflicts, 2026. URL <https://arxiv.org/abs/2605.20926>.
 - [190] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models, 2024. URL <https://arxiv.org/abs/2404.14387>.
 - [191] Yingzhe Tong, Leyu Dai, and Songhui Guo. AID-Guard: Stateful authorization for delegated agent effects, 2026. URL <https://arxiv.org/abs/2608.21159>.
 - [192] Vicenç Torra and Maria Bras-Amorós. Memory poisoning and secure multi-agent systems, 2026. URL <https://arxiv.org/abs/2603.20357>.
 - [193] George Torres, Sharad Shrestha, and Satyajayant Misra. When agents remember too much: Memory poisoning attacks on large language model agents, 2026. URL <https://arxiv.org/abs/2607.06595>.
 - [194] Harold Friedman, Rishi Jha, and Vitaly Shmatikov. Multi-agent systems execute arbitrary malicious code. *arXiv preprint arXiv:2503.12188*, 2025.
 - [195] Isaac Triguero, Daniel Molina, Javier Poyatos, Javier Del Ser, and Francisco Herrera. General purpose artificial intelligence systems (GPAIS): Properties, definition, taxonomy, societal implications and responsible governance. *Information Fusion*, 2023. doi: 10.1016/j.inffus.2023.102135. URL <https://doi.org/10.1016/j.inffus.2023.102135>.
 - [196] Giorgio Volta, Giacomo Longo, and Alessio Merlo. Experimental campaign for political behaviour in LLM agents: Experimental insights on governance, global power asymmetries, security risks, and knowledge construction dynamics. *Zenodo (CERN European Organization for Nuclear Research)*, 2026. doi: 10.5281/zenodo.19052474. URL <https://doi.org/10.5281/zenodo.19052474>.
 - [197] Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling privacy risks in llm agent memory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 25241–25260. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1227. URL <https://aclanthology.org/2025.acl-long.1227/>.
 - [198] Chenxu Wang, Chaozhuo Li, Songyang Liu, Zejian Chen, Jinyu Hou, Ji Qi, Rui Li, Litian Zhang, Qiwei Ye, Zheng Liu, Xu Chen, Xi Zhang, and Philip S. Yu. The devil behind moltbook: Anthropic safety is always vanishing in self-evolving AI societies, 2026. URL <https://arxiv.org/abs/2602.09877>.
 - [199] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
 - [200] Junjian Wang, Lidan Zhao, and Xi Sheryl Zhang. MADRA: Multi-agent debate for risk-aware embodied planning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 6852–6876. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.340. URL <https://aclanthology.org/2026.findings-acl.340/>.
 - [201] Kaixiang Wang, Yidan Lin, Jiong Lou, Zhaojiacheng Zhou, Bunyod Suvonov, and Jie Li. E-mem: Multi-agent based episodic context reconstruction for LLM agent memory, 2026. URL <https://arxiv.org/abs/2601.21714>.
 - [202] Kaixiang Wang, Jiong Lou, Zhaojiacheng Zhou, and Jie Li. Oep: Poisoning self-evolving llm agents via locally correct but non-transferable experiences, 2026. URL <https://arxiv.org/abs/2605.18930>.
 - [203] Kun Wang, Guibin Zhang, Zhenhong Zhou, Jiahao Wu, Miao Yu, Shiqian Zhao, Chenlong Yin, Jinhu Fu, Yibo Yan, Hanjun Luo, et al. A comprehensive survey in llm (-agent) full stack safety: Data, training and deployment. *arXiv preprint arXiv:2504.15585*, 2025.

- [204] Le Wang, Zonghao Ying, Tianyuan Zhang, Siyuan Liang, Shengshan Hu, Mingchuan Zhang, Aishan Liu, and Xianglong Liu. Manipulating multimodal agents via cross-modal prompt injection, 2025. URL <https://doi.org/10.1145/3746027.3755211>.
- [205] Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, Heng Lian, Kunyi Wang, Xiaoming Yu, Jianghao Yin, Sen Hu, Yue Hu, Shaolei Zhang, Yanbing Liu, Ronghao Chen, and Huacan Wang. Memgovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789>.
- [206] Shang Wang, Tianqing Zhu, Bo Liu, Ming Ding, Dayong Ye, Wanlei Zhou, and Philip S. Yu. Unique security and privacy threats of large language models: A comprehensive survey, 2024. URL <https://arxiv.org/abs/2406.07973>.
- [207] Thomas Wang and Haowen Li. Openguardrails: A configurable, unified, and scalable guardrails platform for large language models, 2025. URL <https://arxiv.org/abs/2510.19169>.
- [208] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. Badagent: Inserting and activating backdoor attacks in LLM agents, 2024. URL <https://doi.org/10.18653/v1/2024.acl-long.530>.
- [209] Yiming Wang, Jiahao Chen, Qingming Li, Tong Zhang, Rui Zeng, Xing Yang, and Shouling Ji. AEIOU: A unified defense framework against NSFW prompts in text-to-image models, 2025. URL <https://arxiv.org/abs/2412.18123>.
- [210] Yixu Wang, Xin Wang, Yang Yao, Xinyuan Li, Xibang Yang, Yan Teng, Xingjun Ma, and Yingchun Wang. Agenticval: Toward agentic and self-evolving safety evaluation of large language models. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 14789–14808. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.727. URL <https://aclanthology.org/2026.findings-acl.727/>.
- [211] Yuhao Wang, Shengfang Zhai, Guanghao Jin, Yinpeng Dong, Linyi Yang, and Jiaheng Zhang. Mempot: Defending against memory extraction attack with optimized honeypots, 2026. URL <https://arxiv.org/abs/2602.07517>.
- [212] Yuhui Wang, Tanqiu Jiang, Jiacheng Liang, Charles Fleming, and Ting Wang. MAGE: Safeguarding LLM agents against long-horizon threats via shadow memory, 2026. URL <https://arxiv.org/abs/2605.03228>.
- [213] Zhun Wang, Vincent Siu, Zhe Ye, Tianneng Shi, Yuzhou Nie, Xuandong Zhao, Chenguang Wang, Wenbo Guo, and Dawn Song. AGENTVIGIL: Automatic black-box red-teaming for indirect prompt injection against LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23159–23172, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.1258. URL <https://aclanthology.org/2025.findings-emnlp.1258/>.
- [214] Zhun Wang, Nico Schiller, Hongwei Li, Srijiith Sessa Narayana, Milad Nasr, Nicholas Carlini, Xiangyu Qi, Eric Wallace, Elie Bursztein, Luca Invernizzi, Kurt Thomas, Yan Shoshitaishvili, Wenbo Guo, Jingxuan He, Thorsten Holz, and Dawn Song. ExploitGym: Can AI agents turn security vulnerabilities into real attacks?, May 2026. URL <https://arxiv.org/abs/2605.11086>.
- [215] Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. A-memguard: A proactive defense framework for LLM-based agent memory, 2025. URL <https://arxiv.org/abs/2510.02373>.
- [216] Rongzhe Wei, Peizhi Niu, Xinjie Shen, Tony Tu, Yifan Li, Ruihan Wu, Eli Chien, Pin-Yu Chen, Olgica Milenkovic, and Pan Li. The trojan knowledge: Bypassing commercial LLM guardrails via harmless prompt weaving and adaptive tree search, 2025. URL <https://arxiv.org/abs/2512.01353>.
- [217] Baiqi Wu, Qingming Li, Chunyi Zhou, Ting Wang, and Shouling Ji. A comparative survey of security risks in ai systems: From llms to ai agents and embodied agents. *ACM Computing Surveys*, 58(15):1–38, 2026.
- [218] Xiaodong Wu, Yu Shi, Qi Li, Zhimin Zhao, Xiangman Li, Bram Adams, Ahmed E. Hassan, and Jianbing Ni. EVOMAL: Self-poisoning in self-evolving coding agents, 2026. URL <https://arxiv.org/abs/2608.25776>.
- [219] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023. URL <https://arxiv.org/abs/2309.07864>.

- [220] Boming Xia, Qinghua Lu, Liming Zhu, Zhenchang Xing, Dehai Zhao, and Hao Zhang. Evaluation-driven development and operations of LLM agents: A process model and reference architecture, 2024. URL <https://arxiv.org/abs/2411.13768>.
- [221] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning, 2024. URL <https://arxiv.org/abs/2406.09187>.
- [222] Weiwei Xie, Shaoxiong Guo, Fan Zhang, Tian Xia, Xue Yang, Lizhuang Ma, Junchi Yan, and Qibing Ren. MemEvoBench: Benchmarking safety risks from memory misevolution in LLM agents, 2026. URL <https://arxiv.org/abs/2604.15774>.
- [223] Mingzhe Xing, Rongkai Zhang, Hui Xue, Qi Chen, Fan Yang, and Zhen Xiao. Understanding the weakness of large language model agents within a complex android environment, 2024. URL <https://doi.org/10.1145/3637528.3671650>.
- [224] Wenjun Xiong, Yijin Zhou, Jiaqian Wang, Shangding Gu, Bo Tang, Zhiyu Li, Feiyu Xiong, Ying Wen, and Muning Wen. MAPLE-Guard: Memory-aware link enforcement against memory-link poisoning in multi-agent systems, 2026. URL <https://arxiv.org/abs/2608.00426>.
- [225] Lingxiang Xu, Jiaoyun Yang, Min Hu, Hongtu Chen, and Ning An. When should memory stay silent: Measuring memory-use boundaries in memory-augmented conversational agents, 2026. URL <https://arxiv.org/abs/2606.06055>.
- [226] Naen Xu, Hengyu An, Shuo Shi, Jinghuai Zhang, Chunyi Zhou, Changjiang Li, Tianyu Du, Zhihui Fu, Jun Wang, and Shouling Ji. When agents “Misremember” collectively: Exploring the mandela effect in LLM-based multi-agent systems. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2602.00428. URL <https://openreview.net/forum?id=yIoMqDes70>.
- [227] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents, 2025. URL <http://arxiv.org/abs/2502.12110>.
- [228] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models, 2024. URL <http://arxiv.org/abs/2402.13116>.
- [229] Bingyu Yan, Xiaoming Zhang, Chaozhuo Li, Ziyi Zhou, Yirui Qi, and Litian Zhang. Benign alone, harmful together: Exploiting experience composition in self-evolving LLM agents, 2026. URL <https://arxiv.org/abs/2608.01759>.
- [230] Biwei Yan, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng. On protecting the data privacy of large language models (LLMs) and LLM agents: A literature review. *High-Confidence Computing*, 2025. doi: 10.1016/j.hcc.2025.100300. URL <https://doi.org/10.1016/j.hcc.2025.100300>.
- [231] Hongming Yang, Hao Liu, Xin Yuan, Kai Wu, Wei Ni, J. Andrew Zhang, and Ren Ping Liu. Synergizing intelligence and privacy: A review of integrating internet of things, large language models, and federated learning in advanced networked systems. *Applied Sciences*, 15(12):6587, 2025. doi: 10.3390/app15126587. URL <https://www.mdpi.com/2076-3417/15/12/6587>.
- [232] Tingting Yang, Ping Feng, Qixin Guo, Jindi Zhang, Xiufeng Zhang, Jiahong Ning, Xinghan Wang, and Zhongyang Mao. Autohma-LLM: Efficient task coordination and execution in heterogeneous multi-agent systems using hybrid large language models. *IEEE Transactions on Cognitive Communications and Networking*, 2025. doi: 10.1109/tccn.2025.3528892. URL <https://doi.org/10.1109/tccn.2025.3528892>.
- [233] Wenxian Yang, Hanzheng Qiu, Bangqun Zhang, Chengquan Li, Zhiyong Huang, Xiaobin Feng, Rongshan Yu, and Jiahong Dong. When openclaw meets hospital: Toward an agentic operating system for dynamic clinical workflows, 2026. URL <https://arxiv.org/abs/2603.11721>.
- [234] Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections, 2026. URL <https://arxiv.org/abs/2602.15654>.
- [235] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [236] Chenchen Ye, Ziniu Hu, Yihe Deng, Zijie Huang, Mingyu Derek Ma, Yanqiao Zhu, and Wei Wang. Mirai: Evaluating LLM agents for event forecasting, 2024. URL <http://arxiv.org/abs/2407.01231>.

- [237] Qiming Ye, Peixian Zhang, Yupeng He, Zifan Peng, and Gareth Tyson. Behind EvoMap: Characterizing a self-evolving agent-to-agent collaboration network, 2026. URL <https://arxiv.org/abs/2605.25815>.
- [238] Caglar Yildirim. Differential harm propensity in personalized LLM agents: The curious case of mental health disclosure, 2026. URL <https://arxiv.org/abs/2603.16734>.
- [239] Xingkun Yin and Hongyang Du. GLOVE: Global verifier for LLM memory-environment realignment, 2026. URL <https://arxiv.org/abs/2601.19249>.
- [240] Zonghao Ying, Xiangfan Wu, Huiyu Wu, Xing Zheng, Huangsheng Cheng, Xiaorong Shi, and Jing Guo. SkillJack: Persistent skill backdoors in self-evolving agents, 2026. URL <https://arxiv.org/abs/2608.03509>.
- [241] Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pan, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, et al. A survey on trustworthy llm agents: Threats and countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 6216–6226, 2025.
- [242] Xinyi Yu, Xiang Yin, Shaoyuan Li, and Zhaojian Li. Security-preserving multi-agent coordination for complex temporal logic tasks. *Control Engineering Practice*, 2022. doi: 10.1016/j.conengprac.2022.105130. URL <https://doi.org/10.1016/j.conengprac.2022.105130>.
- [243] Jiayi Yuan, Jonathan Nöther, Natasha Jaques, and Goran Radanović. Agenticrod: Evolving agentic systems for red-teaming, 2026. URL <https://arxiv.org/abs/2601.13518>.
- [244] Lei Yuan, Ziqian Zhang, Ke Xue, Hao Yin, Feng Chen, Cong Guan, Lihe Li, Chao Qian, and Yang Yu. Robust multi-agent coordination via evolutionary generation of auxiliary adversarial attackers. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023. doi: 10.1609/aaai.v37i10.26388. URL <https://doi.org/10.1609/aaai.v37i10.26388>.
- [245] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-judge: Benchmarking safety risk awareness for llm agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1384–1400, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.79. URL <https://aclanthology.org/2024.findings-emnlp.79/>.
- [246] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. ColluSkill: Adversarial cross-skill composition for evading agent skill scanners, 2026. URL <https://arxiv.org/abs/2608.09732>.
- [247] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.624. URL <https://aclanthology.org/2024.findings-acl.624/>.
- [248] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7116–7132, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-naacl.395. URL <https://aclanthology.org/2025.findings-naacl.395/>.
- [249] Qiuyang Zhan, Rui Zhang, Sheng Guo, Lepeng Zhao, and Zhuotao Liu. When memory becomes authority: Benchmarking authority collapse at the memory consolidation boundary, 2026. URL <https://arxiv.org/abs/2608.01679>.
- [250] Boyang Zhang, Yicong Tan, Yun Shen, Ahmed Salem, Michael Backes, Savvas Zannettou, and Yang Zhang. Breaking agents: Compromising autonomous LLM agents through malfunction amplification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 34952–34964. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1771. URL <https://aclanthology.org/2025.emnlp-main.1771/>.
- [251] Ce Zhang, Jinxi He, Junyi He, Katia Sycara, and Yaqi Xie. Evolving contextual safety in multi-modal large language models via inference-time self-reflective memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 41182–41192, 2026. URL https://openaccess.thecvf.com/content/CVPR2026/html/Zhang_Evolving_Contextual_Safety_in_Multi-Modal_Large_Language_Models_via_Inference-Time_CVPR_2026_paper.html.

- [252] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents, 2024. URL <https://arxiv.org/abs/2410.02644>.
- [253] Lingxi Zhang, Guangtao Zheng, and Hanjie Chen. When embedding-based defenses fail: Rethinking safety in LLM-based multi-agent systems, 2026. URL <https://arxiv.org/abs/2605.01133>.
- [254] Wenxiao Zhang, Xiangrui Kong, Conan Dewitt, Thomas Bräunl, and Jin B. Hong. A study on prompt injection attack against LLM-integrated mobile robotic systems. In *2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pages 361–368. IEEE, 2024. doi: 10.1109/issrew63542.2024.00103. URL <https://doi.org/10.1109/issrew63542.2024.00103>.
- [255] Xiao Zhang, Yusheng Wang, Yuhao Fei, Dongyuan Li, Zian Liang, Liuyu Xiang, Hongxun Gu, and Zhaofeng He. Harnesssafe: Evaluating safety across persistent carriers in agent harnesses, 2026. URL <https://arxiv.org/abs/2608.06984>.
- [256] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Library drift: Diagnosing and fixing a silent failure mode in self-evolving LLM skill libraries, 2026. URL <https://arxiv.org/abs/2605.19576>.
- [257] Xinyu Zhang, Huiyu Xu, Zhongjie Ba, Zhibo Wang, Yuan Hong, Jian Liu, Zhan Qin, and Kui Ren. Privacyasst: Safeguarding user privacy in tool-using large language model agents. *IEEE Transactions on Dependable and Secure Computing*, 2024. doi: 10.1109/TDSC.2024.3372777. URL <https://doi.org/10.1109/TDSC.2024.3372777>.
- [258] Xuanye Zhang, Yongsen Zheng, Zhuqin Xu, Kaiyu Zhou, Bowen Shen, Haoran Ou, Tianwei Zhang, and Kwok-Yan Lam. Memmorph: Tool hijacking in llm agents via memory poisoning, 2026. URL <https://arxiv.org/abs/2605.26154>.
- [259] Yao Zhang, Zijian Ma, Yunpu Ma, Zhen Han, Yu Wu, and Volker Tresp. Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. doi: 10.1609/aaai.v39i22.34505. URL <https://doi.org/10.1609/aaai.v39i22.34505>.
- [260] Yechao Zhang, Shiqian Zhao, Jiawen Zhang, Jie Zhang, Gelei Deng, Xiaogeng Liu, Chaowei Xiao, and Tianwei Zhang. When claws remember but do not tell: Stealthy memory injection in persistent personal agents, 2026. URL <https://arxiv.org/abs/2607.05189>.
- [261] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025. doi: 10.1145/3748302. URL <https://doi.org/10.1145/3748302>.
- [262] Zhaoxi Zhang and Xiaomei Zhang. Are you still the agent i authorized? earned authority under a fixed ceiling for evolving agents, 2026. URL <https://arxiv.org/abs/2607.23586>.
- [263] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of llm agents, 2024. URL <https://arxiv.org/abs/2412.14470>.
- [264] Lei Zhao, Abhay Bhaskar, and Edgar Dobriban. LivePI: More realistic benchmarking of agents against indirect prompt injection, 2026. URL <https://arxiv.org/abs/2605.17986>.
- [265] Weixiang Zhao, Yingshuo Wang, Yichen Zhang, Yang Deng, Yanyan Zhao, Wanxiang Che, Bing Qin, and Ting Liu. Large language model agents are not always faithful self-evolvers, 2026. URL <https://arxiv.org/abs/2601.22436>.
- [266] Weixiang Zhao, Yichen Zhang, Yingshuo Wang, Yang Deng, Yanyan Zhao, Xuda Zhi, Yongbo Huang, Hao He, Wanxiang Che, Bing Qin, and Ting Liu. On safety risks in experience-driven self-evolving agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 42145–42169. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.2091. URL <https://aclanthology.org/2026.findings-acl.2091/>.
- [267] Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuandong Tian, and Jishen Zhao. AMA-bench: Evaluating long-horizon memory for agentic applications, 2026. URL <https://arxiv.org/abs/2602.22769>.

- [268] Yihe Zhou, Tao Ni, Wei-Bin Lee, and Qingchuan Zhao. A survey on backdoor threats in large language models (LLMs): Attacks, defenses, and evaluations. *Transactions on Artificial Intelligence*, page 3, 2025. doi: 10.53941/tai.2025.100003. URL <https://www.scittp.com/journals/tai/articles/2505000595>.
- [269] Zhenhong Zhou, Yuanhe Zhang, Hongwei Cai, Moayad Aloqaily, Ouns Bouachir, Linsey Pang, Prakhar Mehrotra, Kun Wang, and Qingsong Wen. MCPShield: A security cognition layer for adaptive trust calibration in model context protocol agents, 2026. URL <https://arxiv.org/abs/2602.14281>.
- [270] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shu Qing Guo, Zhe Wang, Zhihua Wang, Cheng Qian, Robert Tang, Heng Ji, and Jiaxuan You. Multiagentbench : Evaluating the collaboration and competition of LLM agents, 2025. URL <https://doi.org/10.18653/v1/2025.acl-long.421>.
- [271] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23–35. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.eacl-long.2. URL <https://aclanthology.org/2026.eacl-long.2/>.
- [272] Nan Zhuang, Boyu Cao, Yi Yang, Jing Xu, Mingda Xu, Yuxiao Wang, and Qi Liu. LLM agents can be choice-supportive biased evaluators: an empirical study. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 26436–26444, 2025. doi: 10.1609/aaai.v39i25.34843. URL <https://doi.org/10.1609/aaai.v39i25.34843>.